TECHNICAL DOCUMENTATION

AQUIS

Aquifer Query and User Information System

A role-based groundwater monitoring platform for Uttar Pradesh — telemetry ingestion, statistical analysis, trajectory-based ML forecasting, and a station-locked AI assistant.

Authors
Srijan Anand Gupta, Utkarsh Srivastava

Institution
BPIT, Rohini, New Delhi

Table of Contents

1 Project Overview

AQUIS — Aquifer Query and User Information System

A role-based groundwater monitoring platform built on NWDP telemetry and CGWB assessment data, with a pooled XGBoost machine-learning forecasting engine, quantile uncertainty calibration, a station-locked AI assistant, and interactive dashboards.

Authors: Srijan Anand Gupta, Utkarsh Srivastava — BPIT, Rohini, New Delhi

---

Architecture

```
back-end/          Node.js + Express API server
front-end/         Next.js 16 frontend
ml/                Python ML module: pooled XGBoost pipeline + Streamlit analysis dashboard
docs/              Documentation
```

Data flow: NWDP Telemetry API + Assessment Excel files → PostgreSQL → Statistical Analysis + ML → Express APIs → Frontend

The **ML module (`ml/`)** is the forecasting/analytics engine. It consumes a cleaned 6-hourly telemetry archive, trains one **pooled (global) XGBoost** model over 600 Uttar Pradesh groundwater stations, calibrates q05/q50/q95 quantile forecasts into 90% prediction intervals, scans the whole fleet for 30-day moves, and exposes a **4-page Streamlit dashboard** (`ml/app.py`, port 8501).

The forecast surface is a dedicated **trajectory v2** engine (`ml/_trajectory.py`): a genuine **120-step, 6-hourly, 30-day** forecast per station (no interpolation — one real model output per 6 h step), with per-horizon q05/q50/q95 and an **evidence-based confidence label** (HIGH / DIRECTIONAL / LOW). Drivers ahead of the anchor are legitimate forecasts — **Open-Meteo** weather (days 1–16) with climatology beyond, and **CWC 3-day river forecasts** where the district is covered. The Forecast page renders this as a single dark-theme dashboard card (bright q05/q95 band, no dot markers).

A **near-real-time refresh daemon** (`ml/refresh/`) keeps forecast surfaces current: it runs fetch → align → inference → publish on a fixed 6-hourly wall-clock grid (01:00 / 07:00 / 13:00 / 19:00 IST), writes per-station forecasts + `meta.json`, and scores realised readings (verification + drift gating).

> A headless **Flask HTTP API** (`ml/api.py`) runs alongside the Streamlit dashboard — root `/` lists endpoints, `/health`, `/stations`, `/forecast/<slug>`, `/assistant/chat`. The Node gateway can proxy it via `ML_SERVICE_URL` (`http://localhost:5000`).

Scope & data coverage

Uttar Pradesh groundwater monitoring is mostly *manual*, not real-time. This is the single most important constraint on what AQUIS can forecast.

- UP has ~**75 districts**, and groundwater there is monitored through two separate networks on the National Water Data Portal (NWDP/NWIC):
- **Manual** — measured a few times a year by field staff (CGWB ~1,000 UP wells, 4×/yr; UPGW publishes its own "*Ground Water Level (Manual - Quarterly)*" dataset, 1991–2020).
- **Telemetry** — automated 6-hourly Digital Water Level Recorders, a *narrow, recently-deployed slice* of the network.
- **AQUIS only ingests the telemetry feed** (`GWL Telemetry 6-Hourly, UPGW`). That feed wires just **1,353 stations across 34 of UP's ~75 districts**. Districts without telemetry gauges never appear in the archive — verified: NWIC probes return `total=0` for e.g. Allahabad/Prayagraj, Amethi, Amroha, Sambhal, G.B. Nagar even with rename aliases (`ml/data/raw/nwic.py`).

- Therefore, in this repo: **1,353 stations / 34 districts = "everything the telemetry archive has"**, and **600 stations / 29 districts = "the ones still alive in 2026"** (selected by `01_select.py`, ≥8 of 9 Jan–Sep 2026 months with a reading). Neither number is a scope preference — the data decides.
- **Project vs ML scope:** the backend ingests NWDP telemetry countrywide (~7.4M records) + CGWB assessment Excels; the **forecasting** scope is **UP-GWL telemetry only**. A station without a live 2026 telemetry stream is simply not forecast-able by this pipeline.
- More detail + the source links (UPGW portal, CGWB monitoring-station page, CGWB high-frequency-data guidelines, UP GW Year Book) live in `ml/MODEL_CARD.md`.

Setup

1. Prerequisites

- Node.js 18+
- PostgreSQL 14+
- Python 3.11+ (venv/packages below tested on 3.14)

2. Backend

```
cd back-end
cp .env.example .env        # Edit with your database credentials
npm install
npm run migrate             # Apply schema
npm run import-assessments  # Import assessment Excel files
npm run import-telemetry    # Import NWDP telemetry data (large dataset)
npm start                   # Start API at http://localhost:3000
```

3. ML dashboard

```
cd ml
venv/bin/streamlit run app.py     # http://localhost:8501
```

The ML venv lives at `ml/venv` (Python 3.14). Use `ml/venv/bin/python` for all pipelines below.

> The optional **AI assistant** page needs **Ollama**: install `ollama`, then `ollama pull llama3.2:3b` and ensure `ollama serve` is running. The instant-facts panel works even when Ollama is down.

4. Frontend

```
cd front-end
npm install
npm run dev                 # Start Next.js at http://localhost:3000
```

> Note: Express runs on `:3000`; if both run, pick a different port for one of them.

ML module (`ml/`) in detail

Layout

```
ml/
├─ app.py                        # Streamlit dashboard (4 pages: Assistant, Correlation,
│                                #   Forecast, Sources; Verification page exists, unwired)
├─ app_pages/                    # page scripts (assistant, correlation, forecast,
│                                #   data_sources + verification; legacy Overview/Drivers/
│                                #   Stations/Model/Fleet pages removed)
├─ app_charts.py                 # dark-theme Altair helpers for the forecast chart
├─ config.py                     # sources, resource IDs, radii, coverage rules, paths
├─ 00_probe.py … 13_refresh_nwic.py   # classic pooled pipeline
├─ 14_6h_features.py             # causal 6h feature frames (used by 30_traj_datasets)
├─ 16_future_drivers.py          # driver-climatology refresh + future-driver bridge
├─ 18_cwc_river_forecast.py      # CWC 3-day river-forecast fetch (per district)
├─ 20_openmeteo_fetch.py         # Open-Meteo daily weather (365d history + 16d forecast)
├─ 30_traj_datasets.py … 33_traj_reliability.py   # trajectory v2 train/backtest/reliability
├─ _model.py _trajectory.py _assistant.py _utils.py _soil.py _lulc.py  # shared libs
├─ api.py                        # Flask HTTP API (JSON; /health, /stations, /forecast/<slug>, /assistant/chat)
├─ snapshot.py                   # matplotlib PNG export (used by Snapshot tests only)
├─ refresh/                      # refresh pipeline: config, features, inference,
│                                #   model_update, publish, schedule, scheduler, state, verification, CLI
├─ data/
│  ├─ processed/common.parquet   # cleaned 6-hourly GWL archive 2021→2026 (source of truth)
│  ├─ aligned/      table.parquet (daily) + table_6h.parquet (6h grid, ~3M rows)
│  ├─ cfs/          openmeteo_weather_daily.parquet + river_forecast_cwc.parquet
│  ├─ meta/         associations, manifest, probe, station flags, climatology
│  ├─ features_6h/  train/val/test parquet (causal 6h feature frames; built by 14_6h_features)
│  └─ soil/         ISRIC SoilGrids + ISRO LULC (one-off fetch)
├─ models/                      # joblib (pooled) + traj_*.json (trajectory v2)
├─ outputs/                     # correlation, benchmark, honest metrics, fleet, diagnostics
├─ tests/                       # stdlib unittest suite (no pytest dependency)
├─ MODEL_CARD.md                # lifecycle card for the pooled model
├─ STREAMLIT_GUIDE.md           # page-by-page widget-level reference
└─ gate_check.py                # regression gate (frozen baselines + trajectory)
```

Pipeline (run in order, each step from `ml/`)

```
00_probe.py           probe every NWIC resource (archive + live) → meta/probe.json
01_select.py          select GWL anchor stations with full-2026 coverage  → 600 of 1,353
02_fetch_selected.py  fetch each driver for the selected districts (resume-safe)
03_merge_normalize.py plausibility caps → raw/<source>_norm.parquet + manifest
04_align.py           daily station×day table with spatial association (IDW rain, nearest weather)
04b_align_6h.py       6-hourly station×slot table (production-consistent cadence)
05_correlate.py       driver correlation report, recharge-lag curve, charts
06_features.py        feature/target engineering on the 6h grid
06_train.py           pooled XGBoost + Ridge (delta target), writes models/ + model_metadata.json
07_evaluate.py        4-way benchmark (XGB / Ridge / persistence / climatology) + honest re-score
07_soil.py            ISRIC SoilGrids per-station texture (background fetch)
08_lulc.py            ISRO Bhuvan LULC 50K district stats
10_ablate.py          drop-group ablation sweep
11_quantile.py        pooled q05/q50/q95 + empirical coverage calibration
11_fleet.py           whole-fleet 30-day forecast scan + recovery snapshot
12_diagnostics.py     VIF + permutation importance + OAT sensitivity
13_refresh_nwic.py    incremental NWIC refresh (+ optional retrain / deploy sync)
14_6h_features.py     causal 6h feature frames (used by 30_traj_datasets for traj v2)
16_future_drivers.py  driver-climatology refresh + future-driver bridge
18_cwc_river_forecast.py  CWC 3-day river forecast fetch (per district, resume-safe)
20_openmeteo_fetch.py     Open-Meteo daily weather: 365d history + 16d forecast
30_traj_datasets.py       trajectory v2 feature frames (train/val, exact-time targets)
31_train_traj.py          trajectory shared multi-horizon XGBoost (q05/q50/q95)
32_backtest_traj.py       honest non-overlap trajectory backtest + per-horizon calibration
33_traj_reliability.py    reliability bucket rules + evidence weights
```

Data

- **Archive:** `data/processed/common.parquet` — the cleaned 6-hourly telemetry source of truth: **~5.3M records / 1,353 UP stations, 2021-01-01 → 2026-09-05**. Every pipeline step reads it read-only.
- **Refresh:** `13_refresh_nwic.py` pulls *newer-than-current-max* rows per district from the live NWIC 2026 resource, merges (deduped, chronological, dtype-coerced) back into the archive, and can retrain (`--retrain`) or run the

staging check / swap (`--deploy-check` , `--deploy-install`). Run it **manually**: `bash cd ml venv/bin/python -u 13_refresh_nwic.py --dry-run # report new rows venv/bin/python -u 13_refresh_nwic.py --retrain # merge + rerun pipeline to the trained model venv/bin/python 13_refresh_nwic.py --deploy-check # is the loaded build 2026-current?`

- **Cleaning:** telemetry sentinels and `|GWL|>100 m` spikes dropped; per-station 0.5–99.5% quantile trim; stations with a >15 m annual-median regime shift (datum errors) excluded; features are causal (lags only).

Model

One **pooled** model, not one per station: - **Grid + horizon:** 6-hourly track, single **30-day** horizon (`GWL(t+120) − GWL(t)` , delta target). Today's level is the anchor feature — never predicted. - **XGBoost:** `reg:squarederror` , early stopping on the Oct–Dec 2025 validation split; train sampled 1 slot/day for memory. - **Ridge** (linear baseline), **persistence** (0-change), and per-station **day-of-year climatology** are benchmarked in `07_evaluate.py` . - **Quantile uncertainty** (`11_quantile.py`): native `reg:quantileerror` q05/q50/q95 pooled forecasters, empirically calibrated on non-overlapping windows → `quantile_calibration.json` (stride coverage 0.908 ≥ 0.80 target, so `k = 1.0` ; median band ±~1.03 m).

Trajectory v2 (`_trajectory.py`) — the Forecast page model

A separate **direct multi-horizon shared XGBoost** on the 6 h grid: horizon `h ∈ 1..120` is an input feature, each step is a genuine model output (no recursion, no interpolation). Full spec + results: **`docs/ml-trajectory-v2-spec.md`** .

- **Forward drivers (forecasts, never future observations):** Open-Meteo weather → days 1–16 (past-window features only); beyond 16 d the backend falls back to climatology; CWC 3-day river forecasts bridge the near horizon where the district is covered. A missing driver **downgrades** confidence at those horizons.
- **Honest 2026 backtest (full fleet, 73,881 non-overlap windows):** 30-day RMSE **trajectory 2.106 m < production direct-30d 2.132 m < persistence 2.151 m** → trajectory **promoted** (`promote_trajectory = True`). Calibrated to **90% coverage at every horizon** (widening `s ∈ [0.80, 1.28]`).
- **Confidence framework:** per-point label from **weighted evidence** (interval quality, vs-persistence, direction, width, station integrity, driver availability, anchor OOD, stability) — not interval width alone. Anchor = latest observed reading, never predicted; `q05 ≤ q50 ≤ q95` at every step.
- **Forecast page:** single dark-theme card — one "Forecast starts" boundary, observed tail, q50 + a bright q05/q95 band (no dot markers), 6 metric cards (+24h/+7d/+30d/change/confidence), direction banner. The card intentionally references only the trajectory forecast.

Head-to-head (30-day, 2026 held-out, 6h grid)

Model	Level RMSE (m)	Notes
XGBoost (pooled)	**2.242**	beats persistence +2.1% raw
persistence	2.290	
Ridge	2.456	
climatology	3.572	

Honest scale of evidence (overlap-aware, non-trivial): because 6-hourly 30-day windows overlap ~99%, the trustworthy campaign is the **stride** set — **2.339 m vs persistence 2.382 m (+1.8%)** on 3,371 independent windows; effective sample ≈ 6,600 rows (lag-1 ACF 0.86). **Spatial CV** (leave-block-out station retraining) = 1.97 m mean / 1.82 m median — the 2026 temporal holdout, not spatial leakage, is the binding constraint. Feature ablation: rain/weather/calendar add ~+0.03–0.05 m when dropped; river/canal and GWL lags are neutral at 30 d given the anchor.

Verify after editing

```
cd ml
venv/bin/python -m unittest discover -s tests    # 151-unit test suite (AppTest gated via AQUIS_APPTEST=1)
venv/bin/python gate_check.py                    # baseline checks (frozen RMSE/CV/coverage/eff-N/ACF + trajectory;
latest: GATE PASS 10/10)
```

Dashboard (`app.py`, Streamlit, 4 active pages)

Assistant (station-pinned LLM, default page) · Correlation (mode/metric pickers + recharge-lag curve) · **Forecast (single dark-theme trajectory v2 card: 120 genuine 6-hourly points, bright q05/q95 band, confidence, direction)** · Sources (manifest quality, association method, soil/LULC status). Legacy pages (Overview, Drivers, Stations, Model, Fleet) were removed from `app_pages/`; Verification (`app_pages/verification.py`) can be wired in when wanted.

Environment Variables

Variable	Default	Description
DATABASE_URL	—	PostgreSQL connection string
PORT	3000	Backend port
NODE_ENV	development	Environment
ML_SERVICE_URL	http://localhost:5000	Python ML service URL (Flask api.py; proxy via Node gateway)
ML_TIMEOUT_MS	60000	ML request timeout
BACKEND_URL	http://localhost:3000	Data/CSV paths used by some ML helper scripts
AQUIS_OLLAMA_MODEL	llama3.2:3b	Assistant model override
LULC_STATISTICS_API_KEY	—	ISRO Bhuvan LULC API key (Sources page)

API Reference (backend → app)

Full backend/ML endpoint reference lives in **`docs/api.md`**. Summary:

- **Node `:3000`** endpoints: stations, telemetry, assessments (CGWB), trends (Mann-Kendall + Sen's slope), ml-data, data-quality, ingestion.
- **ML Flask `:5000` (`ml/api.py`, live, v3.3.0)** — JSON endpoints: root `/` (lists available routes), `/health`, `/stations` (with district/q filtering, lat/lon per station), `/stations/<slug>` (per-station facts, no LLM), `/stations/<slug>/series` (6-hourly GWL + driver points for relation charts), `/stations/<slug>/alerts` (notification-ready zone + reasons + top drivers, no LLM), `/fleet/alerts` (fleet-wide zones for bell-icon/notification center), `/districts` (district list with water-scarcity status), `/districts/<name>` (district detail + station list), `/fleet/recovery` (recovery/decline ranking), `/forecast/<slug>` (trajectory v2), `/assistant/chat` (Ollama-backed LLM). CORS enabled; the Node gateway can proxy via `ML_SERVICE_URL`.

Testing

```
cd back-end
npm test
```

Backend tests cover classification, statistics, telemetry utilities, ML gateway, and app configuration.

ML validation: stdlib `unittest` suite in `ml/tests/` (151 data-gated tests, no pytest; the full Forecast-page AppTest is gated behind `AQUIS_APPTEST=1` because the trajectory engine is slow) + `ml/gate_check.py` regression gate.

Key Data Facts

- NWDP telemetry: ~7.4M records across India (backend scope); ML archive = **5.3M records / 1,353 UP stations, 6-hourly, 2021-01-01 → 2026-09-05**; **600 stations / 29 districts** quality in as pooled-model anchors. *(Coverage is telemetry-limited — UP GW monitoring is mostly manual/quarterly; see Scope & data coverage.)*
- Assessment years: 2016-2017 … 2025-2026 · 154-column CentralReport Excel (3-level merged headers).
- CGWB classification: Safe (<70%) / Semi-Critical (70–90%) / Critical (90–100%) / Over-Exploited (>100%).

- Forecast uncertainty: **calibrated 90% interval** (q05/q95, `k=1.0`), median half-width ≈1.03 m; 30-day outlooks are **directional**, not exact.
- Forecast surface: the Forecast page shows the **trajectory v2** forecast — **120 genuine 6-hourly steps to +30 d** per station (30-day RMSE 2.106 m, beats both the direct benchmark 2.132 m and persistence 2.151 m on the honest non-overlap 2026 set; calibrated to 90% coverage per horizon). Every point carries an evidence-based confidence label; drivers are Open-Meteo (days 1–16) + CWC river forecasts where available.
- Assistant: Ollama `llama3.2:3b`, station-pinned, grounded in the same data the model consumes.
- Fleet (Sep 2026 scan): **502 stations scored** — median Δ **+0.32 m**, decline 0, recovering 259 at ±0.3 m; no station clears its 90% band (post-monsoon recovery).

Tech Stack

Layer	Stack
Backend	Node.js, Express.js, PostgreSQL
ML Engine	Python, XGBoost, scikit-learn, joblib, scipy
LLM Assistant	Ollama, llama3.2:3b
ML Dashboard	Streamlit, Plotly
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS v4
Charts	Chart.js
Data Sources	NWIC Telemetry API (2021→2026 archive), CGWB Assessment Excel files

2 System Architecture

AQUIS Architecture

System Overview

AQUIS (Aquifer Query and User Information System) is a role-based groundwater monitoring platform with three layers:

1. **Backend (Node.js + Express)** - API server, data ingestion, statistics, ML gateway
2. **ML Service (Python + Flask)** - Machine learning models for forecasting, anomaly detection, risk assessment
3. **Frontend (Next.js)** - User interface for public, officer, and researcher roles

Data Flow

```
RAW DATA
  ↓
INGESTION (telemetry API / assessment Excel)
  ↓
CLEANING + VALIDATION
  ↓
POSTGRESQL (canonical tables)
  ↓
STATISTICAL ANALYSIS (Mann-Kendall, Sen's slope)
  ↓
PYTHON ML (forecasting, anomaly detection, risk)
  ↓
MODEL OUTPUTS → PostgreSQL
  ↓
EXPRESS APIs → Frontend
```

Components

Backend Services

Service	Purpose
`stationService`	Station CRUD, geospatial queries, upserts
`telemetryService`	Observation CRUD, time-series queries
`assessmentService`	Assessment unit + multi-year record management
`telemetryIngestion`	NWDP API paginated ingestion with retry/dedup
`assessmentIngestion`	Excel file parsing, multi-year import
`statisticsService`	Mann-Kendall test, Sen's slope estimator
`dataQualityService`	Quality checks for telemetry and assessment data
`ingestionRunService`	Ingestion run tracking and logging
`mlGateway`	HTTP client for Python ML service
`modelService`	Model metadata and output management
`groundwaterRepository`	Legacy groundwater_data table CRUD
`groundwaterClassification`	CGWB extraction rate classification

Python ML Service

Module	Purpose
`forecast.py`	Linear regression + Random Forest forecasting
`anomaly.py`	Isolation Forest anomaly detection
`risk.py`	Random Forest classifier for deterioration risk
`model_comparison.py`	Model comparison utilities

API Route Groups

Prefix	Purpose
`/stations`	Station listing, details, nearby, summaries
`/telemetry`	Time-series observations
`/assessments`	Multi-year assessment records
`/trends`	Mann-Kendall + Sen's slope analysis
`/ml`	Forecast, anomaly, risk, model metadata
`/ml-data`	Clean ML-ready datasets
`/data-quality`	Data quality reports
`/ingestion`	Trigger and monitor ingestion runs
`/data`	Legacy groundwater_data endpoints
`/analytics`	Legacy analytics (state summary, hotspots)
`/alerts`	Legacy alert endpoints

Communication Pattern

Node ↔ Python ML

```
Express → HTTP POST → Flask ML Service
Express ← JSON response ← Flask ML Service
```

The ML service is independently deployable. Node handles: - Sending clean data - Receiving results - Storing model outputs in PostgreSQL - Serving results to frontend

Python handles: - Model training - Model comparison - Feature engineering - Prediction

3 Data Model

AQUIS Data Model

Tables

stations

Stable telemetry station metadata from NWDP API.

Column	Type	Description
id	SERIAL PK	Internal ID
external_station_id	TEXT UNIQUE	NWDP Station name/key
station_name	TEXT	Human-readable name
agency	TEXT	Operating agency (e.g. UPGW)
state	TEXT	State name
state_lgd_code	TEXT	LGD code
district	TEXT	District name
district_lgd_code	TEXT	LGD code
latitude	NUMERIC(12,8)	GPS latitude
longitude	NUMERIC(12,8)	GPS longitude
first_observed_at	TIMESTAMPTZ	Earliest observation
last_observed_at	TIMESTAMPTZ	Latest observation
observation_count	INTEGER	Total observations

telemetry_observations

Time-series groundwater level readings.

Column	Type	Description
id	SERIAL PK	Internal ID
station_id	INTEGER FK	References stations.id
observed_at	TIMESTAMPTZ	Observation timestamp
groundwater_level	NUMERIC(10,4)	Level in meters
source	TEXT	'nwdp_api'
source_record_id	TEXT	NWDP _id field
ingestion_run_id	INTEGER FK	References ingestion_runs.id

Unique constraint: (station_id, observed_at)

assessment_units

Stable geography for assessment data.

Column	Type	Description
id	SERIAL PK	Internal ID

Column	Type	Description
external_unit_id	TEXT UNIQUE	Composite key: STATE
state	TEXT	State
district	TEXT	District
assessment_unit	TEXT	Assessment unit name

assessment_records

Multi-year assessment data from CentralReport Excel files.

Column	Type	Description
id	SERIAL PK	Internal ID
assessment_unit_id	INTEGER FK	References assessment_units.id
assessment_year	TEXT	e.g. '2024-2025'
rainfall_mm	NUMERIC	Rainfall
recharge_*_ham	NUMERIC	Various recharge components
total_extraction_ham	NUMERIC	Total GW extraction
extraction_rate_pct	NUMERIC	Stage of extraction
status	stress_category	SAFE/SEMI/CRITICAL/OVER
raw_data	JSONB	Original Excel row
source_file	TEXT	Source Excel filename

Unique constraint: (assessment_unit_id, assessment_year)

ingestion_runs

Tracks every ingestion operation.

Column	Type	Description
id	SERIAL PK	Internal ID
source	TEXT	'telemetry_api' or 'assessment_excel'
status	ingestion_status	running/completed/failed
records_seen/inserted/rejected/duplicates	INTEGER	Counters
error_summary	JSONB	Error details

data_quality

Quality issues detected during ingestion or validation.

model_metadata

ML model registry (task, name, version, metrics, features).

model_outputs

Stored predictions (forecasts, anomalies, risk assessments).

groundwater_data

Legacy table preserved for backward compatibility.

Enums

```
stress_category: SAFE, SEMI_CRITICAL, CRITICAL, OVER_EXPLOITED
ingestion_status: running, completed, failed, partial
model_status: training, trained, evaluated, selected, deprecated, failed
```

4 Data Ingestion

AQUIS Ingestion

Telemetry Ingestion

Source

NWDP API: `https://nwdp.nwic.gov.in/api/3/action/datastore_search` Resource ID: `84bfda45-8ead-436d-9c8e-f7a93ee57522` Total records: ~7.4 million

Process

1. Fetch pages from NWDP API (100 records per page)
2. For each record: - Parse and clean all text fields (trim, handle `-`, `NA`, `N/A`) - Parse timestamps (DD-MM-YYYY HH:MM format) - Parse numeric values (groundwater level, coordinates) - Upsert station metadata (deduplicate by external_station_id) - Insert observation with unique constraint (station_id, observed_at)
3. Track in ingestion_runs table
4. Skip duplicates silently (ON CONFLICT DO NOTHING)

Features

- Paginated fetching with offset
- Retry with exponential backoff (3 attempts)
- 30-second request timeout
- Batch processing (100 records per batch)
- Deduplication via unique constraint
- Ingestion run tracking
- Error logging with samples

Commands

```
# Full ingestion
npm run import-telemetry

# Limited ingestion (first 1000 records)
npm run import-telemetry -- --limit 1000

# Custom batch size
npm run import-telemetry -- --batch 200
```

API Trigger

```
curl -X POST http://localhost:3000/ingestion/telemetry \
  -H "Content-Type: application/json" \
  -d '{"batchSize": 100, "limit": 5000}'
```

Assessment Ingestion

Source

Excel files in `back-end/db/centralreport_ingres/`

Available Data

File	Year	States	Records
CentralReport...016614	2025-2026	2 (partial)	75
CentralReport...130368	2024-2025	37	736

File	Year	States	Records
CentralReport...82365	2025-2026	9 (partial)	183
CentralReport...454607	2024-2025	37	736
CentralReport...541485	2023-2024	37	724
CentralReport...580743	2022-2023	37	714
CentralReport...712104	2021-2022	35	706
CentralReport...745929	2019-2020	32	652
CentralReport...770926	2016-2017	29	595

Process

1. Read each Excel file (sheet: "GEC")
2. Parse 3-level merged headers (154 columns)
3. Detect assessment year from data content
4. Extract key metrics (recharge, extraction, classification)
5. Upsert assessment units (deduplicate)
6. Insert/update assessment records (ON CONFLICT DO UPDATE)
7. Store raw_data as JSONB for auditability
8. Track in ingestion_runs

Cleaning Rules

- Trim all text fields
- Convert `-`, `NA`, `N/A`, empty strings to NULL
- Parse numeric values with parseFloat
- Validate: no negative extraction, no negative rainfall
- Derive extraction rate and classification from raw values

Idempotency

- Re-importing the same file updates existing records
- No duplicate unit+year combinations
- Historical years never overwritten

Commands

```
# Import all Excel files
npm run import-assessments

# Import with forced year
npm run import-assessments -- --year 2024-2025

# Import from custom directory
npm run import-assessments -- --dir /path/to/files
```

5 API Reference

AQUIS API Reference

Two HTTP services power the platform:

Service	URL	Status
Node.js API (Express)	`http://localhost:3000`	**Active** — frontend/mobile clients
Python ML service (Flask, headless)	`http://localhost:5000`	**Live** (`ml/api.py`) — trajectory v2 forecast, fleet, assistant

The **frontend talks to the Node API on** `:3000`. ML intelligence (recency-sorted station/district lists, per-station facts, XGBoost forecasts, fleet recovery ranking, the LLM data assistant) is served by a headless Python Flask service (`ml/api.py`) and proxied by the Node backend under `/ml/*`.

- `back-end/services/mlGateway.js` resolves `ML_SERVICE_URL` (default `http://localhost:5000`).
- When the Flask service is down, `/ml/*` gateway routes return `{ "available": false, "error": "ML service unavailable" }` .
- The **live analysis surfaces** are the headless Flask API (`ml/api.py` , port 5000) and the **Streamlit dashboard** (`cd ml && venv/bin/streamlit run app.py` , port 8501). Both read the same artifacts under `ml/outputs/` and `ml/models/` .

> Legacy numeric endpoints (`GET /ml/forecast/:stationId` , `/ml/risk/:unitId` , `/ml/anomalies/...`) are **kept for backward compatibility but deprecated** — they read from the DB model registry, not the live ML stack.

The remainder of this page documents the **live Flask contract** plus the Node endpoints.

Conventions

- **Encoding:** always JSON (`Content-Type: application/json`). Query parameters for GETs.
- **Auth:** none currently — services run on localhost/LAN. CORS open on both services.
- **Errors:** every failure returns `{ "error": "...", "detail": "..." }` with an appropriate status code.
- **Slugs:** ML endpoints identify stations by **slug** — lowercase hyphenated station name, e.g. `ashadha-prathmik-vidyalaya` . Always fetch the current slug from `GET /stations` — never hand-type one. (Fallback: the exact station name also resolves when URL-encoded, e.g. `ASHADHA%20PRATHMIK%20VIDYALAYA` — use `encodeURIComponent` in JS.)
- **Path tolerance:** trailing slashes are ignored (`/health/` == `/health`), and underscore variants route to the same handler (`POST /assistant_chat` == `POST /assistant/chat`). Genuine 404s include a `did_you_mean` hint.

Status codes (ML service)

Code	Meaning
200	OK
400	Missing/invalid request (e.g. no `question` on chat)
404	Unknown slug / no model trained / snapshot missing
500	Internal Python error
502	ML service down or unreachable
503	Assistant unavailable (Ollama not running)

Frontend endpoints (Node `:3000`) — live

1. `GET /ml/health`

Service status + model/Ollama health. Fast, call on app boot.

```
{
  "status": "ok",
  "service": "aquis-ml",
  "version": "3.2.0",
  "stations": 549,
  "dataset_last": "2026-09-10T19:00:00+05:30",
  "ollama": { "server": true, "model": "llama3.2:3b" },
  "forecast_model": "trajectory (120x6h q05/q50/q95)"
}
```

`available:true` in the gateway's `/ml/health` response when Flask is running on `:5000`; see Python service contract below.

2. Stations / telemetry / assessments / trends / ml-data / data-quality / ingestion

All of these are served by the Express backend against PostgreSQL and are **live today**:

```
GET  /stations                          List all stations (paginated)
GET  /stations/:stationId               Station details
GET  /stations/nearby?lat=&lon=         Nearby stations
GET  /stations/state-summary            Station count by state
GET  /stations/district-summary         Station count by district
GET  /telemetry                         All observations (filtered)
GET  /telemetry/latest?stationId=       Latest observation for station
GET  /telemetry/summary                 State/district summary
GET  /telemetry/:stationId              History for station
GET  /assessments                       Assessment records (filtered)
GET  /assessments/:id                   Single record
GET  /assessments/:unitId/history       Multi-year history for unit
GET  /assessments/summary               State/year summary
GET  /assessments/years                 Available assessment years
GET  /trends/:stationId                 Mann-Kendall + Sen's slope for station
GET  /trends/summary                    Trend summary across all stations
GET  /ml-data/telemetry                 Clean telemetry dataset
GET  /ml-data/telemetry/:stationId      Station-specific telemetry
GET  /ml-data/assessment                Clean assessment dataset
GET  /ml-data/assessment/:unitId        Unit-specific assessment history
GET  /data-quality/:stationId           Station quality issues
GET  /data-quality/telemetry            Telemetry quality summary
GET  /data-quality/assessment           Assessment quality summary
GET  /ingestion                         List ingestion runs
GET  /ingestion/:id                     Run details
POST /ingestion/telemetry               Trigger telemetry ingestion
POST /ingestion/assessment              Trigger assessment ingestion
```

Python service contract — live (`ml/api.py`)

The Flask service exposes the surface below; the Node gateway (`back-end/services/mlGateway.js`) forwards path + query string to `ML_SERVICE_URL`.

Service surface (implemented)

```
GET  /                              index: service, version, endpoints, usage
GET  /health                        status, version, stations, dataset_last, ollama, forecast_model
GET  /stations                      ?district=&q=&limit=        recency-sorted station list (slugs + lat/lon)
GET  /stations/<slug>               per-station facts — same rich object as chat "facts", no LLM call
GET  /stations/<slug>/series        6-hourly gwl + driver points for relation charts (?drivers=&from=&to=&limit=)
GET  /stations/<slug>/alerts        notification-ready zone (safe/alert/danger/unknown) + reasons + top drivers (?
n=), no LLM call
GET  /fleet/alerts                  fleet-wide zones in one call (?zone=&district=&sort=&limit=) — bell-icon source
GET  /districts                     district list with water-scarcity status (?sort=&limit=) — scarcity-page source
GET  /districts/<name>              district detail: levels, most-stressed stations, top driver, model accuracy +
station list
GET  /fleet/recovery                recovery/decline ranking (?district=&sort=&limit=)
GET  /forecast/<slug>               trajectory v2 — 120×6h q05/q50/q95 (slug URL-encoded)
POST /assistant/chat                {question, station?, model?}   station-locked LLM answer
```

> **Planned (not yet implemented):** `/models`, `/fleet/forecasts`, `/fleet/scan`.

Series endpoint (relation charts)

`GET /stations/<slug>/series?drivers=rain,temp&from=2026-01-01&to=2026-09-10&limit=2000` returns the aligned 6-hourly table slice for one station — every point carries the observed `gwl` plus the requested drivers, so frontend clients can render per-feature relation charts against GWL (the old Drivers-page overlays) without touching parquet:

- `drivers` — comma list from `rain,temp,river_level,humidity,solar,wind_speed,pressure,canal_level` (default: all); unknown key → `400` with the `available` list.
- `from` / `to` — ISO dates, inclusive; `limit` — default 2000, max 6000, over-limit frames are evenly downsampled (`downsampled: true`). Missing values are `null`.
- Unknown slug → `404`. Example point: `{"time": "2026-09-10T18:00:00", "gwl": -2.84, "rain": 0.0, "temp": 31.5}`.

How the current `ml/` module maps to this contract:

Contract endpoint	Data today (in `ml/`)
`/`	hardcoded service index (service, version, endpoints, usage)
`/health`	`_data()` (aligned 6h table), `ollama_status()`, `_index()` → stations, `forecast_model`
`/stations`	`data/meta/selected_gwl_stations.csv` (549 stations, recency-sorted; `latitude` / `longitude` included per item)
`/stations/<slug>`	`_assistant.StationAssistant().facts()` (same object as chat `"facts"`, no LLM) + `slug` / `latitude` / `longitude` envelope
`/stations/<slug>/series`	aligned 6h table slice (`_data()` filtered by station + date, downsampled to `limit`) — chart-ready gwl+driver points
`/stations/<slug>/alerts`	zone from rule-based `precautions` (action→danger, watch→alert, else safe; fleet-`unreliable`→unknown) + top-`n` drivers by |Spearman| + forecast summary — no LLM, notification-pop ready
`/fleet/alerts`	zones from the published fleet scan (`outputs/fleet_forecast.csv`): `unreliable`→unknown, 30d change ≤−1.0 m→danger, ≤−0.5 m→alert, else safe (+`counts`, `?zone=` / `?district=` / `?sort=` / `?limit=`)
`/districts`	district list with water-scarcity status (`outputs/fleet_district.csv` + history signals): `unknown` if <3 stations or >50% unreliable; `scarce` on broad decline (share≥0.25 / median≤−0.5 m) or deep+corroborated (≥2 wells down ≥2 m); `watch` on any stressed wells; else `healthy` (+`counts`, `?sort=` / `?limit=`)
`/districts/<name>`	district detail: GWL levels, `most_stressed` stations (slug+zone), top driver (`correlation_by_district.csv`), district model accuracy (`eval_by_district.csv`), full station list with scan zones
`/fleet/recovery`	recovery/decline ranking by 30d model change (`?district=`, `?sort=recovery|decline`, `?limit=`) with ±0.15 m direction + band

Contract endpoint	Data today (in `ml/`)
`/forecast/<slug>`	`_trajectory.trajectory_forecast()` (trajectory v2, 120×6h q05/q50/q95)
`/assistant/chat`	`_assistant.py` (station-pinned facts + Ollama)

Quickstarts

Backend (live):

```
curl "http://localhost:3000/stations?limit=5"
curl "http://localhost:3000/telemetry/latest?stationId=<id>"
```

ML Flask API (live):

```
cd ml && venv/bin/python -m api              # http://localhost:5000
curl "http://localhost:5000/"                # service index
curl "http://localhost:5000/health"
curl "http://localhost:5000/stations?district=KANPUR"
curl "http://localhost:5000/stations/ashadha-prathmik-vidyalaya/series?drivers=rain,temp&from=2026-01-01&limit=500"
```

ML dashboard (live):

```
cd ml && venv/bin/streamlit run app.py        # http://localhost:8501
```

Frontend flow via the Node gateway (`/ml/*` proxy): 1. Boot → `GET /ml/health` (EMPTY banner if `available:false`). 2. Station picker → `GET /ml/stations?q=...` (sort by recency; render `last_ts`). 3. Detail view → station KPIs (mirrors `GET /ml/stations/:slug` once implemented). 4. Forecast tab → `GET /ml/forecast/:slug`; chart 120×6h `time` vs `q50` with a `q05`–`q95` band. 5. Fleet view → fleet recovery/scan endpoints once implemented. 6. Assistant → `POST /ml/assistant/chat`.

6 ML Contract

AQUIS ML Contract

Contract between the **Node.js backend** and the **Python ML module** (`ml/`, trajectory v2).

Status

The current `ml/` module includes a **trajectory-v2 model + Streamlit dashboard + headless Flask API** (`ml/api.py` on `:5000`). The Node backend proxies `/ml/*` requests to the Flask service. This document defines:

1. the **data / artifact contract** the current `ml/` produces (live today), and
2. the **HTTP contract** for the Flask API (live).

1. Artifact contract (live)

Everything below lives under `ml/` and is regenerated by the numbered pipeline; `ml/models/` and `ml/outputs/` are git-ignored (reproducible from the archive + pipeline).

`data/processed/common.parquet`

Cleaned 6-hourly NWIC GWL archive — the read-only source of truth: **~5.3M rows / 1,353 UP stations / 2021-01-01 → 2026-09-05**. Columns include `Station`, `District`, `Tehsil`, `Block`, `Latitude`, `Longitude`, `RL_MSL`, `Data Acquisition Time`, `Groundwater Level Telemetry 6 Hourly (meter)`. Refreshed incrementally by `13_refresh_nwic.py`.

`models/` (trained artifacts)

File	Produced by	Contents
`traj_xgb_{q05,q50,q95}.json`, `traj_config.json`	`31_train_traj.py` (offline) / `refresh/model_update.py` (scheduled refit)	trajectory v2 direct multi-horizon quantile XGBoost (120×6h)
`traj_calibration.json`	`32_backtest_traj.py`	per-horizon widening to 90% coverage
`traj_reliability.json`	`33_traj_reliability.py`	bucket rules + evidence weights for confidence
`xgb_multihorizon.joblib`, `linear_multihorizon.joblib`	`06_train.py`	pooled 30-day XGBoost + Ridge (delta target)
`xgb_q{05,50,95}.joblib`	`11_quantile.py`	pooled quantile forecasters
`feature_config.json`	`06_train.py`	model_type, target, horizons, num_cols, stations, districts, split rows, val RMSE, trained_at
`quantile_calibration.json`	`11_quantile.py`	`coverage_stride`, `widen_factor_k`, `half_width_median_m`, `half_width_p90_m`
`model_metadata.json`	`06_train.py`	architecture, split, val RMSE, trained_at + embedded validated metrics (calibration / honest / spatial CV)

`outputs/` (evaluation + fleet)

File	Produced by	Contents
`model_metrics.csv`, `eval_by_station.csv`, `eval_by_district.csv`	`07_evaluate.py`	4-way benchmark on 2026 held-out

File	Produced by	Contents
`honest_metrics.csv`, `overlap.json`, `station_summary.csv`	`validation/overlap.py`	overlap-aware (stride-window) re-score
`spatial_cv_metrics.csv`, `spatial_cv_summary.json`	`validation/spatial_cv.py`	leave-block-out CV
`residual_acf.json`, `residual_spatial.json`	`validation/`	effective sample size, lag-1 ACF, Moran's I / Geary's C
`fleet_forecast.csv`, `fleet_district.csv`, `fleet_forecast_snapshot.json`	`11_fleet.py`	whole-fleet 30-day scan + recovery
`diagnostics.json`, `diagnostics_permutation.csv`	`12_diagnostics.py`	VIF / permutation importance / OAT sensitivity
`correlation_report.csv`, `correlation_by_district.csv`, `lag_curves.csv`	`05_correlate.py`	driver correlation + recharge lag
`ablation.csv`	`10_ablate.py`	drop-group robustness sweep

Baseline numbers (regression gate — `gate_check.py`, latest run GATE PASS 10/10)

- Honest 30-day stride RMSE: **2.339 m** (persistence 2.382 m, +1.8%)
- Spatial CV: mean **1.973 m** / median **1.819 m**
- Effective sample size ≈ **6,635 rows** (lag-1 ACF **0.858**)
- Calibrated q05–q95 coverage: **0.908** (target 0.80) → `k = 1.0`; median half-width **1.025 m**, p90 **2.575 m**
- Trajectory v2 (honest 2026 backtest, 73,881 non-overlap windows): 30-day RMSE **trajectory 2.106 m < direct-30d 2.132 m < persistence 2.151 m** → `promote_trajectory = True`; calibrated to **0.90 coverage at every horizon**
- Fleet scan (Sep 2026): 502 stations scored, median Δ **+0.32 m**, decline 0, recovering 259.

Verification

```
cd ml
venv/bin/python -m unittest discover -s tests   # 151 tests (2 skipped)
venv/bin/python gate_check.py                   # baseline regression checks (GATE PASS 10/10)
```

2. HTTP contract (live — `ml/api.py`, version 3.3.0)

The Flask service runs on `:5000` and exposes:

```
GET  /                              index: service, version, endpoints, usage
GET  /health                        status, version, stations, dataset_last, ollama, forecast_model
GET  /stations                      ?district=&q=&limit=        recency-sorted station list (slugs + lat/lon)
GET  /stations/<slug>               per-station facts — same rich object as chat "facts", no LLM call
GET  /stations/<slug>/series        6-hourly gwl + driver points for relation charts (?drivers=&from=&to=&limit=)
GET  /stations/<slug>/alerts        notification-ready zone + reasons + top drivers (?n=), no LLM call
GET  /fleet/alerts                  fleet-wide zones (?zone=&district=&sort=&limit=) — bell-icon source
GET  /districts                     district list with water-scarcity status (?sort=&limit=)
GET  /districts/<name>              district detail + station list
GET  /fleet/recovery                recovery/decline ranking (?district=&sort=&limit=)
GET  /forecast/<slug>               trajectory v2 — 120×6h q05/q50/q95 (slug URL-encoded)
POST /assistant/chat                {question, station?, model?}   station-locked LLM answer
```

- **URL:** `http://localhost:5000`, configurable via `ML_SERVICE_URL`; gateway forwards path + query string (see `back-end/services/mlGateway.js`).
- **Timeout:** `ML_TIMEOUT_MS` (default 60000).
- **Errors:** JSON `{ "error", "detail" }`; 400 / 404 / 500 / 502 / 503 (see `docs/api.md`).
- **Slugs:** lowercase hyphenated station names (e.g. `ashadha-prathmik-vidyalaya`) — always URL-encode; resolve current slugs from `/stations`. The exact station name also resolves as a fallback.

Planned additions (not yet implemented)

`/models`, `/fleet/forecasts`, `/fleet/scan`.

Behavior notes

- Data sources for each endpoint are already produced by the current artifact contract (table above).
- **No on-demand training:** the Flask service never trains at request time; models are pre-trained artifacts in `ml/models/`.
- **Forecasts:** trajectory v2 serves 120 genuine 6h steps (`/forecast/<slug>`); the pooled direct-30d dict (`_model.forward_forecast`) stays the +30d benchmark. `point` = best estimate; `lower`/`upper` = calibrated 90% interval (`quantile_calibration.json`); only stations in the 600-station anchor set have feature rows.
- **Series:** `/stations/<slug>/series` serves observed history (gwl + drivers) from the aligned 6h table — the chart source for per-feature relation views; never a prediction.
- Assistant requires Ollama (`ollama serve`, `llama3.2:3b`); the instant-facts panel (no LLM) always works.

Service not running?

```
cd ml && venv/bin/streamlit run app.py    # Streamlit dashboard: http://localhost:8501
cd ml && venv/bin/python -m api           # Flask API: http://localhost:5000
```

Legacy (deprecated)

The previous DB-driven Python endpoints — `POST /forecast/:stationId`, `POST /anomalies/:stationId`, `POST /risk/:unitId`, `POST /train`, `POST /models/compare` — are no longer served by the ML module. The Node gateway retains lightweight handlers reading the `model_outputs` table for backward compatibility only.

7 Full-Stack Build Specification

AQUIS Full-Stack Build Specification (ML-Anchored)

> **Purpose:** Single source of truth to build the complete production application (frontend + backend + database + ML integration + chatbot) on top of the **existing** `ml/` module. Every name, JSON shape, function and metric below is taken from the actual code (`ml/*.py`, `back-end/*`, trained artifacts and produced outputs). Where something is **not yet implemented** it is explicitly labelled `[PLANNED]` vs `[LIVE]`.
>
> Related docs: `docs/api.md` (API reference), `docs/ml-contract.md` (module↔gateway contract), `docs/ml-system-spec.md` (ML/deep-dive spec), `ml/MODEL_CARD.md`.

---

1. Project Overview

1.1 Purpose

AQUIS (**A**quifer **Q**uery and **U**ser **I**nformation **S**ystem) is a groundwater monitoring platform for Uttar Pradesh. It ingests **6-hourly Digital Water Level Recorder telemetry** from NWDP/NWIC, cleans it into an archive, trains a **pooled XGBoost 30-day forecasting model** over 600 stations, evaluates drivers (rain, weather, river/canal), runs whole-fleet recovery scans, and explains everything through a **station-locked LLM assistant**.

1.2 Problem statement

UP's groundwater is monitored **mostly manually (quarterly)**. The only real-time slice is a narrow telemetry network (1,353 stations / 34 districts). Officials and researchers have no single tool that (a) serves live levels, (b) forecasts 30-day change with honest uncertainty, (c) ranks declines/recoveries, and (d) answers plain-language questions — without inventing numbers.

1.3 Target users

User	Need
Officer (UPGW/CGWB)	district-level decline alerts, fleet movers, forecast with confidence bands
Researcher	driver correlation, feature importance, honest validation numbers
Public/student	plain-language assistant, station detail, trend charts
Developer/integration	robust REST/JSON contract to build apps on top of

1.4 End-to-end system flow

```
NWIC/NWDP CKAN (datastore_search)                      ← external, free, no auth
   │  00_probe → 01_select → 02_fetch → 03_normalize
   ▼
ml/data/processed/common.parquet  (5.3M rows / 1,353 st / 2021→2026-09-05)
   │  04/04b align (IDW rain, nearest weather, 6h grid)
   ▼
ml/data/aligned/table_6h.parquet  (2.99M rows / 600 st)
   │  06_features (spike-mask, regime-shift, features+target)
   ▼
ml/data/features/{train,val,test}.parquet
   │  06_train (XGBoost+Ridge) · 11_quantile (q05/q50/q95)
   ▼
ml/models/*.joblib + feature_config.json + quantile_calibration.json + model_metadata.json
   │  07_evaluate → validation/* → 11_fleet → 12_diagnostics
   ▼
ml/outputs/{predictions_2026.parquet, fleet_forecast.csv, model_metrics.csv, …}

   ┌──────────────┬──────────────────────────────┐
   ▼              ▼                              ▼
Streamlit app   Express API (:3000) [LIVE]    Flask ML API (:5000) [LIVE v3.3.0]
(ml/app.py,     /stations /telemetry           /stations /forecast
:8501)          /assessments /trends            /assistant/chat …
4 pages         /ml*/ml-data /data-quality      proxied by back-end/mlGateway.js
                /ingestion                       under /ml/live/*
   ▼
Next.js frontend (front-end/) / mobile app
```

1.5 ML module's role

One Python module (`ml/`) that is: the **data pipeline** (00–13), the **forecasting engine** (pooled XGBoost + quantile calibration), the **analytics** (correlation, importance, diagnostics), the **fleet scanner** (`11_fleet.py`), the **assistant** (`_assistant.py`), and a **Streamlit dashboard** (`app.py`, the current live analysis surface).

1.6 Streamlit app features (currently live) — `ml/app.py`

4 pages (`st.navigation`) + Verification (exists, not in nav):

Page	What it shows
Assistant	station-locked chat (Ollama llama3.2:3b); instant facts panel works without LLM
Correlation	mode `raw/deseason/diff` × metric `spearman/pearson` table; recharge-lag curve (GWL vs rainfall); static features (soil/LULC) vs GWL
Forecast	single dark-theme trajectory v2 card: 120 genuine 6-hourly points, observed tail, "Forecast starts" boundary, bright q05/q95 band, direction banner, 6 metric cards, freshness caption
Sources	manifest quality, empty sources, static hydrogeology, LULC/soil status, association method
Verification *(not in nav)*	realised-forecast quality from the refresh pipeline's verification summary + sign-accuracy ledger

> The legacy explorer pages (Overview, Drivers, Stations, Model, Fleet) were removed from `app_pages/`; their pipeline outputs (fleet scan, benchmark, honest validation, ablation, diagnostics) are still produced by the numbered scripts and served via `_model.py` loaders, the Flask API and the assistant.

---

2. Complete Feature List

#	Feature	Purpose	User input	System process	Output	ML needed	Frontend display
F1	Station picker / watchlist	find a station to inspect	text/select (station or district)	`station_recency()` sort (table_6h groupby max)	recency-ordered `[station, district, last_ts]`	no (data only)	dropdown + sortable table + last-updated badge
F2	Station detail KPIs	latest level & history stats	station	`_assistant._series_stats()` (clean, then last/min/max/span/ n_obs/outliers/ change_7/30/60/180d)	facts dict (§10.4)	no	5 KPI cards + caption
F3	GWL trend chart	see level over time	station, interval	`load_table_6h()` slice	date×gwl series	no	line/area chart (price-style)
F4	2026 backtest curve	validate model vs actual	station, model (xgb/ ridge)	`load_predictions()` fiter + melt to long	date×{target, xgb/ ridge, band}	yes (inference on saved preds)	line chart + shaded q05/ q95 error-band
F5	30-day forward forecast	predict future level	station	`forward_forecast(station)` (rebuild features, score xgb/ ridge/q05/q50/q95)	forecast dict (§7.3)	**yes**	KPI table + anchor→+30d segment chart
F6	Confidence interval	show uncertainty	(auto)	`forward_forecast` q05/q95 → `band_half=(q95−q05)/2`	band_half ±, interval	**yes**	shaded band on chart; "±x.xx m" KPI
F7	Fleet score scan	rank all stations	none (snapshot)	`11_fleet.py` → fleet_forecast.csv	stations×Δ/category	**yes**	6 KPI cards + table + histogram
F8	Significant movers	stations moving faster than noise	none	`band_half.notna() & |Δ| >band_half`	movers table	yes	warning banner + table
F9	District rollup	district-level comparison	district	`fleet_district.csv` aggregation	n, n_decline, n_recover, median Δ	yes (from snapshot)	table + heat tiles
F10	Station scan filters	drill into fleet	category multi-select, district, movers-only	dataframe filters	filtered fleet table	no	filter widgets + table
F11	Driver correlation	rank environmental drivers	mode, metric	`correlation_report.csv`	driver×corr×stations_ok	no (stats)	bar chart + table
F12	Recharge-lag analysis	how many days before rain recharges GWL	(auto)	`lag_curves.csv`	lag(0–30d)×pearson median	no	line chart (peak ≈22 d)
F13	Driver overlay	compare one driver vs GWL	station + driver	`load_table()` slice	date×{gwl, driver}	no	dual-axis chart
F14	Model benchmark	compare model quality	metric basis (level/ delta)	`model_metrics.csv`	4-model table	no (eval data)	RMSE bars + table

#	Feature	Purpose	User input	System process	Output	ML needed	Frontend display
F15	Honest validation	see trustworthy numbers	(auto)	honest_metrics.csv, overlap.json, spatial_cv, residual_acf, residual_spatial	stride RMSE, eff-N, Moran's I …	no (eval data)	expander + metrics
F16	Diagnostics	collinearity/ importance/ sensitivity	(auto)	`diagnostics.json` + permutation csv	VIF, permutation, OAT	no (eval data)	charts + metrics
F17	Feature importance	which inputs matter	gain vs | coef|	`feature_importance.csv` / `feature_coefficients.csv`	ranked features	no (eval data)	horizontal bar chart
F18	Station assistant chat	plain-language Q&A	station + question (+history)	`StationAssistant.answer()` (facts→prompt→Ollama)	`{answer, facts, station}`	**yes (LLM + model)**	chat UI + facts panel
F19	Sources/ provenance	data quality per source	(auto)	`manifest.json`, config SOURCES	rows/st/dropped per source	no	tables + captions
F20	Soil/LULC context	static hydrogeology	station	`_soil.load_soil()`, `_lulc.load_lulc()`	soil texture %, LULC class top-4 %	no (gated)	expander caption

---

3. Input Parameters

3.1 Model feature inputs (per 6h row — produced by `06_features.build`)

Columns in `KEEP` in `06_features.py`: `Station, District, time, date, horizon, target, target_d, st_id, dist_id` + 30 floats.

Param	Meaning	dtype	Unit	Range	Req	Example	Source
`gwl`	current GWL (anchor)	float32	m	per-station trimmed (−120…70)	yes	−6.411	telemetry
`lag1`	GWL 1 slot ago	float32	m	same	yes*	−6.5	calc (shift1)
`lag4`	GWL 1 day ago	float32	m	same	yes*	−6.2	calc (shift4)
`lag8`	GWL 2 days ago	float32	m	same	yes*	−6.3	calc
`lag28`	GWL 7 days ago	float32	m	same	yes*	−6.0	calc
`lag120`	GWL 30 days ago	float32	m	same	yes*	−5.8	calc
`gwl_roll7_mean`	7d rolling mean	float32	m	same	yes*	−6.2	calc
`gwl_roll7_std`	7d rolling std	float32	m	≥0	yes*	0.35	calc
`gwl_roll30_mean`	30d rolling mean	float32	m	same	yes*	−6.0	calc
`gwl_roll30_std`	30d rolling std	float32	m	≥0	yes*	0.6	calc
`rain_1d`	rain sum 1d	float32	mm	0…150×4	yes*	0.0	IDW assoc
`rain_7d`	rain sum 7d	float32	mm	0…	yes*	42.0	calc
`rain_30d`	rain sum 30d	float32	mm	0…	yes*	180.0	calc
`temp`	air temp	float32	°C	−10…55	yes*	31.5	AWMS
`temp_7d`	7d mean temp	float32	°C	−10…55	yes*	30.0	calc

Param	Meaning	dtype	Unit	Range	Req	Example	Source
`humidity`	rel humidity	float32	%	0…100	yes*	68.0	AWMS
`humidity_7d`	7d mean humidity	float32	%	0…100	yes*	70.0	calc
`solar`	solar radiation	float32	auto	auto	yes*	350.0	AWMS
`wind_speed`	wind speed	float32	auto	auto	yes*	4.2	AWMS
`pressure`	atm pressure	float32	hPa	auto	yes*	1005.0	AWMS
`river_level`	river water level	float32	m	auto; sparse (2 dist)	yes*	NaN	district proxy
`canal_level`	canal water level	float32	m	auto; 17 st/1 dist	yes*	NaN	nearest gauge
`year`	calendar year	float32	yr	2021…2026	yes	2026	time
`month_sin/cos`	cyclic month	float32	—	−1…1	yes	−0.5	calc
`doy_sin/cos`	cyclic day-of-year	float32	—	−1…1	yes	−0.9	calc
`hour_sin/cos`	cyclic hour	float32	—	−1…1	yes	0.0	calc
`monsoon`	doy − 152	float32	day	−152…213	yes	90.0	calc
`st_id`	station ordinal	int32	—	0…548	yes (XGB only)	0	ordinal encode
`dist_id`	district ordinal	int32	—	0…28	yes (XGB only)	6	ordinal encode
`Station`	station name	str	—	feature_config.stations	yes (Ridge)	"Agra City (UP-017)"	meta
`District`	district name	str	—	feature_config.districts	yes (Ridge)	"KAUSHAMBI"	meta

`*` = may be NaN (XGBoost handles; Ridge median-imputes). Missing driver columns are simply absent → feature builder skips that window group.

3.2 Runtime / API input params

Param	Meaning	dtype	Valid	Req	Example	Source
`station`	station identity	str	any station in `feature_config.stations`	yes (F5/ F18)	"Ramchhitoni Sahawar (UP-011)"	user/DB
`station_slug`	slug form (§12.2)	str	from /stations	alt for chat	"ashadha-prathmik-vidyalaya"	API
`district`	filter	str	29 districts	opt	"KAUSHAMBI"	user
`model` (page)	model choice	str	`xgboost` | `ridge`	opt (def xgb)	"xgboost"	user
`question`	chat text	str	non-empty	yes (chat)	"Is level rising?"	user
`history`	last chat turns	list[dict]	role	content	opt	`[{"role":"user","content":…}]`
`days`	forecast horizon	int	7–90 (default 30)	opt	30	user

Param	Meaning	dtype	Valid	Req	Example	Source
`limit/offset`	pagination	int	≥0	opt	50/0	UI
`q`	search text	str	—	opt	"Agra"	UI

---

4. Dataset & Data Schema

4.1 Sources

All via NWIC/NWDP CKAN `datastore_search` (free, no auth, no key). Resource IDs and value hints are in `ml/config.py` `SOURCES`. Full list in §6.5 of `docs/ml-system-spec.md`.

4.2 `common.parquet` (source of truth) — `[LIVE]`

`data/processed/common.parquet`, ~5.3M rows / 1,353 UP stations / 2021-01-01 → 2026-09-05. Columns: `Station (str), District (str), Tehsil (str, often "-"), Block (str), Latitude (float), Longitude (float), RL_MSL (float, 72–400 m), Data Acquisition Time (timestamp 6h), Groundwater Level Telemetry 6 Hourly (meter) (float, negative = depth below ground; 0 m = ground level; water table amsl = RL_MSL + gwl)`.

4.3 `table_6h.parquet` (model-ready aligned) — `[LIVE]`

`data/aligned/table_6h.parquet`, 2,990,000+ rows / 600 stations. Columns: `Station, District, time (6h slot), gwl, rain, temp, humidity, solar, wind_speed, wind_direction, pressure, river_level, canal_level` (+ `dist_km` where spatial association applied).

4.4 `table.parquet` (daily) — `[LIVE]`

`data/aligned/table.parquet`, daily station×day for correlation/overview.

4.5 Feature tables — `[LIVE]`

`data/features/{train,val,test}.parquet` — columns per §3.1; `target = GWL(t+120)`, `target_d = target − gwl`, `horizon = 30`, `st_id/dist_id` int32.

4.6 `predictions_2026.parquet` — `[LIVE]` (19 cols)

Col	dtype	Meaning
`Station`	str	station
`District`	str	district
`date`	datetime64[us]	day
`time`	datetime64[us]	6h slot
`horizon`	int64	30
`target`	float64	actual GWL(t+120)
`gwl`	float32	anchor
`feat_days`	int8	feature-history days used
`xgb`	float32	XGBoost level pred
`ridge`	float64	Ridge level pred
`persist`	float32	persistence
`clim`	float64	climatology
`err_xgb`	float64	xgb level error
`err_ridge`	float64	ridge level error
`q05_lvl`	float32	calibrated lower

Col	dtype	Meaning
`q95_lvl`	float32	calibrated upper
`step_idx`	int64	index in 120-step window
`window_id`	int64	non-overlap window id
`stride`	bool	True = independent row

4.7 Column-by-column handling rules

Concern	Policy (actual code)
Missing values	kept as NaN in features; XGBoost tolerates, Ridge `SimpleImputer(strategy="median")`
Duplicates	`13_refresh_nwic.py` merges deduped/chronological/dtype-coerced; telemetry unique `(station_id, observed_at)` in DB
Outliers	sentinel `|GWL|>500` dropped; per-station 0.5–99.5% quantile trim; `drop_gwl_spikes` robust MAD z=30; rainfall cap 150 mm/h
Regime shifts	`flag_regime_shift`: per-year median jump >15 m from 2021-24 → station dropped wholesale
Cleaning	daily + 6h medians; water table amsl = RL_MSL + gwl for sanity (corr 0.965)
Split	train <2025-10-01, val Oct–Dec 2025, test ≥2026-01-01; train 1,629,574 / val 11,647 / test 378,554
Sample rows	see §19

---

5. Location Structure

```
India → Uttar Pradesh (UP)
          └── 34 districts in telemetry archive (1,353 stations)
                └── 29 districts "live 2026" → 600 anchor stations
                      ├── Tehsil / Block (published as "-" for most telemetry stations)
                      ├── station: name, lat, lon, RL_MSL
```

- **Registry:** `data/meta/selected_gwl_stations.csv` columns: `Station, District, Tehsil, Block, Latitude, Longitude, n_2026_months, n_2026_records, first_2026, last_record, full26`.
- **Districts:** `data/meta/selected_districts.json` (29, alphabetical; see §4.7 of `docs/ml-system-spec.md`).
- **Model IDs:** `st_id` (sorted station names, 0..548) and `dist_id` (sorted districts, 0..28) — shared across train/val/test + app.
- **API IDs:** lowercase hyphenated slugs (`ashadha-prathmik-vidyalaya`), URL-encoded.
- **Backend:** `stations` table holds same geo fields + `state_lgd_code`, `district_lgd_code` + `rl_msl`.
- **Geo requirements:** lat/lon are **optional for ML** (drivers use spatial association; a station without coords still forecasts — anchored on its own GWL series). Required only for driver association/district proxies.
- **Coverage:** only the 29 districts have data; others never appear (probes return total=0, verified for e.g. Prayagraj, Amethi, Sambhal, G.B. Nagar).

---

6. ML Models

6.1 XGBoost pooled 30-day forecaster — `[LIVE]`

Field	Value
Model name	`xgb_multihorizon.joblib` (XGBRegressor via `joblib`)
Algorithm	Gradient boosting, `reg:squarederror`

Field	Value
Purpose	30-day delta `GWL(t+120)−GWL(t)` for any of 549 stations
Input features	30 `num_cols` + `st_id` + `dist_id` (32 cols)
Target	`target_d` (delta); level = anchor + delta
Training data	`data/features/train.parquet`, sampled 1 slot/day per station
Preprocessing	SHIFT lags, rolling mean/std (min_periods=1), rain sums, cyclic sin/cos, monsoon
Scaling/encoding	none for trees; ordinals `st_id/dist_id`
Hyperparameters	`n_estimators=1500, max_depth=6, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8, min_child_weight=20, tree_method=hist, eval_metric=rmse, random_state=42, n_jobs=6, early_stopping_rounds=60` → best_iteration **15**
Training process	`06_train.py` (early stop on val Oct–Dec 2025)
Evaluation	val delta RMSE **1.2748** m; 2026 test level RMSE **2.242** m; stride RMSE **2.339** m
Performance	beats persistence raw +2.1%, stride +1.8%; M±0.5 m 67.2%, ±1.0 m 85.1%
Limitations	near-agnostic to single drivers at 30d; region/2026 holdout is binding; do not trust per-row deltas < ±0.05 m
Saved file	`ml/models/xgb_multihorizon.joblib`
Load function	`_model.load_xgb_model()`
Predict function	`_model.forward_forecast(station)` (dict)

6.2 Ridge linear baseline — `[LIVE]`

Field	Value
Model name	`linear_multihorizon.joblib` (sklearn `Pipeline`)
Algorithm	Ridge regression (delta target)
Purpose	linear baseline + interpretable coefficients
Input features	30 `num_cols` + `Station` + `District` (categorical OHE)
Preprocessing	numeric: `SimpleImputer(median)` → `StandardScaler`; cat: `OneHotEncoder(handle_unknown="ignore")`; ridge `RidgeCV(alphas=(0.1,1,10,100))` → alpha **0.1**
Evaluation	val delta RMSE 1.708 m; test level RMSE 2.456 m
Extra artifact	`feature_coefficients.csv` (top
Load	`_model.load_linear_model()`

6.3 Quantile models (uncertainty) — `[LIVE]`

Field	Value
Files	`xgb_q05.joblib`, `xgb_q50.joblib`, `xgb_q95.joblib`
Algorithm	pooled XGBoost `reg:quantileerror` at α=0.05/0.50/0.95
Purpose	calibrated 90% prediction interval on level
Calibration	`quantile_calibration.json`: `target_coverage_stride=0.8`, `widen_factor_k=1.0`, `coverage_stride=0.908`, `half_width_median_m=1.025`, `half_width_p90_m=2.575`
Anchor basis	`level = today's GWL + quantile(delta 30d)`
Load	`_model.load_quantile_models()` → dict `{q05,q50,q95}`

6.4 Baselines (not trained, computed) — `[LIVE]`

- **Persistence:** 0-change (carry anchor). Test level RMSE 2.290 m.
- **Climatology:** per-station day-of-year mean. Test RMSE 3.572 m.
- Benchmarked in `07_evaluate.py` → `model_metrics.csv`, `honest_metrics.csv`.

Note: the `model_metadata` / `model_outputs` tables in the backend DB are a **legacy DB registry** — the live stack uses `ml/models/*.json`. All `/ml/*` legacy endpoints (`forecast/:stationId`, `risk/:unitId`, `anomalies`) are **deprecated**.

---

7. Prediction System

7.1 Trigger

`[LIVE]` Streamlit: page load → `forward_forecast(station)`. `[LIVE]` API: `GET /ml/live/forecast/:slug` → `mlGateway.getLiveForecast` → Flask `GET /forecast/:slug` → `_trajectory.trajectory_forecast(station)` (120×6h trajectory v2).

7.2 Required inputs

`station` name (or slug). Everything else is **derived** from `table_6h.parquet` (no user-side feature building).

7.3 Processing pipeline (actual code — `_model.forward_forecast`)

1. `t = load_table_6h()[Station==station]`
2. `t = 06_features.drop_gwl_spikes(t)`
3. `feats = 06_features.build_full(t, keep_na=True)`; `last = feats.sort_values("time").tail(1)`
4. `pred_xgb = xgb_b.predict(last[fnames])` where `fnames = model.feature_names_in_ or cfg["num_cols"]`
5. `pred_ridge = linear_b.predict(last[cfg["num_cols"]+["Station","District"]])`
6. `anchor = last["gwl"]`; `qlev = anchor + quantile_predictions`; calibration `k`
7. `band_half = (q95−q05)/2`; `station_stride_rmse` from `station_summary.csv`
8. Returns dict or `{}` if no feature frame.

7.4 Exact output

```
{
  "station": "Ramchhitoni Sahawar (UP-011)",
  "date_from": "2026-09-05 00:00:00",
  "date_to": "2026-10-05 00:00:00",
  "anchor": -2.841,
  "pred_xgb": 0.121, "pred_ridge": -0.58,
  "xgb_level": -2.72, "ridge_level": -3.421,
  "q05_level": -4.266, "q50_level": -2.691, "q95_level": -1.039,
  "widen_k": 1.0, "band_half": 1.613, "station_stride_rmse": 0.314
}
```

All levels in **metres** (float). `anchor/pred_*` in m, `band_half` in m.

7.5 Categories / confidence

No hard categories in the prediction itself. Uncertainty = calibrated 90% interval (`coverage_stride=0.908`). Fleet scan adds categories (`decline (high)/decline/stable/recovering/unknown/unreliable`) — §9.

7.6 Example input → output

`forward_forecast("Ramchhitoni Sahawar (UP-011)")` → §7.4 JSON (real live output).

---

8. Forecasting

8.1 What is forecast

30-day change `GWL(t+120) − GWL(t)` on the 6-hourly grid; level = anchor + change. Not an event forecast — a **directional water-level outlook** with interval.

8.2 Historical data requirement

For a forward forecast: the station's own series inside `table_6h.parquet` (any length ≥1 row works; more history = better). Fleet eligibility: ≥2000 obs and ≥730-day span. For campaign-level claims, use the 2026 test / stride subset.

8.3 Forecasting algorithm/model

Pooled XGBoost (§6.1) + quantile (§6.3). Persistence = anchor (benchmark).

8.4 Required parameters

`station`. Optional (planned API): `days` 7–90 (single-horizon model — the "days" param selects the interval built from the anchor; the model itself is fixed at 30 d).

8.5 Time granularity

Input grid 6-hourly (00/06/12/18 UTC-ish native). Forecast horizon **30 days (120 steps)** — that is the min/max for a true model prediction.

8.6 Output format

Dict (§7.4). Chart-ready series in the app: anchor date → +30 d segments for XGBoost/Ridge/Persistence.

8.7 Chart data (verified real)


```
[{"date":"2026-09-05 00:00:00","XGBoost":-2.841,"Ridge":-2.841,"Persistence":-2.841},
 {"date":"2026-10-05 00:00:00","XGBoost":-2.72,"Ridge":-3.421,"Persistence":-2.841}]
```


8.8 Example forecast response

§7.4 (that IS the forecast). Fleet-level: `fleet_forecast.csv` row:


```
{"station":"Mahgaon (UP-031)","district":"KAUSHAMBI","anchor":-17.171,
 "date_from":"2026-09-03 18:00:00","age_days":6,"anchor_valid":true,
 "xgb_level":-16.881,"q05_level":-18.637,"q50_level":-16.689,"q95_level":-11.989,
 "change_30d_m":0.482,"change_pooled_m":0.29,"band_half_m":3.324,"plausible":true,"category":"recovering"}
```


---

9. Groundwater Impact / Analysis

9.1 Parameters affecting GWL (+ sign for shallow-rising)

Driver	raw |Spearman|	sign	mode best	meaning
river_level	0.409	+	raw 0.409 / desean 0.168	rising river ↔ rising GWL
pressure	0.199	+	desean 0.205	high pressure → recharge
rain_30d	0.199	− (raw −0.199)	diff +0.013	recent rain raises level (lag ≈22 d)
canal_level	0.146	− (raw −0.146)	desean −0.104	canal-fed influence
humidity	0.085/0.158	+raw/−des	desean −0.158	mixed
wind_speed	0.095	−	raw −0.095	dry conditions
temp	0.041	−	desean +0.092	evap demand
solar	0.055	−	desean −0.073	evap

9.2 Feature importance (XGBoost gain) — `[LIVE]`

Top: `monsoon 25381`, `gwl_roll7_std 21670`, `doy_sin 14347`, `gwl_roll30_std 10718`, `year 10004`, `doy_cos 7990`, `lag1 7534`, `gwl 7330`. (Full table in `docs/ml-system-spec.md` §2.)

9.3 Ridge coefficients — `[LIVE]`

`feature_coefficients.csv`: `num__gwl -9.73`, `num__lag4 -0.35`, `num__lag8 -0.30`, `num__lag1 +0.26`.

9.4 Recharge lag — `[LIVE]`

Best GWL↔rainfall lag ≈ **22 days** (Pearson median +0.136, ≥180 valid days).

9.5 Ablation & diagnostics — `[LIVE]`

Dropping rain/weather/calendar hurts +0.03–0.05 m; river/canal & GWL lags neutral at 30d. No VIF>10. Permutation ≈0 for all. OAT max swing: `doy_cos` 0.385 m, `gwl` 0.148, `gwl_roll30_std` 0.146, `monsoon` 0.132.

9.6 SHAP/explainability

Not implemented (`no shap dependency`). Recommended follow-up: `shap.TreeExplainer` on the stride subset. Today's explainability = gain + coef + permutation + OAT + Spearman.

9.7 Exposure to frontend `[PLANNED API]`

`GET /ml/models` (+`?analysis=`) returns importance/coef/diagnostics; station page embeds driver correlation; assistant phrases these facts.

---

10. Chatbot

10.1 Purpose

Station-locked plain-language Q&A that **phrases precomputed facts only** (no invented numbers).

10.2 Architecture `[LIVE]`

```
User question + station
   → StationAssistant.answer(question, station, history)
       → facts = StationAssistant.facts(station)     (deterministic, from table_6h + model)
       → prompt = _build_prompt(question, facts, history)
       → answer = _invoke_llm(prompt)               (ollama.Client chat, temperature 0.2)
       → return {"answer", "facts", "station"}
```

10.3 Supported questions (examples)

"latest level", "trend over 30 days", "what's the 30-day outlook?", "how many observations?", "district median?", "is it declining?"

10.4 Forecast data how it reaches chatbot

`facts() → _forecast_summary(station)` calls `_model.forward_forecast(station)` and computes `day30_pred, change_30d_pred, direction, plausible, band_half, q05/q95, station_stride_rmse, high_uncertainty`.

10.5 Context/data passed

facts dict (§10.4 of this doc = `_assistant._series_stats` + district median + forecast), plus last 6 chat turns as conversation context.

10.6 LLM / model

`Ollama`, model default `llama3.2:3b`, override `AQUIS_OLLAMA_MODEL` (repo `.env` or env). `ollama_status()` probes server + pulled models.

10.7 Prompt / system prompt

Verbatim template in `_assistant._build_prompt`:

```
You are the AQUIS groundwater assistant. Answer the user's question using
ONLY the given observed facts. No speculation. Be concise (max ~6 lines).
Use metres (m) for levels/changes. State dates where known.
Never refuse or say data is unavailable — use the station/district names …
[CONVERSATION (context only) …, always trust the LATEST FACTS]
FACTS:
station: … district: …
latest level: <last> m on <last_date>
range: <min> to <max> m (span <span> m, <n_obs> observations)
[note: N outliers excluded]
level change: 7d=… | 30d=… | 60d=… | 180d=… m
model forecast (+30 d): level=<day30> m, change 30d=<chg> m (<direction>); 90% band +/- <band_half> m (anchor <anchor>
m); [q05/q95 interval …]
[unstable / high-uncertainty note where flagged]
district median (<dist>): <med> m across <n> stations
QUESTION: <question>
ANSWER:
```

10.8 API `[LIVE logic; planned HTTP]`

`POST /ml/assistant/chat` body `{question, station?, station_slug?, model?}` → `{answer, facts, station}`.

10.9 Request/response format

```
{"question": "Is level rising?", "station": "Ramchhitoni Sahawar (UP-011)"}
→
{"answer": "Latest level is -2.84 m on 2026-09-05 … expects a rise of about +0.12 m
 (to -2.72 m) over 30 days, 90% band +/- 1.61 m.",
 "facts": {"last": -2.841, "last_date": "2026-09-05", "change_30d": …, "forecast": {…}},
 "station": "Ramchhitoni Sahawar (UP-011)"}
```

10.10 Error handling

- No telemetry → `answer: "No telemetry found for station '<X>'."`, facts `{}`.
- Ollama down/pull-missing (page) → warning + facts panel still renders; chat returns error message with `ollama list` / `ollama pull {model}` instructions.
- Controller: missing `question` → `400 {error:"bad request", detail:"\` question` is required"}`; gateway failure → `502`; assistant error → `503`.

10.11 Chat history requirement

`[PLANNED]` persist to `chat_logs` table (station, question, answer, facts JSONB, model, created_at). Streamlit keeps `<6` turns in `st.session_state.as_messages`.

10.12 Forecast → summary/recommendation

`_forecast_summary` turns model numbers into text: `direction` = "expected rise" (change ≥0) / "expected decline" + `band_half` + `plausible` flag (stops the LLM over-trusting runaway predictions) + `high_uncertainty` (station stride RMSE > 2× fleet median).

---

11. Existing Code Structure (`ml/` + backend)

11.1 `ml/` files & functions

File	Purpose	Key functions
`app.py`	Streamlit entry (4 pages + Verification, unwired)	`st.navigation(…)`
`app_pages/*.py`	assistant, correlation, forecast, data_sources + verification (§1.6)	page scripts
`config.py`	sources/radii/coverage/caps/paths	`SOURCES`, `FULL26_*`, `RAIN/WEATHER/RIVER/CANAL_RADIUS_KM`, `MAX_RAIN_MM_H`
`00_probe.py`	probe NWIC resources	→ `probe.json`

File	Purpose	Key functions
`01_select.py`	full-2026 selection	→ 600/1353, `selected_gwl_stations.csv`
`02_fetch_selected.py`	fetch drivers	resume-safe, per-district
`03_merge_normalize.py`	caps+clean	→ `raw/*_norm.parquet`, `manifest.json`
`04_align.py`	daily alignment	IDW rain, nearest weather
`04b_align_6h.py`	6h grid	→ `table_6h.parquet`
`05_correlate.py`	correlation + lag	→ `correlation_report.csv`, `lag_curves.csv`
`06_features.py`	features/target	`drop_gwl_spikes`, `flag_regime_shift`, `build`, `build_full`, `rain_exp_30d`, `split`
`06_train.py`	training	`main`, `_write_model_metadata`
`07_evaluate.py`	benchmark	→ `model_metrics.csv`, `predictions_2026.parquet`
`07_soil.py`	SoilGrids fetch	→ `data/soil/`
`08_lulc.py`	Bhuvan LULC	→ `data/soil/lulc_district.csv`
`10_ablate.py`	ablation	→ `ablation.csv`
`11_quantile.py`	quantile models	→ `xgb_q*.joblib`, `quantile_calibration.json`
`11_fleet.py`	fleet scan	`category`, `main`, `_write_snapshot`, `_write_csvs`
`12_diagnostics.py`	VIF/permutation/OAT	→ `diagnostics.json`
`13_refresh_nwic.py`	incremental refresh + retrain/ deploy	`--dry-run`, `--retrain`, `--deploy-check`
`14_6h_features.py`	causal 6h feature frames (feeds `30_traj_datasets`)	→ `data/features_6h/`
`16_future_drivers.py`	driver climatology + future-driver bridge	→ `data/meta/driver_climatology.parquet`
`18_cwc_river_forecast.py`	CWC 3-day river forecast fetch	→ `data/cfs/river_forecast_cwc.parquet`
`20_openmeteo_fetch.py`	Open-Meteo daily weather (365d history + 16d forecast)	→ `data/cfs/openmeteo_weather_daily.parquet`
`30_traj_datasets.py`	trajectory v2 feature frames	→ `data/features_traj/`
`31_train_traj.py`	trajectory multi-horizon quantile XGBoost	→ `models/traj_xgb_*.json`, `traj_config.json`
`32_backtest_traj.py`	honest trajectory backtest + calibration	→ `traj_backtest_*.json/csv`, `traj_calibration.json`
`33_traj_reliability.py`	reliability buckets + evidence weights	→ `models/traj_reliability.json`
`_trajectory.py`	trajectory v2 engine (Forecast page + `/forecast/<slug>`)	`trajectory_forecast`
`api.py`	Flask HTTP API v3.3.0	`/health`, `/stations`, `/series`, `/forecast/<slug>`, `/assistant/chat`, `/districts`, `/fleet/*`, `/alerts`
`app_charts.py` / `snapshot.py`	forecast chart helpers / PNG export	`trajectory_chart`, `trajectory_snapshot_png`
`refresh/`	fetch → features → inference → publish daemon + retrain gate	`cli.py`, `pipeline.py`, `scheduler.py`, `sources.py`, `features.py`, `inference.py`, `model_update.py`, `publish.py`, `verification.py`

File	Purpose	Key functions
`_model.py`	model loaders + forward forecast	`load_xgb_model`, `load_linear_model`, `load_quantile_models`, `load_predictions`, `forward_forecast`, `load_fleet_table`, `load_diagnostics`, …
`_utils.py`	shared loaders	`load_table_6h`, `station_recency`, `load_manifest`, `load_report`, `load_lag_curves`, `DRIVER_LABELS`, `SOURCE_LABELS`
`_assistant.py`	chatbot	`StationAssistant.facts/answer`, `_build_prompt`, `_forecast_summary`, `ollama_status`, `ollama_model`, `_clean_series`, `_series_stats`
`_soil.py`	soil loader	`load_soil`, `SOIL_COLS`
`_lulc.py`	lulc loader	`load_lulc`, `LULC_COLS`
`validation/*.py`	honest suite	`spatial_cv.py`, `overlap.py`, `residual_acf.py`, `spatial_residual.py`
`gate_check.py`	12-check regression gate (latest GATE PASS 10/10)	`check(name, ok, detail)`
`tests/`	stdlib unittest, 151 tests (2 skipped)	`test_trajectory_v2.py`, `test_refresh_*.py`, `test_verification.py`, `test_api.py`, …
`MODEL_CARD.md`, `README.md`	docs	—

11.2 `back-end/` (Express) `[LIVE]`

- `app.js` — middleware (helmet, cors, json, morgan) + route mounts + 404 + `errorHandler`.
- `routes/` — `station, telemetry, assessment, trend, ml, mlData, dataQuality, ingestion, data, analytics, alert`.
- `controllers/` — one per route family; `ml.controller.js` implements the gateway passthroughs (`getLiveStations`, `getLiveForecast`, `postAssistantChat`, …).
- `services/` — `mlGateway.js` (HTTP proxy→:5000, `makeRequest`), `stationService`, `telemetryService`, `assessmentService`, `modelService`, `statisticsService` (Mann-Kendall, Sen's slope), `groundwaterClassification`, `dataQualityService`, ingestion services.
- `middleware/errorHandler.js`, `middleware/validators.js` (express-validator rules).
- `db/schema.sql`, `db/pool.js`.

11.3 `front-end/` (Next.js 16, React 19, TS, Tailwind v4, Chart.js) `[LIVE scaffold]`

- `app/page.tsx` dashboard home (AlertBanner, StatGrid, Charts, QuickActions).
- `components/dashboard/` (`AlertBanner, Charts, QuickActions, StatGrid, StationList`), `components/ui/` (`Button, Card, Modal, StatCard`), `components/layout/`, `components/profile/`.
- No API wiring yet — components use hard-coded props.

---

12. Backend Integration Requirements — API contracts

> **Status:** Node routes + gateway exist and are live (they proxy to `:5000`), and the **Flask ML service is live** (`ml/api.py` v3.3.0: `/health`, `/stations`, `/stations/<slug>`, `/stations/<slug>/series`, `/forecast/<slug>`, `/assistant/chat`, `/districts`, `/fleet/*`, `/alerts`). When Flask is down, `/ml/live/*` returns `502 {"error":"ML service error","detail":"ML service unavailable"}`.

12.1 Node routes currently mounted `[LIVE]` (see §14 list)

`/health`, `/stations*`, `/telemetry*`, `/assessments*`, `/trends*`, `/ml*`, `/ml-data*`, `/data-quality*`, `/ingestion*`, `/data`, `/analytics`, `/alerts`.

12.2 Slug convention `[LIVE]`

Lowercase hyphenated station name (e.g. `ashadha-prathmik-vidyalaya`): use `encodeURIComponent(slug)`. Always fetch slugs from `GET /stations` — never hand-type. The exact station name also resolves as a fallback.

12.3 Per-ML-feature contracts (target Flask `:5000`)

12.3.1 Forecast

```
Endpoint: GET /ml/live/forecast/:slug
Method:   GET            (proxied to GET /forecast/:slug?days=)
Request:  path=slug (URL-encoded), query=days (7..90, default 30)
Response: 200 forecast dict (§7.4)     404 unknown slug     502 gateway     500 internal
```

```
// GET /forecast/ashadha-prathmik-vidyalaya
{"station":"ASHADHA PRATHMIK VIDYALAYA","date_from":"2026-09-05T00:00:00",
 "date_to":"2026-10-05T00:00:00","anchor":-6.411,"pred_xgb":-0.213,
 "xgb_level":-6.624,"ridge_level":-7.896,"q05_level":-7.835,"q50_level":-6.12,
 "q95_level":-4.558,"widen_k":1.0,"band_half":1.64,"station_stride_rmse":0.597}
```

12.3.2 Stations / location

```
GET /ml/live/stations            ?district=&q=&limit=   → [{station,district,slug,lat,lon,last_ts,full26}]
GET /ml/live/stations/:slug                             → station facts (§10.4) KPIs
GET /ml/live/districts                                  → [{district, n_stations, last_ts}]
GET /ml/live/models                                     → model/artifact index + metrics
```

12.3.3 Fleet

```
GET /ml/live/fleet/forecasts                            → {horizon_days, generated_at, stations:[…]}
GET /ml/live/fleet/recovery   ?window_days=&top=&min_stations=  → district recovery ranking
GET /ml/live/fleet/scan       ?district=&threshold=&horizon=    → decline scan (quality-gated)
```

12.3.4 Assistant

```
POST /ml/assistant/chat
Body:   {"question": "...", "station": "..."} | optional station_slug, model
400     missing question           {"error":"bad request","detail":"`question` is required"}
503     assistant down             {"error":"assistant unavailable", ...}
200     {"answer": "...", "facts": {...}, "station": "..."}
```

12.3.5 Backend 4xx/5xx conventions

- 404 route: `{"error":"Route not found"}`
- validators: `{error: "…"}` / array of `{param, msg}` via errorHandler.
- 502 ML service unreachable: `{"error":"ML service error","detail":"ML service unavailable"}`.

---

13. Database Requirements

13.1 Existing schema (`back-end/db/schema.sql`) `[LIVE]`

Tables: `ingestion_runs`, `stations`, `telemetry_observations`, `assessment_units`, `assessment_records`, `data_quality`, `model_metadata`, `model_outputs`, `groundwater_data`. Key relations: - `telemetry_observations.station_id → stations.id` (FK, cascade), unique `(station_id, observed_at)`, index on observed_at. - `ingestion_run_id → ingestion_runs.id`. - `assessment_records → assessment_units.id`. - `model_outputs → model_metadata.id`. - `stations` has `external_station_id UNIQUE`, `first/last_observed_at`, `observation_count`, indexes on state/district/agency/coords.

13.2 Suggested additions `[PLANNED]`

Table	Fields (types)	Keys	Notes
`ml_stations`	`station PK text`, district, tehsil, block, lat/lon numeric(12,8), rl_msl numeric, n_2026_months int, n_2026_records int, first_2026 ts, last_record ts, full26 bool	PK station	mirror of selected_gwl_stations.csv
`ml_predictions`	`id PK bigserial`, station FK→ml_stations, time ts, horizon int, target/gwl/xgb/ridge/persist/clim/q05_lvl/q95_lvl numeric, err_xgb/err_ridge numeric, stride bool, window_id int	unique(station,time)	bulk-load predictions_2026.parquet (or serve from parquet)
`ml_fleet_snapshot`	`station PK`, district, anchor/date_from, age_days int, anchor_valid bool, xgb_level, q05/q50/q95_level, change_30d_m, change_pooled_m, band_half_m, plausible bool, category text, generated_at ts	PK station	fleet_forecast.csv
`quantile_calibration` (singleton)	coverage_stride, widen_factor_k, half_width_median_m, half_width_p90_m	PK id=1	quantile_calibration.json
`model_meta` (singleton)	trained_at, n_stations, n_districts, val_rmse_xgb, val_rmse_ridge, source_archive	PK id=1	model_metadata.json
`chat_logs`	`id PK bigserial`, station text, question text, answer text, facts jsonb, model text, created_at ts	idx station, created_at	assistant history
`alert_rules` (future)	`id`, station, threshold_m, band_ref text, active bool, created_at	FK station	for push alerts
`fleet_alerts` (future)	`id`, station, change_30d_m, band_half_m, breached bool, snapshot_ts	FK station	significant movers

13.3 Requirements in prose

Store: stations, telemetry (or keep parquet source), predictions (historical), fleet snapshots, calibration + model meta (single-row), chat history, alert rules/alerts, user/profile (if roles added), ingestion runs, assessment data, data-quality records. Indexes on `station`, `observed_at/date`, `district`, `category`, `created_at`. Foreign keys: telemetry→stations, predictions→stations, fleet→stations, outputs→model_meta.

---

14. Frontend Requirements (per screen)

> `[LIVE]` = scaffold exists; `[PLANNED]` = build new.

Page	Purpose	Components	Inputs	Charts	Cards/KPIs	Tables	Needs API
Home/Dashboard `[LIVE scaffold]`	state/district headline	AlertBanner, StatGrid, Charts, QuickActions	district/state select	total/current extraction donut, recharge bar	extraction %, rainfall mm, recharge MCM	—	`GET /ml/live/stations?district=`, `/assessment/summary`
Station Watchlist `[PLANNED]`	pick + search stations	search, virtual list, recency badge	q, district	sparkline per row	—	station×last_ts	`GET /ml/live/stations`
Station Detail `[PLANNED]`	KPIs + GWL + forecast	KPI row, line chart w/ band, tabs	station	GWL series + q05/q95 band + segment	anchor, ±band, RMSE	facts table	`GET /ml/live/stations/:slug`, `/ml/live/forecast/:slug`, `/telemetry/:stationId`

Page	Purpose	Components	Inputs	Charts	Cards/ KPIs	Tables	Needs API
Forecast `[PLANNED]`	backtest + outlook	selectors (station, model), error-band chart, outlook table	station, model	backtest curve + band, outlook segment, rain volume overlay	RMSE, interval ±, anchored reads	metrics	`GET /ml/live/forecast/:slug`, `GET /ml/live/models`
Fleet/ Signals `[PLANNED]`	whole-fleet movers	6 KPI row, histogram, filters	category, district, movers-only	Δ histogram ±0.3 m rules	6 KPIs	movers, district rollup, station scan	`GET /ml/live/fleet/forecasts`, `/fleet/recovery`, `/fleet/scan`
Impact/ Analysis `[PLANNED]`	driver importance	segmented mode/metric, bars, table	mode, metric	importance &	coef	bars, correlation bars	strongest/weakest driver
Assistant `[PLANNED]`	chat + facts	chat list, input, facts panel, model-status	question, station	—	fact KPIs	facts	`POST /ml/assistant/chat`, `GET /ml/live/stations`
Sources `[PLANNED]`	data provenance	manifest tables	source	coverage bars	—	per-source quality, empty sources, soil/ LULC status	`GET /ml/live/models`, `/ml-data/*`
Mobile (design target)	trading-terminal UI	dark theme, watchlist, price-style charts, heatmap, sidebar comparison	station/ district	GWL+band charts, district heatmap, indicator overlays, rain volume	watchlist sparkline	district comparison, movers	same API set

Loading/error/empty states to handle: spinner while `makeRequest` (60s timeout) runs; `available:false` banner on boot; "No feature frame for this station." when forecast `{}`; "run `ml/11_fleet.py` first" when fleet empty; Ollama-down warning in chat; empty charts when station has no data.

---

15. Frontend ↔ Backend ↔ ML data flow

15.1 Forecast flow

```
User ── select station ──► Frontend (Forecast page)
  │ GET /ml/live/forecast/:slug?days=30
  ▼
Backend ml.controller.getLiveForecast → mlGateway.getLiveForecast(slug, days)
  │ HTTP GET http://localhost:5000/forecast/:slug  (makeRequest, 60s timeout)
  ▼
Flask route [LIVE] → trajectory_forecast(station)
  │ anchor row from table_6h.parquet → shared direct multi-horizon quantile
  │ models (h = 1..120) → 120 genuine q05/q50/q95 deltas; level = anchor + delta
  ▼
Backend ← JSON (station, slug, anchor, trajectory[120] with time/gwl/q05/q50/q95/confidence_level, …)
  ▼
Frontend renders the trajectory card + anchor→+30d segment chart + interval caption.
```

15.2 Assistant flow

```
User question ─► Assistant page ─► POST /ml/assistant/chat {question, station}
  ▼ Backend postAssistantChat -> mlGateway.assistantChat
  ▼ Flask /assistant/chat [LIVE]
StationAssistant.facts(station) → _forecast_summary(station) → forward_forecast
_build_prompt(question, facts, history) → ollama.Client.chat(temperature 0.2)
  ▼ {answer, facts, station} ─► Frontend chat bubble + facts panel + (optional) DB chat_logs
```

15.3 Fleet flow

```
Frontend Fleet page → GET /ml/live/fleet/forecasts (+ /fleet/recovery, /fleet/scan)
  ▼ Backend passthrough + mlGateway
  ▼ [PLANNED] Flask reads outputs/fleet_forecast.csv (precomputed by 11_fleet.py --force; the offline scan exists, the
HTTP endpoint does not yet)
  ▼ JSON snapshot ─► 6 KPIs, histogram, district rollup, station table
```

15.4 Static/backend data flow (live Node only)

```
Frontend → GET /stations, /telemetry/:stationId, /assessments, /trends/:stationId
  ▼ station/telemetry/assessment service ⇄ PostgreSQL (schema §13)
  ▼ JSON → tables/charts
```

---

16. External APIs & Services

Service	Purpose	Auth	Env vars	Rate/free	Notes
NWIC/NWDP CKAN `datastore_search`	all telemetry (GWL + drivers)	none	—	free	resource ids in `ml/config.py`; source map `back-end/db/api/api.txt`
ISRO Bhuvan LULC statistics	district land-cover shares	key	`LULC_STATISTICS_API_KEY`	free/ patronised	`bhuvan-app1.nrsc.gov.in/api`
ISRIC SoilGrids v2 REST	per-station texture	none	—	free	`07_soil.py` background fetch
Ollama (local)	LLM `llama3.2:3b` (~2 GB)	none	`AQUIS_OLLAMA_MODEL`	free/local	no SaaS; `ollama serve`
NCEI CFSv2 6h-FLX	seasonal rain (staged)	none	—	free	paused; S3-403 workaround via per-step GRIB
CGWB assessments	Excel files via backend import	none	—	public	not in ML features

> No API keys/secrets appear in this document. `.env` holds `LULC_STATISTICS_API_KEY` (example values in `.env.example`).

---

17. Dependencies & Setup

17.1 Python (ml venv — verified versions)

```
pandas==3.0.5  numpy==2.5.2  matplotlib==3.11.1  scikit-learn==1.9.0
requests==2.34.2  flask==3.1.3  flask-cors==6.0.5  scipy==1.18.0
xgboost==3.4.1  joblib==1.5.3  streamlit==1.62.0  plotly==6.9.0
altair==6.2.2  ollama==0.6.2
```

Root `requirements.txt` (Streamlit Cloud): `pandas, numpy, matplotlib, scikit-learn, requests, flask, flask-cors, scipy, xgboost, joblib, streamlit, plotly`.

> **Add `ollama` to requirements** if the assistant is part of the deploy.

17.2 Node (backend)

Node **v22.23.1** in use (runtime requires Node 18+), Express, PostgreSQL 14+. `cd back-end && npm install && npm run migrate`.

17.3 Frontend

Next.js 16.2.9, React 19.2.4, TypeScript 5, Tailwind CSS v4, Chart.js ^4.5.1. `cd front-end && npm install && npm run dev`.

17.4 Environment variables

`DATABASE_URL, PORT, NODE_ENV, ML_SERVICE_URL, ML_TIMEOUT_MS, BACKEND_URL, AQUIS_OLLAMA_MODEL, LULC_STATISTICS_API_KEY`.

17.5 Model setup / local dev

```
# reproduce models from archive (optional; models already committed for deploy)
cd ml
venv/bin/python 06_features.py
venv/bin/python 06_train.py
venv/bin/python 11_quantile.py
venv/bin/python 11_fleet.py --force       # ~4-6 min
venv/bin/python 30_traj_datasets.py      # trajectory v2 frames (~7M rows)
venv/bin/python 31_train_traj.py         # shared multi-horizon q05/q50/q95
venv/bin/python 32_backtest_traj.py      # honest backtest + calibration
venv/bin/python 33_traj_reliability.py   # reliability buckets + weights
venv/bin/python 13_refresh_nwic.py --retrain   # or run app directly on committed artifacts

# run the live analysis dashboard
venv/bin/streamlit run app.py              # http://localhost:8501

# assistant
ollama serve &  ollama pull llama3.2:3b

# verify after edits
venv/bin/python -m unittest discover -s tests -v
venv/bin/python gate_check.py
```

17.6 Deployment (Streamlit Cloud)

Main file **`ml/app.py`**, root `requirements.txt`, models force-committed (`.gitignore` keeps them; ignore `ml/data/features/`, `ml/data/aligned/*.csv`, `ml/*.log`). For full-stack: deploy Flask ML service (`:5000`) + Point `ML_SERVICE_URL` at it.

---

18. Validation & Error Handling

Case	Where	Expected response
invalid/unsupported station	Flask forecast	`404 {"error": "unknown slug"}` / forecast `{}` → app "No feature frame for this station."
missing `question`	Node assistant	`400 {"error":"bad request","detail":"\` question` is required"}`
wrong datatype (`days`="abc")	planned Flask	`400 {"error":"invalid \` days`; must be int 7..90"}`
out-of-range `days`	planned Flask	`400` (7–90)
`limit/offset` bad	Node validators	`400 {error:…}`
location not in 29 districts	`/districts` / `/stations`	returns whatever exists; district filter yields empty list (not error)
missing data (empty table)	app loaders	page `st.stop()` with "run `ml/11_fleet.py` first" (fleet) / "No data for that selection." (assistant)

Case	Where	Expected response
insufficient history	forward_forecast	`{}` → warning
model failure	Flask	`500 {error, detail}`
ML service down	Node gateway	`502 {"error":"ML service error","detail":"ML service unavailable"}`
ML timeout (>60s)	gateway	`502` "ML service timeout"
assistant/Ollama down	Node/{Flask}	`503`; app shows facts panel + instruction message
DB failure	Node services	`500` via errorHandler
unknown route	Express	`404 {"error":"Route not found"}`

19. Testing

19.1 Existing ML tests `[LIVE]` — `ml/tests/`

151 tests (2 skipped), stdlib `unittest`, data-gated, no pytest: trajectory v2, refresh pipeline (core/future/training/schedule), verification, Flask API, assistant facts, calibration/diagnostics, forecast UI. Run: `venv/bin/python -m unittest discover -s tests -v` (the full Forecast-page AppTest is gated behind `AQUIS_APPTEST=1`). Gate: `venv/bin/python gate_check.py` → 12 checks, latest **GATE PASS 10/10** (stride RMSE 2.339, spatial CV 1.973/1.819, coverage 0.908, half-width 1.025/2.575, eff-N 6635, lag1 ACF 0.858, trajectory 30d 2.106 promoted, +30d coverage 0.90; the 2 recursive-model checks skip).

19.2 Sample inputs → expected outputs `[PLANNED automated]`

Test	Input	Expected
forecast	station "Ramchhitoni Sahawar (UP-011)"	dict §7.4; `anchor≈-2.84`, `band_half≈1.61`, all keys present
forecast empty	"Nonexistent Station XYZ"	`{}`
fleet	`fleet_forecast.csv`	502 scored; median Δ +0.32 m; `category` ∈ enum
correlation	driver=river_level, metric=spearman, mode=raw	corr ≈ 0.409
importance	`feature_importance.csv`	row 0 = `monsoon`
qcal	`quantile_calibration.json`	`widen_factor_k==1.0`, `coverage_stride≥0.8`
assistant	q="Is level rising?", station set	answer contains level/change; `facts.forecast.plausible` in {True,False}
Node forecast	GET /ml/live/forecast/:slug (Flask running)	200 + keys
Node no-Flask	GET /ml/live/forecast/:slug	502 + detail
chat missing question	POST /ml/assistant/chat {}	400
validators	/stations?limit=bad	400

19.3 Edge cases

Stale anchor (>45 d) → fleet `unreliable`; runaway pred (`|Δ|>12` or outside obs±25) → `plausible=false`; NaN quantiles → `band_half` null; station-missing in `station_summary` → `station_stride_rmse` null; driver columns absent → skipped windows.

20. Performance & Deployment

Concern	Value (measured/expected)
Inference (single station)	`forward_forecast` ≈ 0.1–0.2 s (after table cached); `trajectory_forecast` scores 120 horizons in one pass (XGBoost predict on 120 rows negligible)
Table load	`load_table_6h` ≈ 1.7 s (cached 10 m, 2.99M rows, ~58 MB mem)
Predictions read	`load_predictions` 10-col ≈ 0.07 s (18 MB parquet) after 24 h TTL
Groupby recency	`station_recency()` 3M-row groupby ≈ 0.09 s (cached)
Fleet scan (fresh `--force`)	≈ 4–6 min (502 stations)
Full eval/honest rerun	minutes (06/07/11)
Memory	~2–3 GB RSS for app (table6 + preds + models); Ridge fit switched to 1 slot/day sampling to fit 16 GB
CPU/GPU	CPU only (XGBoost `hist`, n_jobs=6 during train); no GPU required
API response (plan)	forecast <500 ms after warm cache; fleet from snapshot <100 ms; chat 15–60 s (LLM)
Concurrency	Streamlit is single-session-per-browser; for production use the flask service + persistent table cache, cache `forward_forecast` (TTL)
Caching	`@st.cache_data` TTLs: 10 m (tables/evals), 24 h (predictions), 15 m (fleet/diag); `@st.cache_resource` for joblib models
Model loading	joblib once per process (`@st.cache_resource`); warm lazy
Deploy	Streamlit Cloud (`app.py`, root requirements) for dashboard; Flask on app host for API; models committed; parquet dirs git-ignored
Production limits	Trajectory v2 outputs 120 genuine 6-hourly steps (direct multi-horizon, no recursion); the pooled direct model is fixed at the 30-day endpoint; UP telemetry coverage limited; chat needs local Ollama (2 GB) or an external host

---

21. Security

- **Auth:** none today (localhost/LAN). `[PLANNED]` token/JWT for the API + roles (Officer/Researcher/Public/Admin) if deployed publicly; mobile needs auth at Node layer.
- **Authorization:** role-based pages (Officer: fleet/alerts; Public: read-only explorer; Researcher: impact/analysis).
- **API security:** Express already uses `helmet` (CSP off), `cors`, JSON body limit; add rate limiting (`express-rate-limit`) + input validation via `middleware/validators.js` (extend for new routes).
- **Secrets:** only in `.env` (git-ignored); `.env.example` has placeholders; no keys in repo.
- **Input sanitisation:** express-validator chains; LLM prompt build is a strict template — no injection surface beyond the fee-text `question` (phrased into the prompt as-is; keep it that way, never add system-level tool calls).
- **Rate limiting:** apply to `/ml/assistant/chat` (LLM is local but slow) and `/ml/live/*`; e.g. 30/min per IP, 300/h.
- **User-data privacy:** telemetry is public government data; chat logs are PII-light but store facts/answers; honour minimal-collection + retention.

---

22. Important ML Limitations (don't over-claim)

1. **Coverage:** only 29 districts/600 stations have live 2026 telemetry → 45 UP districts are NOT forecastable. Manual (quarterly) GWL is not ingested.
2. **Driver scarcity:** rainfall 8 districts, weather ~3, river 2, canal 1 → most stations have NaN drivers; correlation sample is small; river uses a **district-level proxy** (no coords published).
3. **Time:** archive starts 2021 (5.6 y) — seasonality beyond 5 y is under-sampled; regimes pre-2021 ignored.
4. **Horizon single-step:** 30-day is the only trained horizon; "7–90 days" API param is not a multi-step model.

5. **Overlap honesty:** 378k test rows ≈ 6.6k effective; use **stride** numbers (RMSE 2.339 vs persistence 2.382, +1.8%). Deltas < ±0.05 m are noise.
6. **When not to trust:** NaN/stale anchor (>45 d), `plausible=false` (runaway Δ>12 m or outside obs±25 m), `high_uncertainty` (stride RMSE > 2× fleet median), any fleet `category="unreliable"`.
7. **Assumptions:** level = anchor + delta where persistence = 0-change; GWL sign convention negative = depth below ground (0 = ground level); static soil/LULC gated out (redundant with ordinals); no extraction data available (CGWB PDF/Excel only, 1–2 y lag).

---

23. Exact Feature → API → ML Mapping

Feature	Frontend screen	API (planned)	Backend fn	ML model	Input	Output
F1 watchlist	Watchlist	`GET /ml/live/stations`	`getLiveStations` → `mlGateway.getStations`	none	district/q/limit	station list
F2 KPIs	Station Detail	`GET /ml/live/stations/:slug`	`getLiveStation` → `getStation`	none	slug	facts
F3 trend	Station Detail	`GET /telemetry/:stationId`	telemetryService	none	stationId	series
F4 backtest	Forecast	(parquet via `load_predictions`)	`_model.load_predictions`	xgb/ridge	station	date×target/gwl/xgb/qband
F5 forward forecast	Forecast	`GET /ml/live/forecast/:slug` [LIVE]	`getLiveForecast` → `trajectory_forecast`	trajectory v2 (120×6h q05/q50/q95)	station	trajectory dict (§7.4 shape + trajectory/confidence)
F6 confidence	Forecast/Detail	same as F5	`trajectory_forecast`	evidence-weighted (interval + vs-persistence + direction + …)	station	per-point confidence_level + reason
F7 fleet score	Fleet	`GET /ml/live/fleet/forecasts`	`getFleetForecasts`	pooled fwd	none	csv rows
F8 movers	Fleet	`GET /ml/live/fleet/scan`	getFleetScan	pooled fwd	dist/threshold/horizon	movers
F9 district rollup	Fleet	`GET /ml/live/fleet/recovery`	getFleetRecovery	pooled fwd	window/top/min	district stats
F10 scan filters	Fleet	(client-side)	—	—	filters	filtered rows
F11 correlation	Impact	`GET /ml/live/models` (+analysis)	load_report	none (stats)	mode/metric	corr table
F12 recharge lag	Impact	ditto	load_lag_curves	none	—	lag curve
F13 driver overlay	Station Detail	`GET /ml-data/telemetry/:stationId`	mlData controller	none	driver	dual series
F14 benchmark	Impact	`GET /ml/live/models`	load_model_metrics	all	basis	4-model table
F15 honest	Impact	ditto	honest/metric loaders	—	—	stride metrics

Feature	Frontend screen	API (planned)	Backend fn	ML model	Input	Output
F16 diagnostics	Impact	ditto	load_diagnostics	—	—	VIF/perm/OAT
F17 importance	Impact	ditto	load_importance/coefs	xgb/ridge	kind	ranked
F18 assistant	Assistant	`POST /ml/assistant/chat`	`postAssistantChat` → `assistantChat`	LLM+pooled	question/station	answer+facts
F19 sources	Sources	`GET /ml/live/models`, `/ml-data/*`	manifest/mlData	none	—	per-source quality
F20 soil/LULC	Sources/Detail	(artifacts)	load_soil/load_lulc	gated	station/district	texture/classes

---

24. Final Integration Checklist — Streamlit ML → Full-Stack

Phase 0 — freeze artifacts: `.joblib` + `models/*.json` committed; parquet dirs git-ignored; `13_refresh_nwic.py --retrain` reproducible.

Phase 1 — ML API (the big gap): - [ ] Ship a **Flask app** exposing the contract in §12 (`/health`, `/stations`, `/stations/:slug`, `/districts`, `/models`, `/forecast/:slug`, `/fleet/forecasts`, `/fleet/recovery`, `/fleet/scan`, `/assistant/chat`). - [ ] Map each route to the existing fn (`forward_forecast`, `StationAssistant`, `load_fleet_table`, manifests) — no new ML logic needed. - [ ] Slug↔name resolver built on `selected_gwl_stations.csv`. - [ ] Persistent cache (in-process dict / tinykv) for table6 + forward forecasts. - [ ] Error mapping per §18 (400/404/500).

Phase 2 — Backend: - [ ] Verify `back-end/services/mlGateway.js` against the Flask surface (already has all callers + `_qs` + timeout). - [ ] Point `ML_SERVICE_URL` at Flask in production `.env`. - [ ] Add tables §13.2 (ml_stations, ml_predictions opt, ml_fleet_snapshot, chat_logs, alerts); migrate script. - [ ] Extend `validators.js` for new live routes (slug, days, question).

Phase 3 — Frontend (Next.js): - [ ] Wire `components/dashboard/*` to API (remove hardcoded props). - [ ] Build pages §14: Watchlist → Detail → Forecast → Fleet → Impact → Assistant → Sources, each with loading/error/empty states (§18). - [ ] API client module (`lib/api.ts`) hitting Node `/ml/live/*`. - [ ] Chart components (Chart.js) for GWL+band, backtest, histogram, heatmap, driver overlays.

Phase 4 — Mobile (trading-app-style): - [ ] Watchlist (recency-sorted) · station detail with GWL "price chart" + 90% band (Bollinger-like) · indicator overlays (7d/30d MA, rolling-std band, rain volumes) · seasonal/monsoon overlay · district/UP heatmap · district sidebar comparison · fleet movers · alerts (Δ > band push) — all fed by the same APIs.

Phase 5 — Hardening: - [ ] Auth/JWT + roles; rate-limit chat; input sanitisation; CSRF not applicable (API practices) / proper CORS origins. - [ ] `gate_check.py` + `unittest` in CI; AppTest smoke for the 4 live pages; API tests (§19.2). - [ ] Monitoring: `setuptools` read of `model_metadata.trained_at` vs NWIC max ts (`13_refresh_nwic.py --deploy-check`). - [ ] Chat history persistence + retention policy.

Everything above is extracted from the shipping code and artifacts. Numbers (metrics, hyperparameters, JSON examples) were read live from the repo on 2026-09-09/10.

8 ML System Specification

AQUIS ML / System Specification

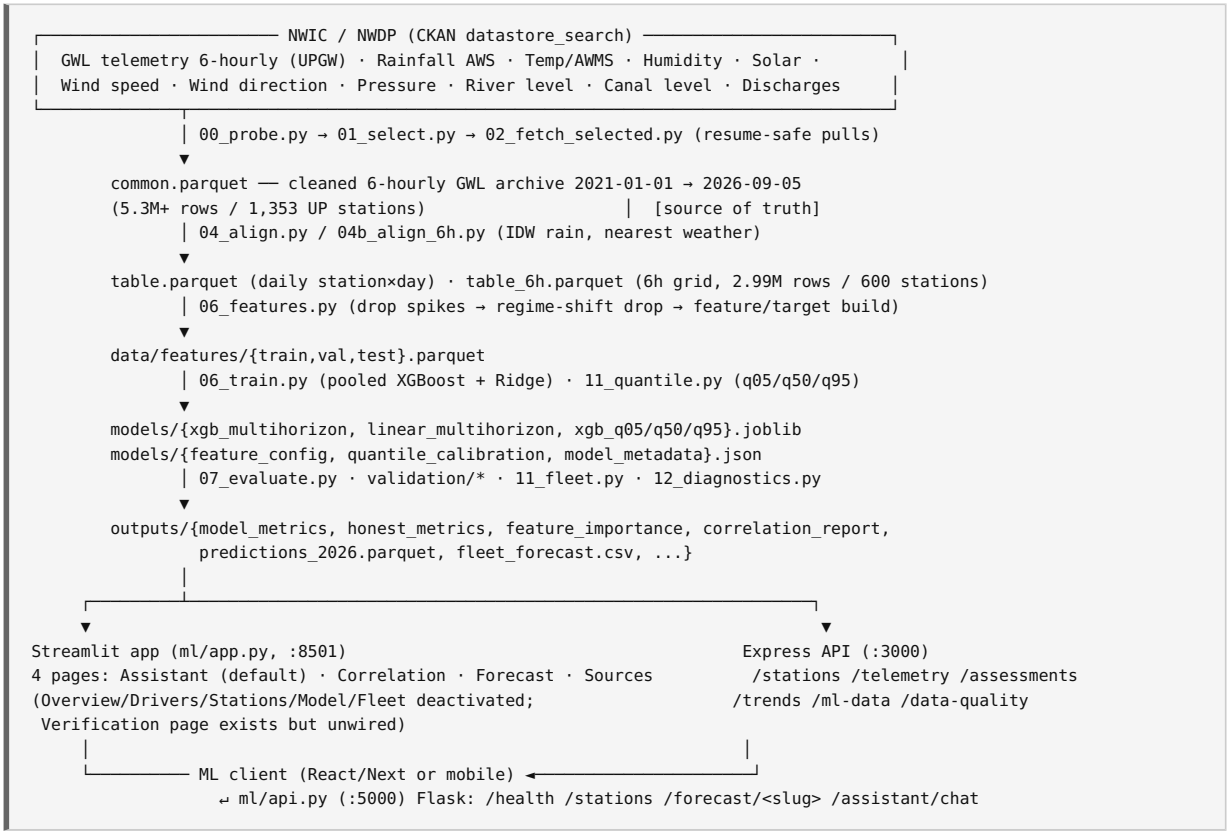
Purpose: Single source of truth for building the complete AQUIS application (frontend + backend + mobile) on top of the existing Python ML module (`ml/`). Every name, schema, JSON shape and function signature below is taken from the actual implemented code and produced artifacts — not a generic design.

Scope: Uttar Pradesh groundwater telemetry forecasting (pooled XGBoost, 30-day horizon, 6-hourly grid) + correlation/impact analysis + fleet scan + station-locked LLM assistant.

Live today: - Streamlit dashboard — `ml/app.py` (port 8501) — 4 pages in nav: Assistant (default) · Correlation · Forecast · Sources - Headless Flask ML API — `ml/api.py` (port 5000) — `/`, `/health`, `/stations`, `/forecast/<slug>`, `/assistant/chat` - Node.js Express API — `back-end/` (port 3000), proxying `/ml/*` to the Flask service via `mlGateway.js` - Refresh daemon — `ml/refresh/` — 6-hourly fetch→inference→publish grid on `{01,07,13,19}:00 IST` (see `ml/README.md` "Refresh pipeline")

---

1. Complete end-to-end application flow



Production data flow (repo README): NWDP Telemetry API + CGWB Assessment Excel → PostgreSQL → Statistical Analysis + ML → Express APIs → Frontend.

Key constraint: UP groundwater monitoring is *mostly manual* (quarterly), AQUIS ingests only the **telemetry** feed → 1,353 stations / 34 districts known to the archive → 600 stations / 29 districts "still alive in 2026" qualify as model anchors. A station without a live 2026 telemetry stream is not forecast-able.

2. ML features

One feature row per **(station, 6h-slot)**. `gwl` is the **anchor** — used to build the 30-day change but never predicted. Target = `GWL(t+120) − GWL(t)` (30 days × 4 slots/day = 120 six-hour steps).

Full feature list = `ml/models/feature_config.json` → `num_cols` (30) + 2 ordinal columns (`st_id`, `dist_id`) = **32 model inputs** for XGBoost.

#	Feature	Group	Meaning / formula
1	`gwl`	anchor	Current reading (day-0 anchor, never target)
2	`lag1`	GWL lags	GWL 1 slot earlier (=6 h)
3	`lag4`	GWL lags	GWL 4 slots earlier (=1 day)
4	`lag8`	GWL lags	GWL 8 slots earlier (=2 days)
5	`lag28`	GWL lags	GWL 28 slots earlier (=7 days)
6	`lag120`	GWL lags	GWL 120 slots earlier (=30 days)
7	`gwl_roll7_mean`	GWL rolling	Rolling mean over 28 slots (=7 d), min_periods=1
8	`gwl_roll7_std`	GWL rolling	Rolling std over 28 slots (=7 d)
9	`gwl_roll30_mean`	GWL rolling	Rolling mean over 120 slots (=30 d)
10	`gwl_roll30_std`	GWL rolling	Rolling std over 120 slots (=30 d)
11	`rain_1d`	driver	6h-rain sum over 4 slots (=1 d), mm
12	`rain_7d`	driver	6h-rain sum over 28 slots (=7 d), mm
13	`rain_30d`	driver	6h-rain sum over 120 slots (=30 d), mm
14	`temp`	driver	Air temperature, °C
15	`temp_7d`	driver	28-slot rolling mean of `temp`
16	`humidity`	driver	Relative humidity, %
17	`humidity_7d`	driver	28-slot rolling mean of `humidity`
18	`solar`	driver	Solar radiation (auto-detected unit)
19	`wind_speed`	driver	Wind speed (auto-detected unit)
20	`pressure`	driver	Atmospheric pressure (auto-detected unit)
21	`river_level`	driver	River water level, m
22	`canal_level`	driver	Canal water level, m
23	`year`	calendar	Calendar year (2012→2026)
24	`month_sin`	calendar	`sin(2π·month/12)`
25	`month_cos`	calendar	`cos(2π·month/12)`
26	`doy_sin`	calendar	`sin(2π·doy/365.25)`
27	`doy_cos`	calendar	`cos(2π·doy/365.25)`
28	`hour_sin`	calendar	`sin(2π·hour/24)` (fractional hour)
29	`hour_cos`	calendar	`cos(2π·hour/24)`
30	`monsoon`	calendar	`doy − 152` (positive during monsoon ≈ Jun 1 onward)
31	`st_id`	ordinal	Stable integer per station (sorted by name, train/val/test shared)
32	`dist_id`	ordinal	Stable integer per district (alphabetical)

> **Gated/experimental features** (fetched, shown in app, but disabled in the shipped model — see §6 note and README items 6–9): `sand_0_30`, `silt_0_30`, `clay_0_30`, `bdod_0_30`, `sand_60_100`, `silt_60_100`, `clay_60_100`, `bdod_60_100` (ISRIC SoilGrids v2, `ENABLE_SOIL_FEATURES = False`), `lulc_*_pct` (9 class shares, ISRO Bhuvan 50K, kept out via comment), `rain_exp_30d` (`ENABLE_RAIN_EXP = False`), CGWB extraction `ann_gw_draft_mcm` / `stage_of_development_pct` (merged when available).

Feature ranking by XGBoost gain (`outputs/feature_importance.csv`):

Rank	Feature	gain	freq
1	monsoon	25381.8	149
2	gwl_roll7_std	21670.7	172
3	doy_sin	14347.3	265
4	gwl_roll30_std	10718.3	369
5	year	10004.1	103
6	doy_cos	7990.7	226
7	lag1	7534.9	85
8	gwl	7330.6	333
9	month_sin	7019.1	50
10	hour_sin	6426.8	72
…	gwl_roll30_mean	6329.9	121
…	lag120	5723.3	184
…	lag28	5447.7	160
…	lag4	5311.8	81
…	rain_30d	2589.3	201
…	st_id	2239.0	243
…	dist_id	1748.2	167

---

3. Input parameters — name, meaning, unit, datatype, range, status

3.1 Model features (per 6h row, dtype float32 in feature tables)

Param	Meaning	Unit	dtype	Typical range	Required
`gwl`	Groundwater level (depth below ground, negative convention)	m	float32	−120 … +70 (per-station trimmed)	yes
`lag1/4/8/28/120`	past GWL at 6h/1d/2d/7d/30d lag	m	float32	same as `gwl`	yes (NaN allowed in-lag)
`gwl_roll7_mean/std`	7-d rolling mean/std of GWL	m	float32	−120 … +70	yes
`gwl_roll30_mean/std`	30-d rolling mean/std of GWL	m	float32	−120 … +70	yes
`rain_1d/7d/30d`	cumulative rainfall	mm	float32	0 … ~2000 (30d)	yes
`temp`, `temp_7d`	air temperature	°C	float32	−10 … 55	yes

Param	Meaning	Unit	dtype	Typical range	Required
`humidity`, `humidity_7d`	relative humidity	%	float32	0 … 100	yes
`solar`	solar radiation	auto	float32	varies	yes
`wind_speed`	wind speed	auto (km/h or m/s)	float32	varies	yes
`pressure`	atmospheric pressure	hPa	float32	varies	yes
`river_level`	river water level	m	float32	varies	yes (sparse)
`canal_level`	canal water level	m	float32	varies	yes (sparse)
`year`	calendar year	yr	int (float32)	2012–2026	yes
`month_sin/cos`, `doy_sin/cos`, `hour_sin/cos`	cyclic encodings	dimensionless	float32	−1 … 1	yes
`monsoon`	day-of-year − 152	day	float32	−152 … 213	yes
`st_id`	station ordinal	dim-less	int32	0 … 548 (549 stations)	yes (XGBoost only)
`dist_id`	district ordinal	dim-less	int32	0 … 28 (29 districts)	yes (XGBoost only)
`Station`	station name (raw string)	—	str	see `feature_config.stations`	yes (Ridge OHE)
`District`	district name (raw string)	—	str	see `feature_config.districts`	yes (Ridge OHE)

> **Missing-data policy:** XGBoost natively handles NaN (default via `tree_method hist`); Ridge uses sklearn `SimpleImputer(strategy="median")` inside the numeric branch of a `ColumnTransformer`. Features are causal (lags only) — never future.

3.2 Runtime request inputs (API / app)

Param	Meaning	dtype	Range / example	Required
`station` (or slug)	station identifier	str	`"Ramchhitoni Sahawar (UP-011)"`	yes (forecast/chat)
`district`	district filter	str	`"KAUSHAMBI"`, `"GORAKHPUR"`	optional
`model`	model choice for the page	str	`"xgboost"` | `"ridge"`	optional (default xgboost)
`question`	free-text for chatbot	str	non-empty	yes (chat)
`history`	prior chat turns	list[dict]	`[{"role":"user","content":…}]`	optional
`model` (chat)	LLM model override	str	`"llama3.2:3b"`	optional
`days`	forecast horizon (API, fixed at 30)	int	7–90, default 30	optional

4. Dataset & schema

4.1 `data/processed/common.parquet` — raw cleaned GWL archive (source of truth)

~5.3M rows / 1,353 UP stations / 2021-01-01 → 2026-09-05. Key columns: `Station, District, Tehsil, Block, Latitude, Longitude, RL_MSL, Data Acquisition Time, Groundwater Level Telemetry 6 Hourly (meter)`. Refreshed incrementally by `13_refresh_nwic.py`.

4.2 `data/aligned/table_6h.parquet` — 6-hourly station×slot (model input)

2,990,000+ rows / 600 stations. Columns: `Station, District, time, gwl, rain, temp, humidity, solar, wind_speed, wind_direction, pressure, river_level, canal_level` (+ `dist_km` variants where relevant). `time` is 6-hourly (00/06/12/18); drivers re-binned to the same 6h steps.

4.3 `data/aligned/table.parquet` — daily station×day (correlation/overview)

`Station, District, date, gwl, <drivers>` collapsed daily.

4.4 `data/features/{train,val,test}.parquet` — feature/target tables

Column order (from `KEEP` in `06_features.py`): `Station, District, time, date, horizon, target, target_d, st_id, dist_id` + 30 `NUM_COLS` (all float32). `target = GWL(t+120)`, `target_d = target − gwl`.

4.5 `outputs/predictions_2026.parquet` — 2026 test predictions (app chart)

Schema (verified live):

Col	dtype	Purpose
`Station`	str	station name
`District`	str	district
`date`	datetime64	prediction date
`time`	datetime64	6h slot
`horizon`	int64	30
`target`	float64	actual GWL(t+120)
`gwl`	float32	anchor
`feat_days`	int8	feature-history days used
`xgb`	float32	XGBoost level prediction
`ridge`	float64	Ridge level prediction
`persist`	float32	persistence (anchor)
`clim`	float64	climatology
`err_xgb`, `err_ridge`	float64	level errors
`q05_lvl`, `q95_lvl`	float32	calibrated 90% interval (level)
`step_idx`	int64	index within 120-step window
`window_id`	int64	non-overlap window id
`stride`	bool	True = independent (non-overlap) row

4.6 `data/meta/selected_gwl_stations.csv` — anchor registry (§11)

`Station, District, Tehsil, Block, Latitude, Longitude, n_2026_months, n_2026_records, first_2026, last_record, full26`.

4.7 `data/meta/selected_districts.json` — the 29 live districts

`["AGRA","AZAMGARH","BAGHPAT","BANDA","BAREILLY","BUDAUN","CHITRAKOOT", "DEORIA","ETAH","FATEHPUR","GHAZIABAD","GORAKHPUR","HAMIRPUR","JALAUN", "JHANSI","KAUSHAMBI","KANSIRAM NAGAR","LALITPUR","MAHOBA","MEERUT", "MIRZAPUR","MUZAFFARNAGAR","PRATAPGARH","SAHARANPUR","SANT RAVIDAS NAGAR", "SHRAWASTI","SIDDHARTHNAGAR","SONBHADRA","VARANASI"]`

---

5. Data sources & preprocessing

5.1 Sources (all via NWIC/NDWP CKAN `datastore_search`)

IDs live in `ml/config.py` (`SOURCES`); authoritative source map is `back-end/db/api/api.txt`. Fetch done per-district with resume markers.

Source	archive res id (uuid)	live res id (uuid)	value hint	agg
gwl	`84bfda45-…-f7a93ee57522`	`31c66a49-…-a0ea0d9c8b0c`	Groundwater Level Telemetry 6 Hourly (meter)	mean
rainfall	`2c0805b7-…-cf7bc1ecd148`	`46e9afa0-…-49e0b5b1b6d3`	Telemetry Hourly Rainfall (mm)	sum
temperature	`feb54802-…-8311ae04df0e`	`c2c7b5fa-…-18eb5297f7b0`	Air Temperature Telemetry Hourly (AoC)	mean
river_level	`9672a7e9-…-a4128792d070`	`94b5345e-…-d93cdede2025`	River Water Level Telemetry Hourly (meter)	mean
humidity	`14a3cfa8-…-c64693725779`	`0fcb6700-…-41397f417c8c`	Telemetry Hourly Relative Humidity (%)	mean
solar	`6128c3ff-…-5b2ef492e700`	`c1666f7a-…-39ec192ad004`	auto-detect	mean
wind_speed	`fc1314ef-…-5b1ec7a90037`	`beb454fb-…-bf8b4b282efa`	auto-detect	mean
wind_direction	`b9acc862-…-57d3580a75b6`	`9e8132bc-…-af05-adbac43f7b24`	auto-detect	mean
pressure	`32a8fd55-…-ac24c447f28a`	`3525c93c-…-3033fd21a645`	auto-detect	mean
canal_level	`ea65ac74-…-f927633def4d`	`8075a3b8-…-ba7a-94b193f3936b`	auto-detect	mean
canal_discharge	`a6158de8-…-6cf5296f161a`	`14a80ee5-…-1f5857b843ff`	—	mean
5× reservoir discharge	listed in config	listed in config	—	mean

> `canal_discharge` and reservoir discharges: **0 rows within selected districts** (barrages outside them) — confirmed-empty, not re-probed per rerun.

Covariates in the unioned selected districts (from `data/meta/manifest.json`): gwl 2,279,048 rows/600 st · rainfall 42,745/50 st/8 dist · temperature 107,950/5 · humidity 118,947/6 · solar 109,555/6 · wind_speed 122,527/6 · pressure 110,949/4 · river_level 295,302/6/2 · canal_level 88,089/17/1.

5.2 Spatial association

- **Rainfall:** IDW (`1/d²`) over 3 nearest gauges within `RAIN_RADIUS_KM=150`.
- **Weather/river/canal:** nearest gauge within radius (`WEATHER_RADIUS_KM=150`, `RIVER_RADIUS_KM=100`, `CANAL_RADIUS_KM=100`); river/canal add `dist_km`.
- **River (no published coords):** district-level proxy (mean across district gauges), flagged `dist_km=NaN`.

5.3 Preprocessing / hygiene steps

1. **Sentinel & spike drop** (`03`): `|GWL|>500 m` and implausible records dropped; per-station **0.5–99.5 % quantile trim**. Manifest counts e.g. gwl dropped 20,273 rows by caps; rainfall max 149.5 mm/h (cap `MAX_RAIN_MM_H` = 150 — a real gauge once reported 2.24e6 mm/h).

2. **Daily/6h median:** daily-median GWL for correlation; 6h-median per slot (`04b`) for modelling.
3. **Spike mask** (`06_features.drop_gwl_spikes`): per-station robust mask, drop readings `> 30 × (1.4826 × MAD)` from the median → set to NaN.
4. **Regime-shift drop** (`06_features.flag_regime_shift`): stations whose per-year median jumps `> 15 m` from the 2021–2024 baseline are dropped wholesale (telemetry datum errors, e.g. −6 → −97 m overnight wells; 51 dropped on the 6h grid).
5. **Coverage selection** (`01_select`): full-2026 rule — first 2026 record ≤ `2026-01-15`, ≥ 8 of 9 months (Jan–Sep 2026) have a record, last record ≥ `2026-08-01` → **600 of 1,353 stations / 29 districts**.
6. **Feature/target build** (`06_features`): lags/rollings always causal, per-sample 6h bins, cyclic calendar encodings.
7. Static ISRIC soil per-station texture (`07_soil`) and Bhuvan LULC district stats (`08_lulc`) are fetched/app-visible but **gated out** of the model. CFSv2 seasonal rain (`09_cfs_rain.py`) was **removed** (NCEI gridded services returned S3-403; Open-Meteo replaced the driver feed).

---

6. ML models — detail

6.1 Task

Pooled **global** model (one model, not per-station). Grid 6-hourly, horizon **30 days = 120 steps**. Target **delta**: `y = GWL(t+120) − GWL(t)`. Level = `anchor + delta`. Persistence baseline = 0-change. `gwl` at t is anchor — never a target.

6.2 Models & hyperparameters (from `06_train.py` + `11_quantile.py`)

XGBoost regressor (`xgb_multihorizon.joblib`, `reg:squarederror`): - `n_estimators=1500`, `max_depth=6`, `learning_rate=0.05`, `subsample=0.8`, `colsample_bytree=0.8`, `min_child_weight=20`, `tree_method="hist"`, `objective=reg:squarederror`, `eval_metric=rmse`, `random_state=42`, `n_jobs=6` - Early stopping on validation split, `early_stopping_rounds=60` - Trained: **best_iteration n_trees = 15** (per `feature_config.json`) - Input matrix: `NUM_COLS(30) + st_id + dist_id` - Train rows *sampled 1 slot/day per station* (`cumcount % 4 == 0`) for memory (Ridge dense solve 7.5 GB→1.9 GB)

Ridge (`linear_multihorizon.joblib`, ridge-as-model): - `RidgeCV(alphas=(0.1, 1.0, 10.0, 100.0))` → chosen `alpha = 0.1` - Pipeline: `ColumnTransformer` = numeric branch (`SimpleImputer(median)` → `StandardScaler`) over `NUM_COLS` + categorical `OneHotEncoder(handle_unknown="ignore")` over `["Station","District"]`

Quantile forecasters (`xgb_q05/q50/q95.joblib`, `reg:quantileerror`): - Same pooled XGBoost recipe at α = 0.05 / 0.50 / 0.95, output delta quantiles. - Calibrated empirically on non-overlap windows → `quantile_calibration.json` (below).

6.3 Split & sizes (`feature_config.json` / `model_metadata.json`)

- Train: `< 2025-10-01` — val: `2025-10-01 .. 2025-12-31` — test: `>= 2026-01-01`
- `train_rows` 1,629,574 · `val_rows` 11,647 · `test_rows` 378,554 (model_metadata: train_val_rows 1,641,221, n_stations 549, n_districts 29)
- `trained_at: 2026-09-09 17:03:49`

6.4 Validation metrics (all 30-day horizon)

Head-to-head on 2026 held-out — `outputs/model_metrics.csv` (level basis):

Model	RMSE	MAE	R²	±0.5 m	±1.0 m
xgboost	**2.242**	0.704	0.901	67.2%	85.1%
persistence	2.290	0.744	0.897	63.9%	83.2%
ridge	2.456	1.260	0.881	33.3%	57.9%
climatology	3.572	1.657	0.748	32.7%	54.6%

Honest (overlap-aware) re-score — `outputs/honest_metrics.csv` & `overlap.json`: 378,554 raw test rows are ~99% overlapping (120-step windows on a 6h grid → `effective_N_ratio = 0.0089` → **3,371 independent stride windows**; effective sample ≈ 6,635 from lag-1 ACF 0.858).

Model	raw RMSE	stride RMSE	window med RMSE
xgb	2.2418	**2.3389**	0.4548
ridge	2.4559	2.5123	0.9315
persist	2.2896	2.3824	0.4932
clim	3.5716	3.6225	1.0291

> XGBoost beats persistence by **+1.8%** on independent windows (raw +2.1%). Per-row RMSE deltas below ~±0.05 m are within noise.

Spatial CV — `outputs/spatial_cv_metrics.csv` + `spatial_cv_summary.json`: leave-one-block-out station retraining, 5 folds: fold RMSE mean **1.9733** / median **1.8191** (blocks: 140/129/163/106/62 stations). Temporal 2026 holdout, not spatial leakage, is the binding constraint. Residual: Moran's I = 0.042 (p=0.051), Geary's C = 1.083 (ns) — no strong spatial clustering.

Quantile calibration — `quantile_calibration.json`: `target_coverage_stride 0.8`, `widen_factor_k 1.0`, `coverage_stride 0.908` (raw 0.9003), `half_width_median_m 1.025`, `half_width_p90_m 2.575`, `half_width_mean_m 1.376`. Anchor basis: `level = today's GWL + quantile(delta 30d)`.

Regression gate (`ml/gate_check.py`, 12 checks; latest run GATE PASS 10/10): honest stride RMSE 2.339 · spatial CV mean 1.973 / median 1.819 · coverage 0.908 · half-width median 1.025 / p90 2.575 · eff-N 6,635 · lag1 ACF 0.858 · trajectory 30d 2.106 (promoted) · +30d calibrated coverage 0.90. (The 2 recursive-model checks skip — the `backtest_6h_*` artifacts were removed.)

---

7. Prediction input/output schema

7.1 Forward forecast — `_model.forward_forecast(station: str) -> dict`

Rebuilds the feature frame for the station (via `06_features.build_full`), takes the **last row** (latest time), scores with XGBoost (32 cols), Ridge (30 num + Station + District), and the q05/q50/q95 boosters; quantile interval calibrated with `k = widen_factor_k`.

Real example (live output):


```
{
  "station": "Ramchhitoni Sahawar (UP-011)",
  "date_from": "2026-09-05 00:00:00",
  "date_to": "2026-10-05 00:00:00",
  "anchor": -2.841,
  "pred_xgb": 0.121,
  "pred_ridge": -0.58,
  "xgb_level": -2.72,
  "ridge_level": -3.421,
  "q05_level": -4.266,
  "q50_level": -2.691,
  "q95_level": -1.039,
  "widen_k": 1.0,
  "band_half": 1.613,
  "station_stride_rmse": 0.314
}
```


Field	Meaning
`date_from`	anchor timestamp (latest 6h slot)
`date_to`	`date_from + 30 days`
`anchor`	latest observed GWL (KPI, never predicted)

Field	Meaning
`pred_xgb` / `pred_ridge`	predicted 30-day *change* (delta)
`xgb_level` / `ridge_level`	level = anchor + change
`q05_level / q50_level / q95_level`	calibrated 90% interval on level
`band_half`	`(q95 − q05) / 2`
`station_stride_rmse`	honest non-overlap RMSE for this station (if in test set)

Returns `{}` if the station has no usable feature frame.

7.2 2026 backtest loader — `_model.load_predictions()` (returns DataFrame)

Reads only 10 columns from `predictions_2026.parquet` (TTL 24 h): `Station, District, date, time, target, gwl, xgb, ridge, q05_lvl, q95_lvl`.

7.3 Feature config — `load_feature_config()` (feature_config.json) keys

`model_type, target, horizons[30], num_cols[30], stations[549], districts[29], train_rows, val_rows, test_rows, xgb:{n_trees,val_rmse_delta,val_rmse_level}, ridge:{alpha,val_rmse_delta,...}, trained_at`.

7.4 Model metadata — `model_metadata.json` keys

`generated_by, architecture, target, horizons, n_stations, n_districts, train_val_rows, test_rows, val_rmse_level_m:{xgb,ridge}, split, trained_at, generated_at, source_archive, validated:{quantile_calibration, honest_metrics, spatial_cv, spatial_cv_summary}`.

---

8. API-style specification (per ML functionality)

Two services: - **Node** `:3000` (live): `/stations`, `/telemetry`, `/assessments`, `/trends`, `/ml-data`, `/data-quality`, `/ingestion` — full list in §14. - **Python** `:5000` **Flask** (`ml/api.py`, live) — surface below. The Node gateway (`backend/services/mlGateway.js`) forwards path+query to `ML_SERVICE_URL`. Status codes: 200 / 400 / 404 / 422 / 500 / 502 / 503 (`{ "error": …, "detail": … }`).

Endpoint	Method	Input	Response (shape)	Backed by
`/`	GET	—	`{service, version, status, endpoints, usage}`	hardcoded index
`/health`	GET	—	`{status, service, version, stations, dataset_last, ollama:{server,models,model}, forecast_model}`	`_data()`, `_index()`, `_assistant.ollama_status()`
`/stations`	GET	`?district=&q=&limit=`	`{count, stations:[{station, district, slug, latitude, longitude, last_ts,…}]}`	`selected_gwl_stations.csv`, `_index()`
`/forecast/<slug>`	GET	slug (URL-encoded)	120×6h trajectory `time/q05/q50/q95` + confidence + drivers	`_trajectory.trajectory_forecast()`
`/assistant/chat`	POST	`{question, station?, model?}`	`{answer, facts, station}`	`_assistant.StationAssistant.answer()`

Planned additions (not yet implemented): `/stations/:slug` (per-station facts), `/districts`, `/models`, `/fleet/forecasts`, `/fleet/recovery`, `/fleet/scan`.

Slugs vs names: API uses lowercase hyphenated slugs (e.g. `ashadha-prathmik-vidyalaya`); always fetch the slug from `GET /stations` — never hand-type it. The exact station name also resolves as a fallback when URL-encoded. Internally the model uses the raw station name string (e.g. `"Ramchhitoni Sahawar (UP-011)"`).

Errors: - 400 — e.g. chat without `question`, or `days` out of 7–90 - 404 — unknown slug / no model trained / fleet snapshot missing - 500 — internal Python error - 502 — ML service down/`ML_SERVICE_URL` unreachable - 503 — assistant unavailable (Ollama not running) - Legacy `/ml/forecast/:stationId`, `/ml/risk/:unitId`, `/ml/anomalies` — **deprecated**, DB registry only.

---

9. Forecasting complete logic

1. **Anchor** = latest observed GWL for the station (`date_from` = last 6h slot; never predicted). If no feature frame → empty response.
2. **Feature rebuild** — same path as training: filter `table_6h` by station → `06_features.drop_gwl_spikes` → `build_full(keep_na=True)` → last row.
3. **Score**: - `pred_xgb = xgb_b.predict(last[feature_names])` — delta - `pred_ridge = linear_b.predict(last[NUM_COLS + [Station, District]])` - quantiles: `med = anchor + q50`, `lo = anchor + q05`, `hi = anchor + q95`; calibrated `q05 = med − k(med−lo)`, `q95 = med + k(hi−med)`, `k=1.0` - `band_half = (q95 − q05)/2`
4. **Interval semantics:** 90% calibrated interval (`coverage_stride = 0.911 ≥ 0.80`). Median half-width ≈ **1.03 m** (p90 ≈ 2.58 m).
5. **Level outputs** = `anchor + delta`. **Persistence** = unchanged anchor.
6. **Fleet scan (`11_fleet.py`)**: eligible = ≥ 2000 obs and ≥ 730-day span (596 stations). Per station `forward_forecast()`; headline = `q50 − anchor`; keep `change_pooled_m` as secondary (pooled delta saturates to identical extremes on stations with empty recent lag/driver columns). Thresholds: decline ≤ −0.3 m, high decline ≤ −0.6 m, recovery ≥ +0.3 m. Categories: `decline (high) / decline / stable / recovering / unknown / unreliable`. **Quality gate →** `unreliable` if: NaN anchor, `|change| > 12 m`, predicted level outside `[obs_min − 25, obs_max + 25]`, or anchor older than 45 days. Snapshot result (2026-09): **502 scored**, median Δ **+0.32 m**, decline **0** / recovering **259** at ±0.3 m — nearly all changes inside their 90% interval (post-monsoon recovery).

---

10. Groundwater impact analysis

Available in `ml/outputs/`:

1. **Feature importance (XGBoost gain)** — `feature_importance.csv` (`feature, gain, freq`). Top: `monsoon`, `gwl_roll7_std`, `doy_sin`, `gwl_roll30_std`, `year`, `doy_cos`, `lag1`, `gwl`. (§2 table.)
2. **Ridge coefficients** — `feature_coefficients.csv` (`feature, coef, abs_coef`). Top |coef|: `num__gwl −9.73`, `num__lag4 −0.35`, `num__lag8 −0.30`.
3. **Correlation report** — `correlation_report.csv` (`driver, metric(spearman/pearson), mode(raw/deseason/diff), corr, stations_ok`). Headlines (|spearman| raw): `river_level +0.409` (deseason 0.168) · `pressure +0.199` · `rain_30d −0.199` · `canal_level −0.146` · `humidity +0.085` · `solar −0.055` · `temp −0.041` · `wind_speed −0.095`. **Recharge lag:** best GWL-vs-rainfall lag ≈ **22 days** (Pearson median +0.136, min 180 valid days) — `lag_curves.csv`.
4. **Ablation** — `ablation.csv` (drop-one-group retrain): rain (+0.03), weather (+0.05), calendar (+0.05) add marginal RMSE when dropped; river/canal (−0.005) and GWL lags (≈0) neutral given anchor + calendar; all within overlap noise ~±0.05 m.
5. **Diagnostics** — `diagnostics.json` + `diagnostics_permutation.csv` (stride subset, n=3,371): - **VIF:** nothing > 10; `constant_excluded: [year]`; no collinearity. - **Permutation importance (Δ RMSE):** ~all ≈ 0 (top `gwl_roll30_std` +0.0026, `temp_7d` +0.0016; most below zero-noise). - **Sensitivity (OAT, 2.5→97.5 pct):** max swing `doy_cos` ≈ **0.385 m**, then `gwl` 0.148, `gwl_roll30_std` 0.146, `monsoon` 0.132 — season/level/ monsoon carry the largest single-driver influence.
6. **SHAP/explainability: not implemented** in this codebase (no `shap` dependency). Grounded explanation today = XGBoost gain + Ridge coef + permutation importance + OAT sensitivity + Spearman correlation. Adding SHAP is recommended as a follow-up on the stride subset (32 features → `shap.TreeExplainer`).

---

11. Geography hierarchy & IDs

India → Uttar Pradesh → District → Tehsil → Block → Station.

- The archive has 1,353 UP stations across 34 of UP's ~75 districts; only **600 stations / 29 districts** pass the full-2026 rule and enter the model.
- `selected_gwl_stations.csv` gives per station: `Station, District, Tehsil, Block, Latitude, Longitude, n_2026_months, n_2026_records, first_2026, last_record, full26`. Note: Tehsil/Block are `-` (not published) for most telemetry stations.
- **Model ordinals** — `st_id` (sorted station names, 0..548) and `dist_id` (sorted district names, 0..28) are fit on the combined train/val/test station set and reused in the app so forward-forecast shares byte-identical columns.
- Backend stations table keeps the same identity columns (`state, district, rehsil… tehsil, block, village, river, basin, latitude, longitude, rl_msl`, LGD codes).
- Two endpoint-visible geo keys: station **slug** (planned API) and raw **Station name** (models/fleet).

---

12. Streamlit app — live features & purpose

`ml/app.py` (port 8501), 4 pages via `st.navigation` (+ Verification, unwired):

Page	File	Purpose / content
Assistant	`app_pages/assistant.py`	Station-pinned chat (default page); instant-facts panel works even if Ollama is down; Ollama status probe
Correlation	`app_pages/correlation.py`	Driver×metric×mode pickers, recharge-lag curve (GWL vs rainfall), static-feature (soil/LULC) vs GWL summary
Forecast	`app_pages/forecast.py`	Single dark-theme trajectory v2 card: 120 genuine 6-hourly points, observed tail, "Forecast starts" boundary, bright q05/q95 band (no dot markers), direction banner, 6 metric cards, freshness caption
Sources	`app_pages/data_sources.py`	Per-source manifest quality (rows/stations/districts/dropped), empty-source table, static hydrogeology (soil/LULC status), association method + radius
Verification *(not in nav)*	`app_pages/verification.py`	Realised-forecast quality from `data/refresh/verification_summary.json` + sign-accuracy ledger; wire into `app.py` to enable

> The legacy explorer pages (Overview, Drivers, Stations, Model, Fleet) were **removed** from `app_pages/`; their pipeline outputs (`fleet_forecast.csv`, `model_metrics.csv`, honest/ablation/diagnostics) are still produced by the numbered scripts and served through `_model.py` loaders, the Flask API and the assistant facts.

---

13. Chatbot — flow, inputs, outputs, prompt

Implemented in `ml/_assistant.py` (class `StationAssistant`), page `app_pages/assistant.py`.

Design principle: station-locked, deterministic facts only — the LLM phrases precomputed numbers and never invents values.

Flow: 1. `facts(station)` → builds a fact dict (§13.1). 2. If no telemetry → `{"answer": "No telemetry found for station 'X'.", "facts": {}, "station": X}`. 3. Else `_build_prompt(question, facts, history)` → 4. `_invoke_llm` → `ollama.Client(timeout=None).chat(model, messages=[{role:"user", content:prompt}], options={"temperature": 0.2})` → 5. returns `{"answer", "facts", "station"}`.

13.1 Facts dict (station level): `level:"station", station, district, last, last_date, min, max, span, n_obs, outliers, change_7d, change_30d, change_60d, change_180d, district_median, district_n_stations, forecast:{anchor, day30_pred, change_30d_pred, direction("expected rise"|"expected decline"), plausible, obs_min, obs_max, band_half, q05_level, q95_level, station_stride_rmse, high_uncertainty}`.

13.2 Prompt template (verbatim structure):

```
You are the AQUIS groundwater assistant. Answer the user's question using
ONLY the given observed facts. No speculation. Be concise (max ~6 lines).
Use metres (m) for levels/changes. State dates where known.
Never refuse or say data is unavailable…
[CONVERSATION history, "always trust the LATEST FACTS" if numbers differ]
FACTS:
station: …  district: …
latest level: <last> m on <last_date>
range: <min> to <max> m (span <span> m, <n_obs> observations)
[note: N outlier(s) excluded]
level change: 7d=… | 30d=… | 60d=… | 180d=… m
model forecast (+30 d): level=<day30_pred> m, change 30d=<…> m (<direction>); 90% band +/- <band_half> m (anchor
<anchor> m); q05/q95 interval …m..…m
[note: model UNSTABLE … or high-uncertainty station … where flagged]
district median (<district>): <median> m across <n> stations
QUESTION: <question>
ANSWER:
```

13.3 Quality notes fed to prompt: - `plausible = |change| ≤ 12 m AND day30 ∈ [obs_min−25, obs_max+25]`; else a "runaway" note tells the LLM to trust only observed facts. - `high_uncertainty` when `station_stride_rmse > 2 × fleet median (xgb_stride_rmse`.

13.4 LLM setup: local Ollama, default model `llama3.2:3b` (`ollama list` probe first); override env `AQUIS_OLLAMA_MODEL` (from repo `.env` or environment). Server down → facts panel still renders; page shows an Ollama status warning.

---

14. Database design (PostgreSQL) + suggested tables

14.1 Existing backend schema (`back-end/db/schema.sql`, 9 tables)

- `ingestion_runs` — `id, source, source_ref, status(enum running/completed/ failed/partial), started_at, finished_at, records_seen/inserted/updated/ rejected/duplicates, error_count, error_summary JSONB, metadata JSONB`
- `stations` — `id, external_station_id UNIQUE, station_name, agency, state, state_lgd_code, district, district_lgd_code, tehsil, block, village, river, basin, tributary, subtributary, sub_subtributary, local_river, latitude, longitude, rl_msl, first_observed_at, last_observed_at, observation_count, timestamps` + indexes on state/district/agency/coords/external
- `telemetry_observations` — `id, station_id FK→stations, observed_at, groundwater_level, source(default 'nwdp_api'), source_record_id, ingestion_run_id FK, created_at`; unique `(station_id, observed_at)`
- `assessment_units` / `assessment_records` — CGWB assessment geography + multi-year records
- `data_quality` — per-station/telemetry/assessment quality issues
- `model_metadata` / `model_outputs` — DB model registry (legacy; the live ML stack uses `ml/models/*.json` instead)
- `groundwater_data` — legacy GWL table

14.2 Suggested additions for the ML-backed app

Table/collection	Key fields	Relation
`ml_stations` (mirror of `selected_gwl_stations.csv`)	station(PK name), district, tehsil, block, lat, lon, n_2026_months, n_2026_records, first_2026, last_record, full26	1:N telemetry
`ml_features` (optional, parquet mirrors)	station, time(6h), 30 features, st_id, dist_id	FK station
`ml_predictions` (from `predictions_2026.parquet`)	station, time, horizon, target, gwl, xgb, ridge, persist, clim, err_*, q05_lvl, q95_lvl, step_idx, window_id, stride	FK station
`ml_fleet_snapshot` (from `fleet_forecast.csv`)	station, district, anchor, date_from, age_days, anchor_valid, xgb_level, q05/ q50/q95_level, change_30d_m, change_pooled_m, band_half_m, plausible, category	FK station
`ml_quantile_calibration` / `ml_model_meta`	mirrors `quantile_calibration.json` / `model_metadata.json`	singleton

Table/collection	Key fields	Relation
`chat_logs`	id, station, question, answer, facts JSONB, model, created_at	optional FK station
`alert_rules` / `alerts` (future)	station, threshold_m, band_ref, status, created_at, triggered_at	FK station

15. ML project folder / file structure (with purpose)

```
ml/
├─ app.py                        Streamlit entry — st.navigation of 4 pages, :8501
├─ app_pages/{assistant,correlation,forecast,data_sources}.py   live pages (§12)
│             + verification.py (refresh verification UI, not in nav)
├─ config.py                     SOURCES (archive/live resource ids), radii, coverage
│                                rules (FULL26_*), caps (MAX_RAIN_MM_H), field names, paths
├─ 00_probe.py                   probe every NWIC resource (archive + live) → meta/probe.json
├─ 01_select.py                  full-2026 coverage selection → 600 st / 29 dist
├─ 02_fetch_selected.py          per-district driver fetch, resume-safe
├─ 03_merge_normalize.py         plausibility caps → raw/<source>_norm.parquet + manifest
├─ 04_align.py                   daily station×day, spatial association (IDW rain, nearest weather)
├─ 04b_align_6h.py               6-hourly station×slot table (model track)
├─ 05_correlate.py               correlation report + recharge-lag curves/charts
├─ 06_features.py                spike mask, regime-shift drop, feature/target build → features/*.parquet
├─ 06_train.py                   pooled XGBoost + Ridge, deltas, joblib + feature_config/metadata
├─ 07_evaluate.py                4-way benchmark (XGB/Ridge/persistence/climatology) + honest re-score
├─ 07_soil.py                    ISRIC SoilGrids v2 per-station texture fetch
├─ 08_lulc.py                    ISRO Bhuvan LULC 50K district stats
├─ 10_ablate.py                  drop-group ablation sweep → outputs/ablation.csv
├─ 11_quantile.py                pooled q05/q50/q95 + empirical coverage calibration
├─ 11_fleet.py                   whole-fleet 30-day scan + recovery ranking
├─ 12_diagnostics.py             VIF + permutation importance + OAT sensitivity
├─ 13_refresh_nwic.py            incremental NWIC refresh (+ --retrain / --deploy-check)
├─ _model.py                     model/output loaders + forward_forecast() (§7)
├─ _utils.py                     shared loaders: manifest, selected CSV, table/table_6h,
│                                station_recency() (cached groupby), label maps
├─ _assistant.py                 StationAssistant facts + prompt + Ollama client (§13)
├─ _soil.py / _lulc.py           static soil / LULC loaders
├─ validation/                   P0 honesty suite: spatial_cv.py, overlap.py,
│                                residual_acf.py, spatial_residual.py, _spatial_folds.py
├─ gate_check.py                 12-check regression gate (run after edits; latest GATE PASS 10/10)
├─ tests/                        stdlib unittest (data-gated, no pytest)
├─ MODEL_CARD.md                 model lifecycle card
├─ README.md                     module doc + decisions log
├─ data/processed/common.parquet cleaned 6-hourly GWL archive (source of truth)
├─ data/selected|raw|aligned|meta|features|soil   intermediate + meta dirs
├─ models/*.joblib + *.json      xgb/linear/q05/q50/q95 + feature_config/quantile_calibration/model_metadata
└─ outputs/*.csv|json|parquet    evaluation, fleet, diagnostics, predictions
```

16. Dependencies, env vars, external APIs

16.1 requirements.txt (repo root — used by Streamlit Cloud)

```
pandas  numpy  matplotlib  scikit-learn  requests  flask  flask-cors
scipy  xgboost  joblib  streamlit  plotly
```

App additionally imports at runtime (present in the ml venv): `altair` (Streamlit bundles it), `ollama` (Python SDK, via `/opt/venv` — **not in root requirements; add `ollama` if deploying the assistant**). Python version: **3.14** (ml venv, pipelines + app tested); backend needs Node 18+/PostgreSQL 14+.

16.2 Environment variables

Variable	Default	Used by
`AQUIS_OLLAMA_MODEL`	`llama3.2:3b`	assistant model override (`_assistant.py`)
`LULC_STATISTICS_API_KEY`	—	Bhuvan LULC district stats (`08_lulc.py`)

Variable	Default	Used by
`LULC_AOI_WISE_API_KEY`	(in example)	Bhuvan AOI endpoint (unused)
`DATABASE_URL`	—	backend PostgreSQL
`PORT`	3000	backend
`ML_SERVICE_URL`	`http://localhost:5000`	backend ML gateway (planned)
`ML_TIMEOUT_MS`	60000	backend ML timeout
`BACKEND_URL`	`http://localhost:3000`	some ML helper scripts

Env file pattern: plain `KEY=VALUE` lines in repo `.env`, parsed by `_assistant.read_env(repo)` / `08_lulc` (same pattern).

16.3 External APIs

- **NWIC/NWDP CKAN** `datastore_search` — all telemetry (GWL + drivers); ids in `config.py`; authoritative map `back-end/db/api/api.txt`.
- **ISRO Bhuvan LULC statistics** (`bhuvan-app1.nrsc.gov.in/api`) — district class shares; key in `.env`.
- **ISRIC SoilGrids v2 REST** — per-station sand/silt/clay/bdod; background fetch.
- **Ollama (local)** — `llama3.2:3b` ~2 GB, `ollama serve`; not an external SaaS.
- **CGWB assessments** — Excel files (imported via backend), manual/quarterly UPGW dataset documented in `MODEL_CARD.md` (not ingested into ML).
- NCEI CFSv2 seasonal rain was **removed** (`09_cfs_rain.py` deleted — NCEI gridded services returned S3-403); Open-Meteo is the driver feed.

---

17. Validation rules & edge cases

17.1 Data-quality gates

- Coverage rule: first 2026 ≤ 2026-01-15, ≥ 8/9 months (Jan–Sep) present, last ≥ 2026-08-01 → else station excluded (can't forecast).
- Sentinels: `|GWL| > 500 m` dropped; per-station 0.5–99.5% trim.
- Rainfall hourly cap 150 mm/h; caps per-source via manifest.
- `drop_gwl_spikes`: > 30×(1.4826·MAD) deviation → NaN.
- Regime shift: per-year median jump > 15 m from 2021-24 baseline → station dropped.
- Fleet quality gate → `unreliable`: NaN anchor, `|Δ|>12 m`, level outside `[obs_min−25, obs_max+25]`, or anchor age > 45 days.

17.2 Edge cases / errors (exact behaviours)

- `forward_forecast(station)` with no feature frame → returns `{}` (app shows "No feature frame for this station.").
- Assistant with unknown station → `answer: "No telemetry found for station 'X'."`.
- Fleet with no snapshot → page shows warning "run `ml/11_fleet.py` first" + `st.stop()`.
- Ollama down → chat raises; page surfaces status; facts panel still renders.
- Missing driver columns → feature builder skips those windows (conditionals), XGBoost handles NaN, Ridge median-imputes.
- Static soil/lulc incomplete → skipped with a printed note; app shows "no profile mapped" caption.
- Ridge on new station not in OneHotEncoder → `handle_unknown="ignore"` (all-zero category vector).
- Stale model vs live NWIC → `13_refresh_nwic.py --deploy-check` verifies the loaded build is 2026-current.

17.3 Honest-validation rules (don't overclaim)

- Never quote per-row RMSE deltas < ~±0.05 m as signal (overlap noise).
- Use stride (non-overlap) metrics for claims: XGBoost 2.339 vs persistence 2.382 m, coverage 0.908 on 3,371 windows, eff-N 6,635.

- Significance of a fleet move: |Δ| must exceed the 90% band half-width.

---

18. Sample request–response examples

18.1 Forecast (app/API)

```
// GET /ml/forecast/Ramchhitoni%20Sahawar%20(UP-011)?days=30
{
  "station": "Ramchhitoni Sahawar (UP-011)",
  "date_from": "2026-09-05T00:00:00",
  "date_to": "2026-10-05T00:00:00",
  "anchor": -2.841,
  "pred_xgb": 0.121,
  "pred_ridge": -0.58,
  "xgb_level": -2.72,
  "ridge_level": -3.421,
  "q05_level": -4.266,
  "q50_level": -2.691,
  "q95_level": -1.039,
  "widen_k": 1.0,
  "band_half": 1.613,
  "station_stride_rmse": 0.314
}
```

18.2 Fleet scan

```
// GET /ml/fleet/forecasts
{
  "horizon_days": 30,
  "decline_threshold_m": 0.3,
  "eligible_stations": 596,
  "scanned_stations": 502,
  "generated_at": "2026-09-09 …",
  "stations": [
    {"station": "Mahgaon (UP-031)", "district": "KAUSHAMBI", "anchor": -17.171,
     "date_from": "2026-09-03 18:00:00", "age_days": 6, "anchor_valid": true,
     "xgb_level": -16.881, "q05_level": -18.637, "q50_level": -16.689,
     "q95_level": -11.989, "change_30d_m": 0.482, "change_pooled_m": 0.29,
     "band_half_m": 3.324, "plausible": true, "category": "recovering"}
  ]
}
```

18.3 Impact analysis

```
// GET /ml/models  (subset)
{
  "n_stations": 549, "n_districts": 29,
  "val_rmse_level_m": {"xgb": 1.2748, "ridge": 1.708},
  "validated": {
    "honest_metrics": [{"model": "xgb", "basis_stride_rmse": 2.3389, "n_windows": 3371}],
    "spatial_cv_summary": {"fold_rmse_mean": 1.9733, "fold_rmse_median": 1.8191},
    "quantile_calibration": {"coverage_stride": 0.908, "half_width_median_m": 1.025}
  }
}
```

18.4 Chatbot

```
// POST /ml/assistant/chat
{ "question": "Is level rising?", "station": "Ramchhitoni Sahawar (UP-011)" }

// 200
{
  "answer": "Latest level is -2.84 m on 2026-09-05. Over 180 days the level changed -0.06 m. The model expects a rise
of about +0.12 m (to -2.72 m) over the next 30 days, inside a 90% band of +/- 1.61 m.",
  "facts": {
    "station": "Ramchhitoni Sahawar (UP-011)", "district": "KANSIRAM NAGAR",
    "last": -2.841, "last_date": "2026-09-05", "min": …, "max": …,
    "change_7d": …, "change_30d": …, "change_60d": …, "change_180d": …,
    "district_median": …, "district_n_stations": …,
    "forecast": { "anchor": -2.841, "day30_pred": -2.72, "change_30d_pred": 0.121,
      "direction": "expected rise", "plausible": true, "band_half": 1.613,
      "q05_level": -4.266, "q95_level": -1.039, "station_stride_rmse": 0.314,
      "high_uncertainty": false }
  },
  "station": "Ramchhitoni Sahawar (UP-011)"
}
```

---

19. Sample dataset (real rows)

`outputs/predictions_2026.parquet` (station ASHADHA PRATHMIK VIDYALAYA, KAUSHAMBI):

date	time	target	gwl	xgb	ridge	q05_lvl	q95_lvl
2026-01-01	2026-01-01 00:00:00	-6.267	-6.411	-6.624	-7.896	-7.835	-4.558
2026-01-01	2026-01-01 06:00:00	-6.414	-6.165	-6.378	-7.720	-7.589	-4.673
2026-01-01	2026-01-01 12:00:00	-6.156	-6.284	-6.497	-7.721	-7.708	-4.856

`outputs/fleet_forecast.csv` (subset):

station	district	anchor	date_from	age_days	anchor_valid	xgb_level	q05_level	q50_level	q95_level	change_30d_m
Ramchhitoni Sahawar (UP-011)	KANSIRAM NAGAR	-2.841	2026-09-05 00:00:00	4	True	-2.72	-4.266	-2.691	-1.039	0.15
Mahgaon (UP-031)	KAUSHAMBI	-17.171	2026-09-03 18:00:00	6	True	-16.881	-18.637	-16.689	-11.989	0.482

`data/meta/selected_gwl_stations.csv` (subset):

Station	District	Tehsil	Block	Latitude	Longitude	n_2026_months	n_2026_records	first_2026	last_record	full26
ASHADHA PRATHMIK VIDYALAYA	KAUSHAMBI	-	-	25.441184	81.392245	9	822	2026-01-01 00:00:00	2026-09-05 18:00:00	True
Aagan Wadi Kendra Chandni	JALAUN	-	-	25.929918	79.141493	9	900	2026-01-02 00:00:00	2026-09-05 18:00:00	True

`outputs/station_summary.csv` (per-station honest RMSE):

station	district	n_windows	stride_rows	raw_rows	xgb_raw_rmse	xgb_stride_rmse	ridge_raw_rmse	ridge_stride_rmse
ASHADHA PRATHMIK VIDYALAYA	KAUSHAMBI	7	7	754	0.457	0.597	1.786	1.698

---

20. Feature-to-API mapping (build plan)

UI / applet	Model feature(s)	Endpoint(s) (planned)	Data/artifact today
Health banner / boot check	—	`GET /health`	`model_metadata.json`, `quantile_calibration.json`, `_assistant.ollama_status()`
Station picker / watchlist (recency-sorted)	`gwl` anchor	`GET /stations?q=&district=`	`selected_gwl_stations.csv` + `station_recency()` (table_6h groupby)
Station detail KPIs (level, min/max, changes 7/30/60/180 d)	`gwl`, lags	`GET /stations/:slug`, `GET /telemetry/:stationId`	`table_6h.parquet`, backend `telemetry_observations`
Price-style GWL chart + 90% band (Bollinger-like)	`q05_level/q95_level`	`GET /forecast/:slug?days=30`	`predictions_2026.parquet` (q05_lvl/q95_lvl) + `forward_forecast()`
Forecast outlook card / alerts (±band)	`anchor + delta`, `band_half`	`GET /forecast/:slug`	`forward_forecast()`
Indicator overlays (7d/30d MA, rolling-std vol band, rain volume)	`gwl_roll7/30_mean/std`, `rain_1d/7d/30d`	`GET /stations/:slug` (extended)	`table_6h.parquet`
Seasonal climatology overlay + monsoon shading	`doy_sin/cos`, `month_sin/cos`, `monsoon`	`GET /telemetry/:stationId?years=`	`table.parquet` + seasonal median
Weather fundamentals panel	`temp(+7d)`, `humidity(+7d)`, `solar`, `wind_speed`, `pressure`, `river_level`, `canal_level`	`GET /stations/:slug?drivers=`	`table_6h.parquet`, `correlation_report.csv`
District / UP heatmap	`dist_id`, fleet Δ	`GET /districts`, `GET /fleet/recovery`	`fleet_district.csv`, `selected_districts.json`
District station comparison (sidebar)	q50 change, band	`GET /fleet/scan?district=`	`fleet_forecast.csv`
Fleet movers (top gainers/losers)	`change_30d_m`, `category`	`GET /fleet/forecasts`	`fleet_forecast.csv` / `fleet_forecast_snapshot.json`
Model / metrics screen	—	`GET /models`	`model_metrics.csv`, `honest_metrics.csv`, `feature_importance.csv`, `feature_coefficients.csv`, `spatial_cv_summary.json`, `diagnostics.json`
Impact analysis tab	gain/coef/permutation/OAT	`GET /models` (+ `?analysis=importance`)	`feature_importance.csv`, `feature_coefficients.csv`, `diagnostics.json`, `correlation_report.csv`
Assistant chat	station facts + forecast	`POST /assistant/chat`	`_assistant.StationAssistant`
Sources / provenance page	—	`GET /models`	`manifest.json`, `config.py` SOURCES

Generated from actual code (`ml/06_features.py`, `ml/06_train.py`, `ml/_model.py`, `ml/_assistant.py`, `ml/11_fleet.py`, `ml/config.py`), trained artifacts (`ml/models/*.json`), and produced outputs (`ml/outputs/*`).

9 Trajectory v2 — Forecast Engine Spec

Trajectory v2 — genuine 6-hourly groundwater forecast (ML-only spec)

Status: **implemented, validated and promoted** (2026-09-10). The production Forecast page + `refresh` daemon + `/forecast/<slug>` API all serve the trajectory engine (see `ml/MODEL_CARD.md`, `ml/README.md`).

Objective (as commissioned)

Build a *genuine* 6-hourly groundwater-level forecast trajectory for the next 30 days (120 steps) per station, expanding the single direct-30d level forecast, with an **explicit, evidence-based confidence/reliability label at every horizon**. Non-negotiables:

- The latest **observed** GWL is the anchor; it is never a prediction.
- No future observed GWL or drivers are used at inference or in backtest.
- 120 genuine model outputs on the 6h grid — no interpolation.
- q05 ≤ q50 ≤ q95 at every horizon; quantile band calibrated to ≥ 90% coverage per horizon.
- Confidence reflects **data/model reliability**, not interval width alone; it may decline with horizon; a missing driver must downgrade it.
- The production direct-30d model stays the numerical +30d benchmark; the trajectory is promoted only on honest backtest evidence.

Decisions locked with the user

Decision	Choice
Architecture	Direct multi-horizon **shared** XGBoost; horizon `h` (1..120) is an input feature
Recursive one-step engine	Rejected (31d-validated: RMSE 1.856 vs persistence 1.840 vs direct 1.083)
Open-Meteo weather	**Past-window driver features only**; also feeds driver-source reliability (≤16d FORECAST / 17–30d CLIMATOLOGY / UNAVAILABLE)
Backtest scope	150 stations (SEED 42) first, then full fleet (549) in the background
UI	Single dark-theme card on the Forecast page (observed tail, "Forecast starts" boundary, q50 + bright q05/q95 band)
Freeze	all round-1 baselines frozen; gate extended to 12 checks

Pipeline (`ml/`)

Step	Script	Output
Datasets	`30_traj_datasets.py`	`data/features_traj/{train,val}.parquet` + `prep_traj.json` (train 6.53M rows, EMA stride 8; targets set by exact-time lookup)
Models	`31_train_traj.py`	`models/traj_xgb_{q05,q50,q95}.json` + `traj_config.json` (33 features incl. h,h_sin,h_cos)
Honest backtest	`32_backtest_traj.py [n_stations]`	`outputs/traj_backtest_metrics.csv`, `traj_backtest_summary.json`, `models/traj_calibration.json`
Reliability tables	`33_traj_reliability.py`	`models/traj_reliability.json` (bucket rules + weights)
Runtime engine	`_trajectory.py`	Forecast page card + `/forecast/<slug>`; unit-tested causal contract
Weather	`20_openmeteo_fetch.py`	`data/cfs/openmeteo_weather_daily.parquet` (37 districts, 365d history + 16d forecast)

Backtest design (honest)

- 150 stations (SEED 42), anchors every 14 days, 2025-10-01 → 2026-08-20 → **non-overlap** forecast windows only.
- Baselines at every horizon: persistence, per-doy 6h-drift climatology, production direct-30d (at +30d; previous recursive 1.856 kept as reference).
- Calibration: bisection widening `s` per horizon to 90% coverage.

Results

Full fleet (549/600 stations, 73,881 scored windows; 150-station sample in parens)

horizon	traj RMSE	persistence	climatology	calibrated cov.	sign-acc
6h	1.147 (0.303)	1.147 (0.303)	1.180 (0.437)	0.90	0.50 (0.47)
12h	0.992 (0.352)	0.993 (0.354)	1.097 (0.712)	0.90	0.64 (0.62)
24h	1.133 (0.348)	1.135 (0.348)	1.468 (1.291)	0.90	0.60 (0.59)
7d	1.157 (0.528)	1.165 (0.538)	6.575 (8.840)	0.90	0.62 (0.63)
15d	1.602 (0.787)	1.631 (0.839)	13.860 (19.102)	0.90	0.71 (0.72)
30d	**2.106 (1.028)**	**2.151 (1.120)**	28.723 (39.332)	**0.90**	**0.75 (0.76)**

- 30d reference compares on the same scope (full fleet): trajectory **2.106** < production direct-30d **2.132** < persistence 2.151. (Previous recursive 1.856 was measured on the 150-station scope only — not directly comparable to fleet-wide numbers.) `promote_trajectory := True`.
- widening `s` ∈ [0.80, 1.28] across horizons; coverage 0.90 bang-on at every horizon.
- Note: 6h–24h sign-acc is ~0.5–0.6 — the band is informative but sub-daily *direction* reads are only DIRECTIONAL by design (dense/noisy stations dominate these horizons).

Confidence framework

Per-point confidence = weighted evidence, NOT interval width alone:

```
score = 0.15·interval_quality + 0.20·perf_vs_persist + 0.20·direction
      + 0.10·width_scaled   + 0.15·station_integrity + 0.10·driver_availability
      + 0.05·anchor_ood     + 0.05·trajectory_stability
```

- HIGH: score ≥ 0.70 ∧ coverage ok ∧ station fresh ∧ driver ≥ forecast ∧ direction supported.
- DIRECTIONAL: score ≥ 0.45 ∧ station ≥ 0.4 ∧ direction supported/marginal.
- LOW: otherwise.
- Inference-time downgrades: stale reading / low recent coverage, missing or climatology-only drivers, anchor outside training range, oscillating trajectory — each carries a reason string.

Gate (12/12 PASS)

New checks: trajectory promoted above persistence+direct30 (`promote_trajectory`), trajectory `+30d` calibrated coverage = 0.90. All round-1 frozen baselines unchanged (2.338 / 1.973 / 1.819 / 0.908 / 1.025 / 6635 / 0.858).

Provenance / leakage safeguards

- Training targets: exact-time lookup (`searchsorted`) because the aligned 6h grid has gaps — never positional shift across gaps.
- Anchor = last observed eligible reading; features rebuilt by `06_features.build_full` (`keep_na=True`, 30d horizon label) exactly like training, with no future observations.
- Backtest uses only windows whose target exists at exact 6h offsets; non-overlap by 14-day anchor spacing > forecast length.

Outstanding

- Sub-daily DIRECTIONAL reads: acceptable, documented; revisit only if a denser objective (e.g., pumpage) ever arrives.

10 Model Card

Model Card — AQUIS groundwater-level forecasting (ml)

Field	Value
Model	Two production forecasts: **(a) trajectory v2** direct multi-horizon XGBoost (`31_train_traj.py` → `models/traj_xgb_q{05,50,95}.json`) for the Forecast page + API; **(b) pooled 30-day XGBoost (delta)** (`06_train.py` → `models/xgb_multihorizon.joblib`) + Ridge baseline for the assistant, fleet scan and +30 d benchmark
Task	Forecast the **30-day trajectory** in groundwater level on a 6-hourly grid: horizon `h ∈ 1..120` (trajectory v2, q05/q50/q95 each step) or the single change `GWL(t+120) − GWL(t)` (pooled)
Package	`ml` — trajectory v2: `30_traj_datasets.py` → `31_train_traj.py` → `32_backtest_traj.py` → `33_traj_reliability.py` ; pooled: `06_features.py` → `06_train.py` → `07_evaluate.py`
Intended use	30-day per-station outlook (Forecast page, `/forecast/<slug>`), whole-fleet scan (`11_fleet.py`), station-locked assistant
Not for	Sub-day decisions, wells outside the 511 test stations, attribution/causal claims

> This card documents the model that powers the forecast surface. Data scope, the pooled-model internals, honest performance and diagnostics below remain the authority for **both** models' inputs; trajectory v2 specifics (training frames, backtest, calibration, confidence framework) are in `ml/README.md` ("Trajectory v2") and `../docs/ml-trajectory-v2-spec.md` .

Data coverage & sources

Scope is determined by the telemetry archive, not by choice. The model consumes *only* the NWIC/NWDP real-time feed — `GWL Telemetry (Six Hourly), Uttar Pradesh Ground Water Department` (UPGW). That feed wires **1,353 stations across 34 of UP's ~75 districts**. UP's remaining groundwater monitoring is **manual/quarterly** and surfaces as *separate* datasets, which this pipeline cannot forecast from in near-real-time.

Coverage facts (all verified against the local archive):

- **1,353 stations / 34 districts** = the full extent of the 6-hourly telemetry network (2021-01-01 → 2026-09-05, ~5.3M raw records in `data/processed/common.parquet`).
- **600 stations / 29 districts** = the subset still "live" in 2026 (`01_select.py` : ≥8 of 9 months Jan–Sep 2026 with a reading, first 2026 record ≤ Jan 15, last record ≥ Aug 1). 5 districts' stations all went stale.
- **34 ≠ scope target**: districts absent from the archive were probed and return `total=0` — e.g. Allahabad/Prayagraj, Amethi, Amroha, Sambhal, G.B. Nagar — even under rename aliases (`ml/data/raw/nwic.py:54`). They have no telemetry gauges at all.

Source material (why most UP groundwater monitoring is manual):

Topic	Link
UPGW datasets on NWDP portal (Manual-Quarterly vs Telemetry-Hourly vs Telemetry-Six-Hourly)	https://www.nwdp.nwic.gov.in/organization/upgw
UPGW "Ground Water Level (Manual - Quarterly)" dataset page (1991–2020)	https://nwdp.nwic.gov.in/dataset/ground-water-level-manual-quarterly-upgw
CGWB monitoring-network page (4×/year manual nationwide; DWLR telemetry is the exception)	https://cgwb.gov.in/en/ground-water-level-monitoring
CGWB Guidelines on High-Frequency GW Data (2026): ~84 k wells nation-wide, ~26 k CGWB (62% dug wells, manual mode), ~39 k state manual stations	https://cgwb.gov.in/sites/default/files/2026-02/final_guidelines_on_high_frequency_gw_data.pdf
CGWB UP Ground Water Year Book 2022–23 (1,007 UP monitoring wells, 4×/yr)	https://cgwb.gov.in/cgwbpnm/public/uploads/documents/17032384351875462524file.pdf

Implication: a station without a live 2026 telemetry stream is not forecast-able here; forecasting scope is UP-GWL *telemetry* only, while the backend's ingestion scope is NWDP-telemetry countrywide + CGWB assessment Excels.

Data

- **Sources**: NWIC 6-hourly observed GWL (600 stations, 29 UP districts, ~2.99M rows in the 6h table), IMD/Open-Meteo reanalysis drivers (rain, temp, humidity, solar, wind, pressure), river/canal level, calendar encodings.
- **Splits**: train 2022–2025 (549 stations), validation (same period, held-out slots), test 2026 (378,554 rows, 511 stations, horizon=30 only).
- **Target**: delta 30-day change. Today's GWL is an **anchor feature** and is never itself predicted.
- **Features (30 numeric)**: gwl + 5 lags (1/4/8/28/120 steps), 2 rolling stats (7/30 d), rain 1/7/30 d, weather drivers, river/canal level, year, calendar (month/doy/hour sin+cos), monsoon flag; plus station id / district id ordinals (32 total model inputs).

Training procedure

- Per-station alignment + spike trimming (0.5–99.5% quantile) in `06_features.py`.
- One random 6-hourly slot per station-day sampled for training (`cumcount() % 4 == 0`).
- XGBoost with early stopping on the validation split (best_iteration ≈ 15, ~few ×100 trees), objectives/ min_child_weight tuned in `06_train.py`.
- Feature matrix materialized to `data/features/{train,val,test}.parquet` (reused by spatial CV, ablation, diagnostics).
- **Trajectory v2:** causal 6h feature frames (`14_6h_features.py` → `data/features_6h/`) are expanded to multi-horizon long-form (`30_traj_datasets.py`, exact-time target lookup, 6.53M rows) and trained as a **shared direct multi-horizon XGBoost** (`31_train_traj.py`, horizon `h` as an input feature, q05/q50/q95 heads). Confidence bucket rules come from `33_traj_reliability.py`.

Performance (honest numbers)

Metric	Value
Raw 2026 test RMSE (all rows)	2.242 m
Non-overlap window RMSE (headline)	**2.339 m** (3,371 independent 30-day windows)
Persistence (non-overlap)	2.382 m → margin **+1.8%**
Spatial CV (leave-block-out, 5 folds)	mean 1.973 m / median 1.819 m
Effective test sample	≈6,635 rows (ACF-based; lag-1 residual ACF 0.858)
Quantile coverage (q05–q95, stride)	0.908 vs target 0.80 → widen factor k=1.0
Interval half-width	median ~1.03 m (1.025), p90 ~2.58 m (2.575, calibrated)
Trajectory v2 30-d RMSE (honest, 2026)	**2.106 m** (73,881 non-overlap windows; < pooled direct-30d 2.132 m < persistence 2.151 m)
Trajectory v2 calibrated coverage	**0.90 at every horizon** (`traj_calibration.json`, `s ∈ [0.80, 1.28]`)

Pre-row deltas below ~±0.05 m are within overlap noise; driver features are additive but marginal at 30 d (ablation, diagnostics).

Diagnostics

- **VIF**: nothing > 10 (no collinearity redline).
- **Permutation importance (stride)**: every feature ≈ 0 Δ RMSE on independent windows; `gwl_roll30_std` + `temp_7d` lead. Seasonal/level features (`doy_cos`, `gwl`, `monsoon`) have the largest one-at-a-time swing (up to ~0.39 m).
- **Residuals**: mild positive spatial autocorrelation (Moran's I 0.042, p≈0.051); strong temporal autocorrelation is the binding constraint on sample size.

Expected behavior & limitations

- Forecasts by **both** engines are **near-flat** at 30 days for most stations; real moves are rare and only meaningful when |Δ| exceeds the calibrated 90% interval width. The trajectory v2 forecast is calibrated to **90% coverage at every horizon**.
- Stations with missing recent drivers or a spike-flagged last reading produce unreliable forecasts (NaN anchor → delta saturation); Fleet gates these as `unreliable` (43/545).
- Static soil/LULC are fetched and shown for context but deliberately **excluded** (proven redundant with station/district ordinals: adding them raised RMSE and collapsed early stopping).
- Trajectory v2 relies on **forward driver forecasts** (Open-Meteo days 1–16, climatology beyond, CWC river where covered); a missing driver **downgrades confidence** at those horizons (`refresh/future_drivers.py`).
- Production surfaces refresh on a **6-hourly daemon grid** (`refresh/`); the verdict (`data_status`, freshness) and realised-score verification keep the forecasts honest.
- Re-running: any change to feature engineering or training data requires `06_train` → `07_evaluate` → validation suite → regenerate quantile/fleet/diagnostics, and re-run `30_traj_datasets` → `31_train_traj` → `32_backtest_traj` → `33_traj_reliability` for trajectory v2.

11 Streamlit Dashboard Guide

AQUIS dashboard — Streamlit component guide

Every Streamlit widget in the AQUIS groundwater dashboard, explained at the same depth as the Correlation page below. The app is a multi-page Streamlit app under `ml/`; every page is a plain script that runs top-to-bottom on each rerun.

Live navigation (4 pages): Assistant (default) · Correlation · Forecast · Sources — declared in `app.py` via `st.navigation`. The **Verification** page (`app_pages/verification.py`) reads the refresh pipeline's realisations and is **not wired into the nav** (add an `st.Page` to enable it). The legacy explorer pages (Overview, Drivers, Stations, Model, Fleet) were **removed** — the fleet/model/correlate outputs they displayed are still produced by the numbered pipeline (`11_fleet.py`, `07_evaluate.py`, `05_correlate.py`, `10_ablate.py`, `12_diagnostics.py`) and covered below via the live pages.

Run it: `cd ml && streamlit run app.py` (opinionated dark theme is in `ml/.streamlit/config.toml`).

Convention used throughout: `GWL` (groundwater level) is a water-table level in *metres* — a rising level means the value goes **up**. "Rising" name = value up, "declining" = value down.

Table of contents

---

1. Application entry — `ml/app.py`

```
st.set_page_config(page_title="AQUIS — groundwater explorer",
                   page_icon=":material/analytics:",
                   layout="wide")
```

- **`st.set_page_config`** — global page options. `layout="wide"` removes the narrow centered column and uses the full browser width. `page_icon` uses a **Material Symbols** name (`:material/analytics:`) — Streamlit renders these without shipping an image asset. This must be the *first* Streamlit command in a script.

```
page = st.navigation([...])   # list of st.Page definitions
page.run()
```

- **`st.navigation`** — declares the app's pages in one place (the modern replacement for `st.Page`-based multipage apps). Each item is an **`st.Page("app_pages/<file>.py", title="…", icon=":material/…:")`**. Order in this list = order in the sidebar. `page.run()` renders the currently selected page.

#	Page file	Title	Icon	In nav
1	`app_pages/assistant.py`	Assistant	`:material/smart_toy:`	✅ (default)
2	`app_pages/correlation.py`	Correlation	`:material/query_stats:`	✅
3	`app_pages/forecast.py`	Forecast	`:material/troubleshoot:`	✅
4	`app_pages/data_sources.py`	Sources	`:material/database:`	✅
—	`app_pages/verification.py`	Verification	`:material/verified:`	off

2. Theme — `ml/.streamlit/config.toml`

```
[theme]
primaryColor = "#4ecca3"             # teal — buttons, focus, chart accents
backgroundColor = "#0a0a0a"          # nearly-black page background
secondaryBackgroundColor = "#1a1a2e" # cards / alternate panels
textColor = "#e0e0e0"
font = "monospace"

[server]
headless = true

[browser]
gatherUsageStats = false
```

What the theme gives you: every widget picks up `primaryColor` for its active state, `secondaryBackgroundColor` for bordered containers and inputs, and the monospace font everywhere. The charts in `app_charts.py` and `snapshot.py` reuse these exact hex values so inline charts, Altair charts and the exported PNG all look the same.

---

3. Page-by-page, widget-by-widget

Each widget is documented as: **what it is**, **its options**, **what changes when you use it**, and **which backend data/function it reads**. Repeated helper patterns (e.g. `st.caption` explaining a number) are described once per page to avoid noise.

3.1 Correlation — `app_pages/correlation.py`

This is the depth template used for every other page.

Goal: which environmental drivers move groundwater, measured per-station and aggregated (median) across stations.

Element	Code	What it is / does
`st.title` / `st.caption`	"Correlation report"	Heading + a one-line caption summarising the three modes.
`st.segmented_control`	`st.segmented_control("Correlation mode", ["raw", "deseason", "diff"], default="raw")`	**Segmented button row** — a single-select control (modern toggle instead of dropdown). Choosing a mode re-filters the whole page below.
`st.stop`	`if not mode: st.stop()`	**Guard**: if the user *clears* the segmented control (possible — clicking an option again deselects it) `mode` is `None`; `st.stop()` halts the script so the rest of the page doesn't render with a broken filter.

What each mode actually means (backend in `ml/05_correlate.py`):

- **`raw`** — full per-station history correlation of `driver` vs `gwl` (Spearman or Pearson). Requires ≥ 20 jointly-observed days per station; else that station's correlation is `NaN`. Stores the **median across stations** and how many stations were valid (`stations_ok`). Mixed signal: contains both the station's absolute base level and its seasonal cycle.
- **`deseason`** — each series is turned into an **anomaly**: subtract the per-`(station, day-of-year)` mean from every day (`deseason()` in `05_correlate.py`). Only computed for *level* columns. Isolates the **non-seasonal** co-movement, so a rain spike → GWL dip is visible, while both series' long seasonal envelopes cancel out. Interpretation: "when the *unusual* part of rainfall moves, does the *unusual* part of the level move?"
- **`diff`** — **first-difference** of both series (day-to-day change) before Spearman. Only for level columns and `rain_*` windows. **Datum-free**: it can never be dominated by one station sitting 5 m deeper than another, because absolute level is removed. Interpretation: "does today's increment in the driver predict the increment in the level on the same/aligned day?"

Element	Code	What it is / does
`st.segmented_control`	`st.segmented_control("Metric", ["spearman", "pearson"], default="spearman")`	Second axis of the filter: **rank**-based (`spearman` , robust to outliers / non-linear monotonic) vs **linear** (`pearson` , sensitive to scale of movement). Note the report only computes *both* for `raw` ; the other modes are Spearman-only.
`st.stop`	`if not metric: st.stop()`	Same deselection guard.
`st.columns`	`col1, col2 = st.columns([2, 3])`	Splits the row **2:3** — narrower table, wider chart. Children are drawn with `with col1: …` .
`st.container(border=True)`	`with col1: with st.container(border=True):`	**Bordered box** — visually groups the mini-table as a card on the dark background.
`st.markdown`	`st.markdown(f"**{metric} · {mode}**")`	Lightweight text (supports `**bold**` and inline Markdown).
`st.dataframe`	`st.dataframe(sub[[...]].rename(...), hide_index=True)`	**Interactive table**. `hide_index=True` drops the row-number column (the driver name becomes the key of each row). Sorting by correlation descending.
`st.bar_chart`	`st.bar_chart(chart, x="label", y="corr", color="#4ecca3")`	Bar ranking of driver correlations; labels come from `_utils.nice()` (human driver names like "Rain, 7 days").
`st.header`	"Recharge lag — GWL vs rainfall"	Second section.
`st.line_chart`	`st.line_chart(lag, x="lag", y="pearson_med")`	Line over `outputs/lag_curves.csv` . Each point = median (across stations) Pearson of `rain (shifted by lag days)` vs `gwl` ; **the peak marks when rainfall historically shows up in the water level** (the recharge lag).
`st.success`	`st.success(f"Best rainfall recharge lag: **{…} days** …")`	**Green status callout**. Here it summarises the *argmax* |*r*| row (`lag.iloc[lag["pearson_med"].abs().idxmax()]`) into a single headline number.
`st.expander`	`with st.expander("Full report table", icon=":material/table_chart:"):`	**Collapsed section** — content hidden until toggled (keeps the page short). `icon=` adds a Material icon to the toggle row.
`st.dataframe`	inside expander, `height=420`	The *entire* report (driver × metric × mode), fixed 420 px height → its rows scroll inside the table.
`st.dataframe` / `st.altair_chart`	"Static features vs station groundwater" section	Soil-texture block: merges per-station GWL summary (`mean/std/median`) with ISRIC soil features, then Spearman of each soil fraction vs mean GWL and vs GWL spread. Left: `st.dataframe` of the ρ values; right: `st.altair_chart` bar of ρ vs mean GWL (teal). `st.caption` caveat: **exploratory only — soil is NOT fed to the model**.
same for LULC	"LULC class share vs district GWL"	Reads `outputs/correlation_lulc.csv` (28 districts). Left table = Pearson r per class; right chart = top-8 classes (rose `#e67676`). Same "not fed to the model" caption.

3.2 Forecast — `app_pages/forecast.py` (trajectory v2)

Goal: one clean dark card presenting the **120 genuine 6-hour forecast points** of the multi-horizon trajectory engine. Observed history stops at the anchor; the forecast starts 6 h later and never re-predicts the anchor value. This is the *only* forecast shown here.

Element	Code	What it is / does
`st.set_page_config` / `st.title` / `st.caption`	heading block	icon `:material/troubleshoot:` ; caption: "120 genuine 6-hour forecast points · multi-horizon trajectory model anchored on the latest observed reading".
`st.markdown(…, unsafe_allow_html=True)`	**CSS injection**	Tiny `<style>` block (bordered `stVerticalBlockBorderWrapper` , `.aquis-legend` , `.aquis-dir`). `unsafe_allow_html` is the deliberate escape hatch — the string is developer-authored, not user input.
`st.sidebar.selectbox` ×2	`sidebar_station_picker(table6, stations)` (`_utils.py`)	**Shared sidebar pickers** (keys `sb_district` / `sb_station` , same on every page): District dropdown, then Station dropdown filtered to that district (recency-first, preds-backed). Selection persists across pages — Assistant follows along. Page shows a `**{station}** · {district}` caption instead of its own picker.
`st.caption`	"Latest reading: **…** — the forecast anchor"	Reads `station_recency()` for the anchor timestamp.
`st.caption` ×2	refresh-pipeline freshness	`refresh.publish.freshness_for(station)` : 🟢/🟠/⚪ data status, forecast-generated time, model version, anchor, staleness reason. Wrapped in `try/except` — **decorative, never breaks the page**.
`st.container(border=True)`	the whole trajectory card	Single bordered container holds everything below (the rounded-corner card).
`st.markdown` + `st.caption`	header	`"#### 30-day groundwater outlook"` + caption (120 points · anchor ↦ forecast range).
`st.markdown`	manual **compact legend** HTML	Recreates the chart legend by hand (observed line, q50 line, bright 90% band, q05·q95 edges) because the Altair chart hides its official legend for a cleaner card.
`st.altair_chart`	`trajectory_chart(pts, tail, anchor_t, height=480)` , `width="stretch"`	The card's centerpiece (`app_charts.py`): observed tail (grey, dashed) → anchor rule **"Forecast starts"** → 120-point q50 line (teal, wide) with a **bright q05–q95 band** (`COL_BAND` , 30% opacity) + bright edge lines (`COL_QEDGE`), continuous lines — **no dot markers**. X-domain = anchor−15 d → +30 d.
`st.markdown`	direction banner	Big ↑ / ↓ / → glyph + "Rising/Falling/Stable" + q50 change over 30 d (+ sign-accuracy when available), coloured per `DIR_GLYPH` .
`st.columns(6)` + `st.metric` ×6	metric strip	**Anchor GWL (observed)** (never predicted), **+24 h**, **+7 d**, **+30 d** (the genuine 120th point, delta = change), **30 d change** (q50 at +30 d − anchor, delta = sign-accuracy), **Confidence** (label from `CONF_STYLE` ; tooltip = reason + guidance).
`st.warning` ×2	trajectory failure paths	"Trajectory engine unavailable: `{e}`" or the backend's `error` string — the page degrades to a warning instead of crashing.

Caching (visible behaviour): `_trajectory_cached(station, version)` caches the trajectory run keyed by a content fingerprint (`_traj_version()` hashes model files, calibration/reliability tables, the aligned 6h table and today's date), so the forecast refreshes exactly when data/weights change or daily.

3.3 Assistant — `app_pages/assistant.py`

Goal: station-locked natural-language Q&A. Facts are computed deterministically from `table_6h` + the 30-day model; `llama3.2:3b` via Ollama (local) only phrases them. The instant data panel works even without Ollama.

Element	Code	What it is / does
`st.set_page_config` / `st.title`	heading	icon `:material/smart_toy:`.
`st.warning`	"Ollama server not reachable…"	shown only when the server is down; stresses the instant panel still works.
`st.info` + `st.code`	"Ollama reachable, but model not pulled" + `st.code(f"ollama pull {model_name}", language="bash")`	Blue info box + a **copyable code block** (bash).
`st.expander` + `st.json` + `st.caption`	"Model status"	Full status dict as JSON + "press R to re-check" note.
`st.sidebar.selectbox` ×2	`sidebar_station_picker(df, stations)` (`_utils.py`)	**Shared sidebar pickers** — same `sb_district` / `sb_station` keys as Forecast, so both pages stay on one station. District dropdown + district-filtered Station dropdown (recency-first).
`st.columns(5)` + `st.metric` ×5	fact KPIs	Latest level (delta = date), 30-day change, 7-day change, Obs. range (min–max), Samples (n_obs).
`st.caption`	district context line	District median across analysed stations.
`st.success` / `st.warning` / `st.info`	30-day outlook status	Green success with `<b>30-day outlook (XGBoost)</b>` (value + band + anchor); yellow warning if the forecast is unstable; blue info if no data for the selection.
`st.columns(2)` + `st.expander` ×2	two fact panels	Left: **"Factors driving this station"** — `st.dataframe` of per-factor Spearman r / p / pairs with `column_config={"Spearman r": st.column_config.NumberColumn(format="%+.3f"), "p-value": …("%.4f")}` (custom number formatting per column), plus markdown for recent rain and an annual-table dataframe. Right: **"Precautions & strategy"** — one colour-coded markdown block per advisory (🔵/🟡/🔴 + material icon), separated by `st.divider()`, with a caption giving example prompts.
`st.markdown("---")`	horizontal rule	Divides the deterministic panels from the chat.
`st.session_state`	station pinning	`st.session_state.get("as_pinned_station")` / `as_messages` — **widget-independent state** that survives reruns. When the station changes, it seeds a fresh conversation from `as_messages` and writes `as_pinned_station`.
`st.chat_message` + `st.markdown`	past-messages loop	`with st.chat_message(msg["role"]): st.markdown(msg["content"])` renders each prior turn as a chat bubble (avatar + styling differ for user/assistant).
`st.chat_input`	`st.chat_input(f"Ask about {station}")`	**Chat input box** anchored at the bottom of the page. Truthy prompt → append user message → render it → render assistant bubble.
`st.spinner`	`with st.spinner("Querying AQUIS data + generating answer…")`	Shows a temporary spinner while `StationAssistant.answer(prompt, station, history)` runs (last 6 messages as context). Wrapped in `try/except` → friendly warning instead of a traceback.
`st.expander` + `st.markdown`	"About this assistant"	Stack/method notes: LangChain + Ollama local, model override via `AQUIS_OLLAMA_MODEL`, data/forecast/factor/precaution sources, scope, and the "GWL is a level in metres" convention.

Design note: the LLM only *phrases* pre-computed facts — no hallucinations of numbers; the deterministic panels above the chat remain the source of truth.

3.4 Sources — `app_pages/data_sources.py`

Goal: per-source quality summary for the selected 29-district / 600-station set.

Element	Code	What it is / does
`st.title` / `st.caption`	"Data sources"	Heading + scope note.
`st.columns(2)` + `st.container(border=True)`	source cards	Each source with rows>0 becomes a bordered card in a 2-column grid (cards alternate columns). Inside: `st.markdown` with the human name **plus a markdown hyperlink to the NWDP dataset** (`[NWDP dataset ↗](url)`), a bordered `st.metric("Rows", …)` , and a caption with stations · districts · span · cap-dropped count. A second caption marks Gate-1-only discharge series with an `:orange-badge[discharge]` pill.
`st.subheader` + `st.caption`	"Sources with no data in selected districts"	A single caption joining markdown links for empty sources, minus their explanation (canal discharge/barrage gauges lie outside the 29 districts).
`st.header`	"Static hydrogeology"	Two-col section: **Soil texture** card (metric "Stations mapped", ISPIC/ SoilGrids links, feature description) and **Groundwater extraction** card (metric "Assessments", CGWB link). Missing-data states show `:orange-badge[not loaded]` captions.
`st.header`	"Land use / land cover (static, ISRO Bhuvan)"	3-col row: `st.metric("Districts")` + cycles caption
`st.header` + `st.write`	"Association method"	`st.write` — plain paragraph (IDW 1/d² over 3 nearest gauges; nearest-gauge joins; district proxy for river).
`st.expander` + `st.dataframe`	"Association link counts"	Counts well↔gauge links per driver from `meta/assoc_*.csv` .
`st.caption`	"All values from `ml/data/` . Nothing here is committed or production."	Footer honesty note.

3.5 Verification — `app_pages/verification.py` (not in nav)

Goal: realised-forecast quality. Reads `refresh/verification.py` 's `data/refresh/verification_summary.json` + the sign-accuracy ledger — archived forecasts scored against readings that have since materialised.

Element	Code	What it is / does
`st.title` / `st.caption`	"Forecast verification"	Heading + description of archive/realised scoring.
`st.info`	no-summary guard	"No verification summary yet — it appears after forecast cycles…"
`st.columns` + `st.metric` ×5	KPI strip	`Ledger rows` , 24 h / 7 d / 30 d `Sign accuracy` (with `delta`), RMSE, coverage vs 90% target — each `border=True` .
`st.subheader` + `st.dataframe`	"Windows (rolling)"	Per-window RMSE/sign-accuracy rows from `windows` ; caption when nothing is scored yet.
`st.subheader` + `st.dataframe`	"Ledger sample"	Latest 200 ledger rows (`target_time` desc), `width="stretch"` , `hide_index=True` .

Not wired into `app.py` nav — add an `st.Page("app_pages/verification.py", …)` to enable it.

---

4. Cross-page component reference

Every distinct Streamlit element used in the app, where it appears, and what it does — the quick lookup table.

Component	Where used	Purpose
`st.set_page_config`	app entry + Forecast / Assistant pages	Global options (title, material icon, wide layout).
`st.navigation` + `st.Page`	`app.py`	Declares the sidebar structure (4 active pages + Verification when wired); `page.run()` renders the active page.
`st.title`	every page	Page heading.
`st.header`	Correlation, Sources	Section heading (level 2).
`st.subheader`	Verification	Block heading (level 3).
`st.caption`	everywhere	Small muted annotation, empty states, methodology notes.
`st.markdown`	everywhere	Inline/brief rich text; also `unsafe_allow_html=True` for CSS + custom cards.
`st.write`	Sources	Plain paragraphs / data-frames equivalent; safe fallback text.
`st.code`	Assistant	Copyable bash block (`ollama pull …`).
`st.divider`	Assistant	Horizontal separator between advisories.
`st.json`	Assistant	Syntax-coloured JSON detail viewer.
`st.metric`	Correlation, Forecast, Assistant, Sources, Verification	KPI card (label, value, optional delta line, `border=True` card).
`st.selectbox`	Forecast, Assistant	Single-choice dropdown.
`st.multiselect`	— (removed with the Drivers/Fleet pages)	Multi-choice dropdown pattern; retained here for reference.
`st.segmented_control`	Correlation (×2)	Single-select segmented button toggle.
`st.radio`	— (removed with the Model page)	Horizontal choice pattern; retained here for reference.
`st.checkbox`	— (removed with the Fleet page)	Boolean filter toggle pattern; retained here for reference.
`st.chat_input` + `st.chat_message`	Assistant	Conversational input + bubble-styled messages.
`st.sidebar`	— (removed with the Stations page)	Moves its children into the sidebar.
`st.columns`	every page	Responsive vertical split; ratios like `[1,2]` , `[2,3]` , `[3,1]` , `[2,1]` , 3/5/6-way for metric strips.
`st.container(border=True)`	Correlation, Forecast, Sources	Bordered card grouping.
`st.container(horizontal=True)`	— (removed with the Overview/Stations pages)	Forces its children into one horizontal row.
`st.expander`	Correlation, Forecast, Assistant, Sources	Collapsible section (+ optional material `icon=`).
`st.dataframe`	Correlation, Forecast, Assistant, Sources, Verification	Interactive table; `hide_index` , `width` / `height` , `use_container_width` , `column_config` formats.
`st.altair_chart`	Correlation, Forecast, Sources	Full Vega-Lite/Altair chart (the only way to get bands, facets, rules, dashed lines, interactivity).
`st.bar_chart`	Correlation	Quick native bar chart (no Altair needed).
`st.line_chart`	Correlation	Quick native line chart.

Component	Where used	Purpose
`st.download_button`	Forecast	PNG snapshot download.
`st.spinner`	Assistant	Loading indicator around an expensive call.
`st.success` / `st.warning` / `st.info`	Correlation, Forecast, Assistant, Sources, Verification	Status callouts (green / yellow / blue).
`st.stop`	Correlation, Forecast, Assistant, Verification	Early halt guard (missing data / deselected control).
`st.session_state`	Assistant	Cross-rerun state (pinned station + message history).
`st.cache_data`	`_utils`, forecast page	Decorator that caches loader results (TTL, max_entries) and expensive trajectory runs.

---

5. Under the hood

Data loaders (`ml/_utils.py`) — all `@st.cache_data(ttl="10m", …)`

Loader	Reads	Returns
`load_manifest()`	`data/meta/manifest.json`	Per-source QC summary dict (rows, stations, districts, span, dropped).
`load_selected_gwl()`	`data/meta/selected_gwl_stations.csv`	The 29-district / ~600-station GWL selection.
`load_district_set()`	`data/meta/selected_districts.json`	The selected district names.
`load_report()`	`outputs/correlation_report.csv`	The 3-mode correlation report.
`load_lag_curves()`	`outputs/lag_curves.csv`	Rain-lead vs GWL lag curve.
`load_table()`	`data/aligned/table.parquet` (daily)	Daily aligned driver+GWL table.
`load_table_6h()`	`data/aligned/table_6h.parquet` (6-hourly)	The 6-hourly grid the forecast engine uses.
`station_recency()`	deriv. from `table_6h`	Most-recent reading per station (cached on top of `load_table_6h`).
`nice()`, `source_nice()`	`DRIVER_LABELS` / `SOURCE_LABELS`	Machine names → human labels ("rain_7d" → "Rain, 7 days").

Supporting modules

- `ml/app_charts.py` — `trajectory_chart()`, colour constants (`COL_OBSERVED` , `COL_TRAJECTORY` , `COL_BAND` , `COL_QEDGE`) shared by the Forecast page; `snapshot.py` reuses the same palette for its PNG export (kept for the export tests).
- `ml/snapshot.py` — `trajectory_snapshot_png()` : matplotlib (Agg) render of the forecast card; no longer wired to a button on the Forecast page (exercised by tests).
- `ml/_model.py` — loaders behind the live pages (`load_importance` , `load_predictions`) plus the shared forecast path (`forward_forecast` , quantile loaders) and the offline fleet/model loaders (`load_fleet_table` , `load_model_metrics` , `load_honest_metrics` , …) used by the numbered pipeline outputs.
- `ml/_trajectory.py` — `trajectory_forecast(station)` — the multi-horizon engine behind the Forecast card and `/forecast/<slug>` .
- `ml/_assistant.py` — `StationAssistant` (facts + Ollama phrasing), `ollama_status` , station/district name lists.
- `ml/_soil.py` , `ml/_lulc.py` — static ISRIC soil + ISRO Bhuvan LULC loaders.
- `ml/refresh/` — the 6-hourly daemon: `publish.freshness_for()` (nav-level freshness caption), `verification.py` (Verification page), `future_drivers.py` (CWC, Open-Meteo) used by the trajectory engine.

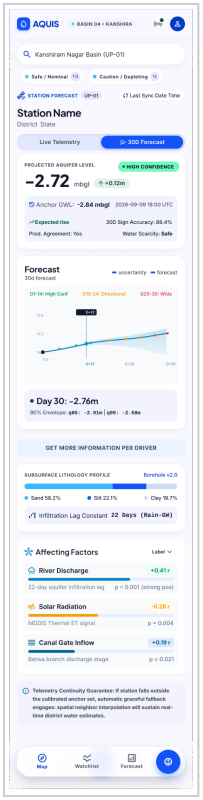
Key outputs read by the app

`ml/outputs/correlation_report.csv` · `lag_curves.csv` · `correlation_lulc.csv` · `predictions_2026.parquet` · `fleet_forecast.csv` (offline scan output, read by `_model.load_fleet_table`) · `ml/models/traj_*.json`.

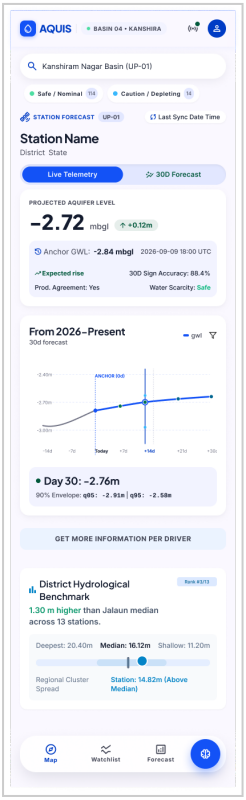
> The dashboards' own conventions: status callouts (`success`=good, `warning`=degrade-but-continue, `info`=no-data hint), `st.stop()` guards all missing-data paths, and every static feature is explicitly captioned as "not fed to the model".

A Appendix — Product UI Reference

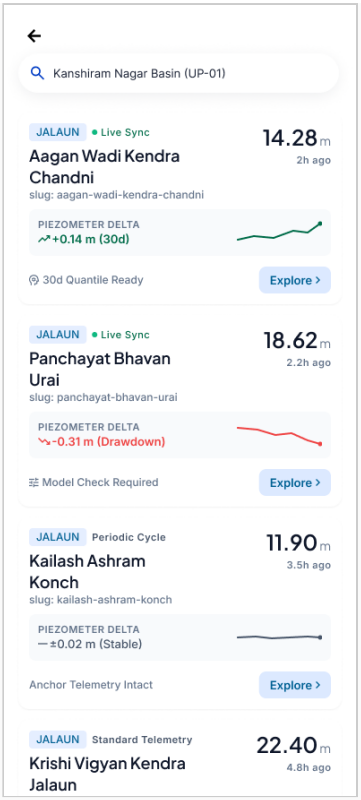
Representative screens from the AQUIS mobile app, referenced for context alongside the specifications above.



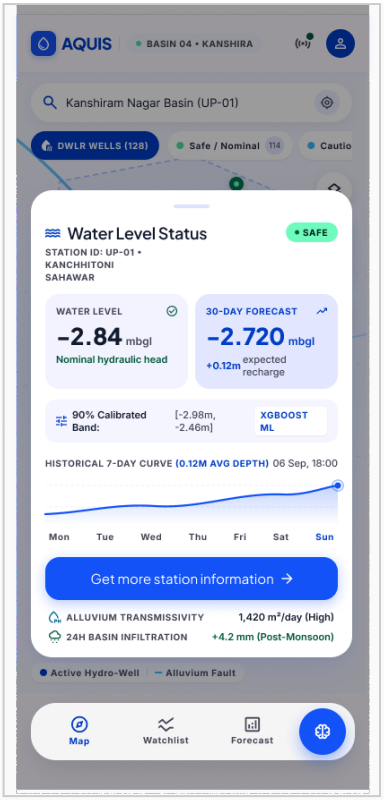
Forecast tab — 30-day trajectory card with confidence band, anchor GWL, affecting factors.



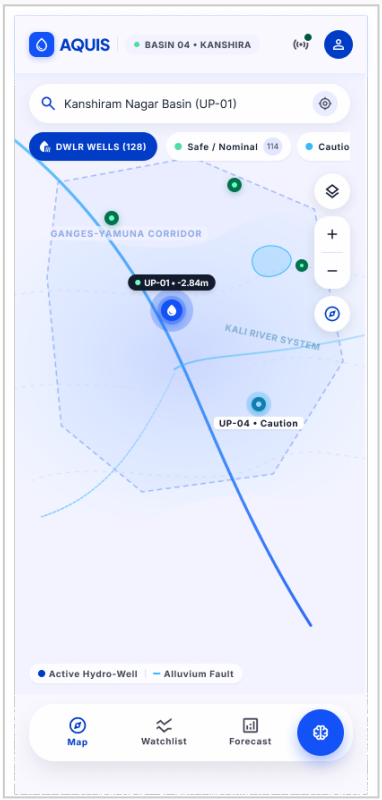
Live Telemetry tab — current reading view with district hydrological benchmark ranking.



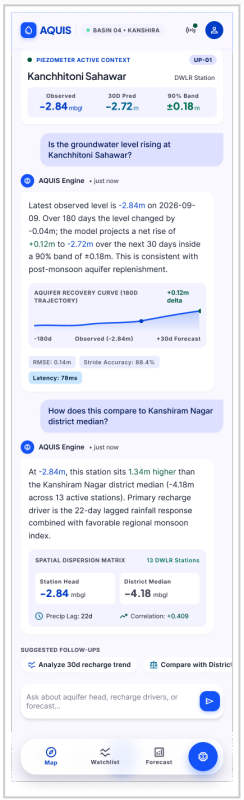
Station search results list — live sync status, piezometer delta, and quantile readiness badges.



Map marker popup — water level status card with 90% calibrated band and basin infiltration stats.

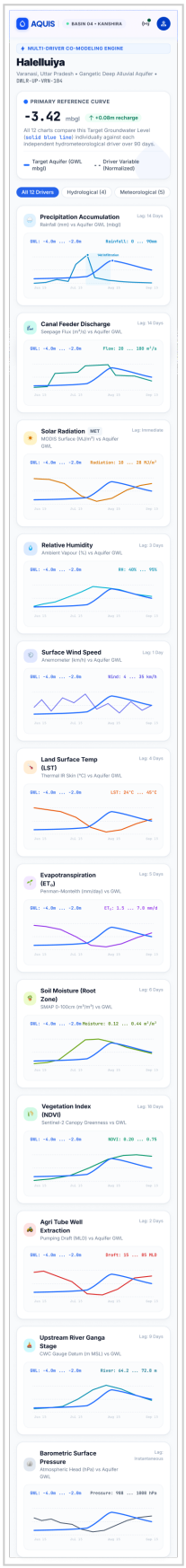


Basin map view — DWLR well markers, safe/caution zones, hydro-fault overlays.



AQUIS Assistant — station-pinned chat with recovery curve, spatial dispersion matrix, and suggested follow-ups.

Multi-driver co-modeling view



Multi-driver co-modeling view — all 12 hydrometeorological drivers plotted against target aquifer GWL.